



RAMAIAH
Institute of Technology

Department of Information Science & Engineering

Term: 03rd Oct 2024 – 25th Jan 2025

Semester: 5

Subject Code: IS53

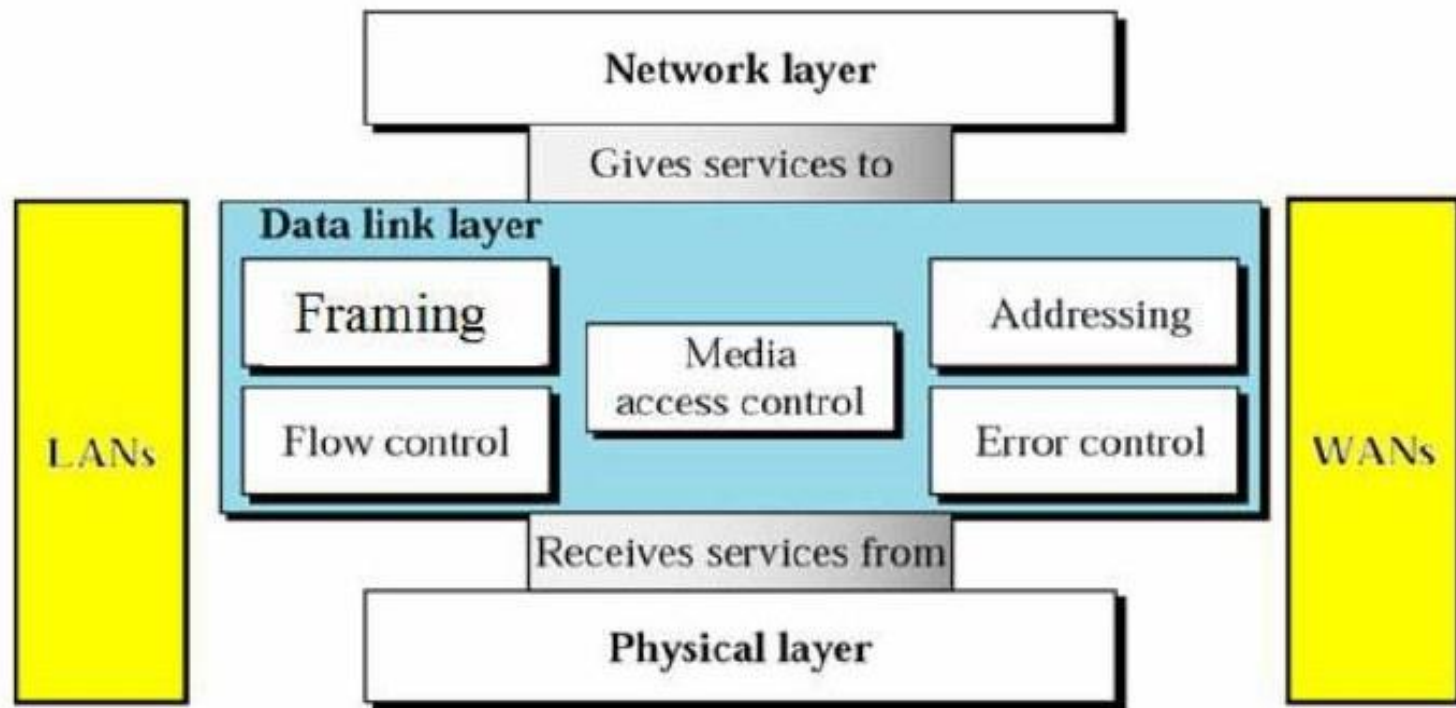
Credits: 4:0:0

Subject Name: Computer Networks

UNIT 2

DATA LINK LAYER

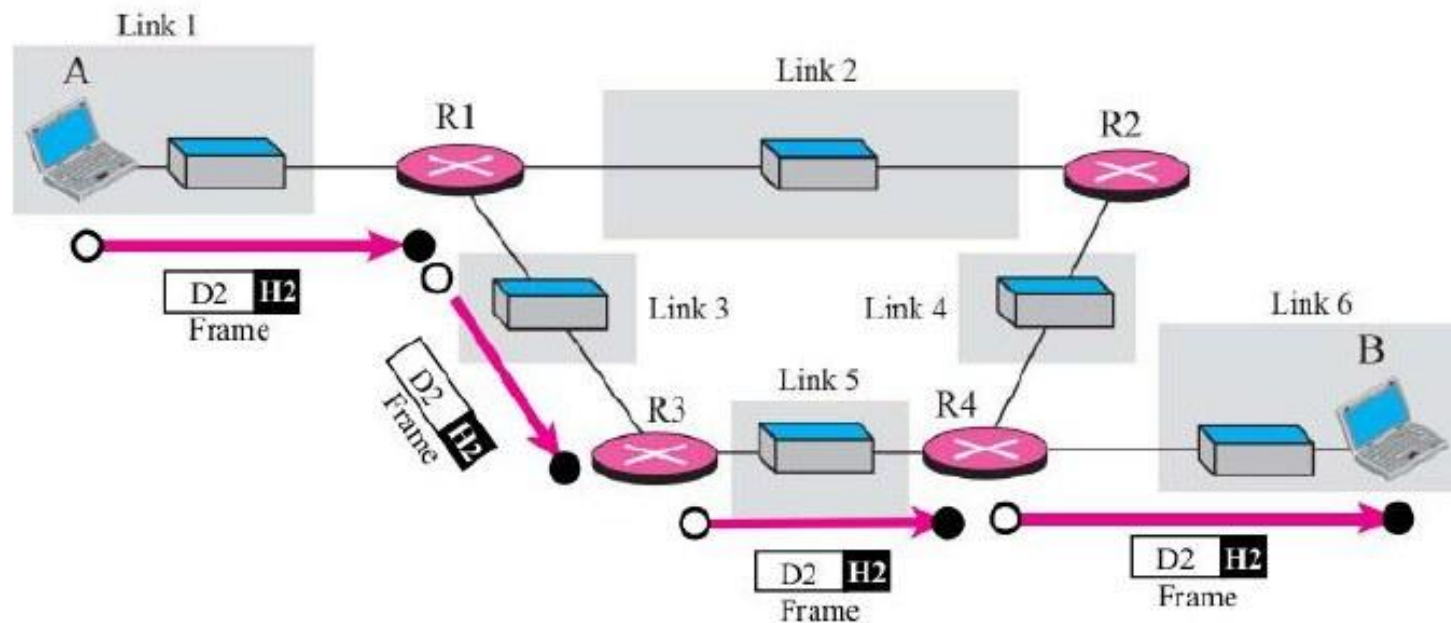
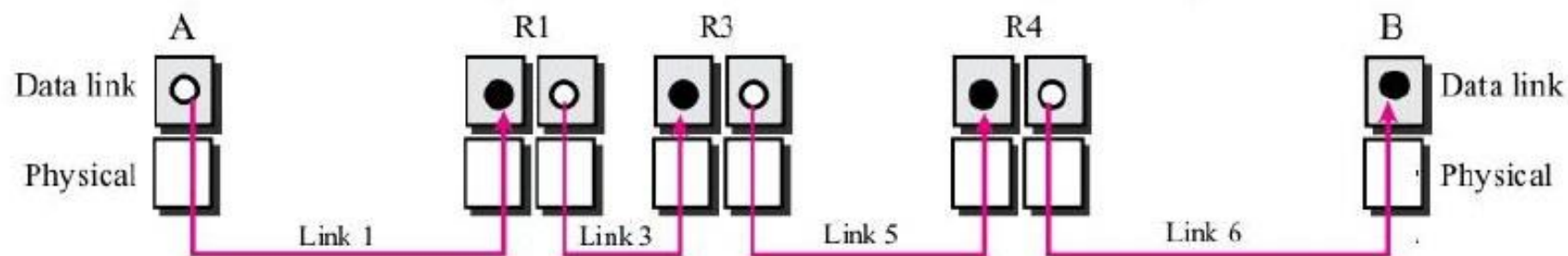
Data Link Layer

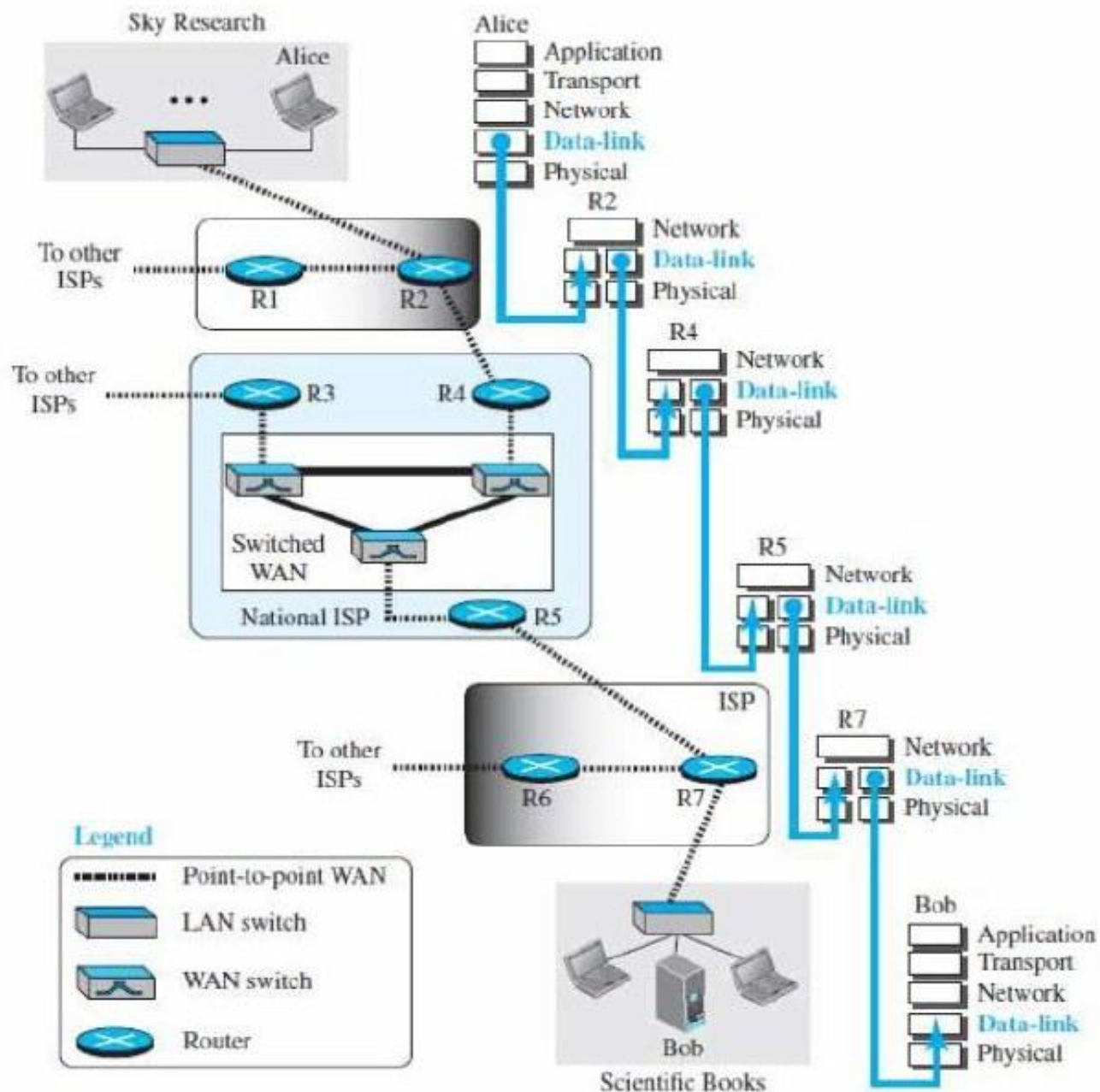


Position of the data-link layer

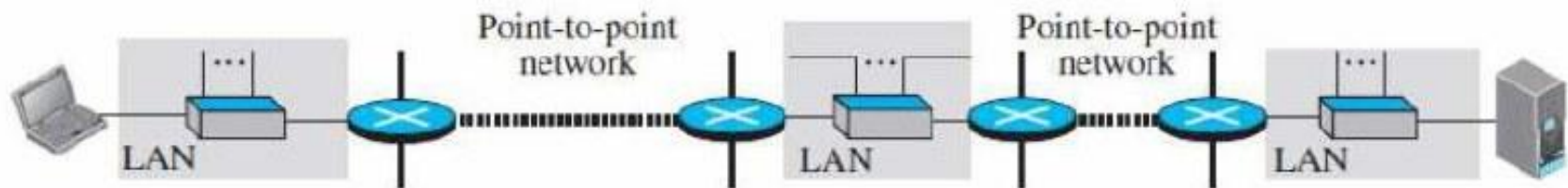
Hop-to-hop delivery

Legend ○ Source ● Destination D Data H Header





Nodes and Links



a. A small part of the Internet



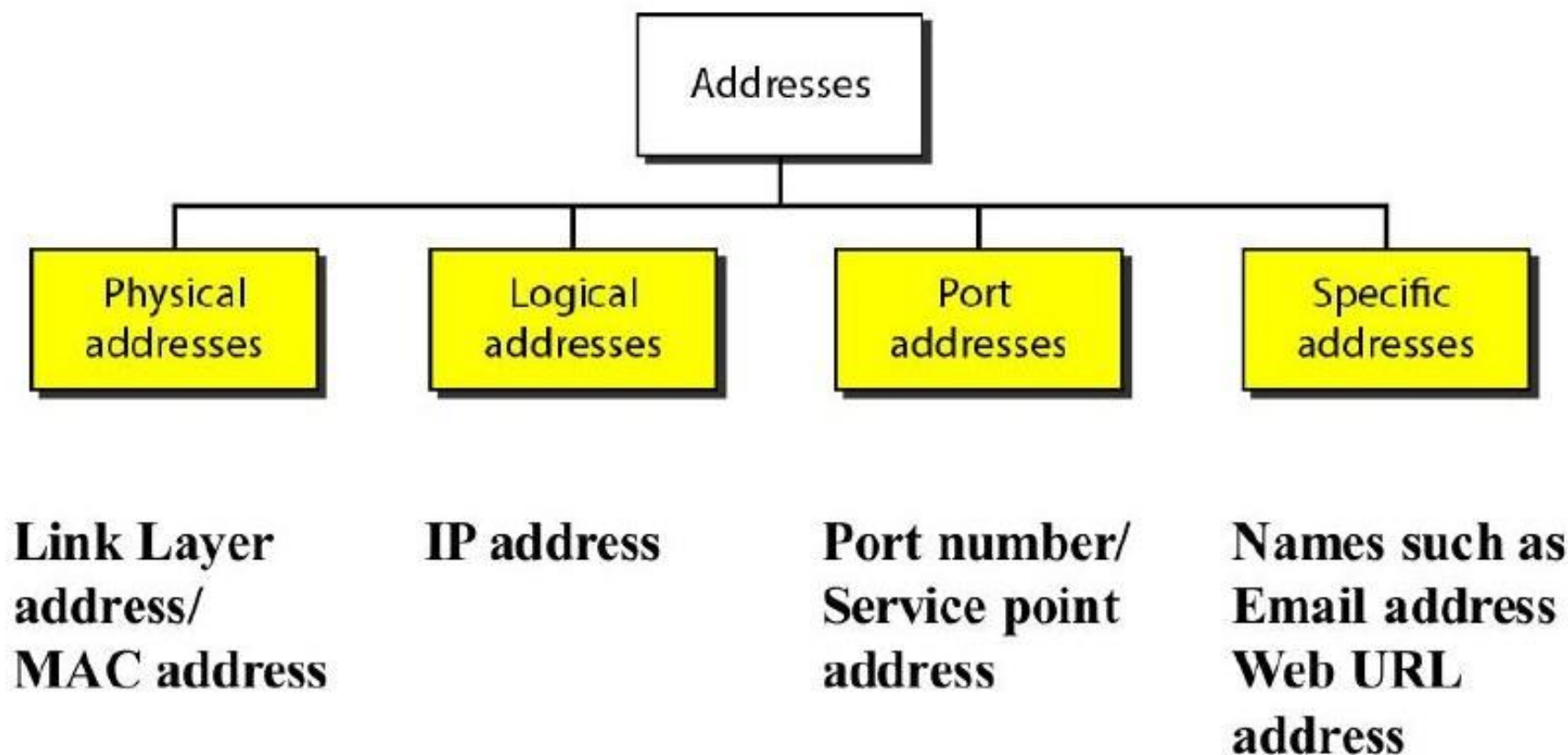
b. Nodes and links

Responsibilities / Functions / Services of Data Link Layer

- Hop to hop delivery
 - Framing
 - Physical addressing
 - Flow control
 - Error control
 - Access control
-

ADDRESSING

- Four levels of addresses are used in an internet employing the TCP/IP protocols
- Each address is related to a one layer in the TCP/IP architecture

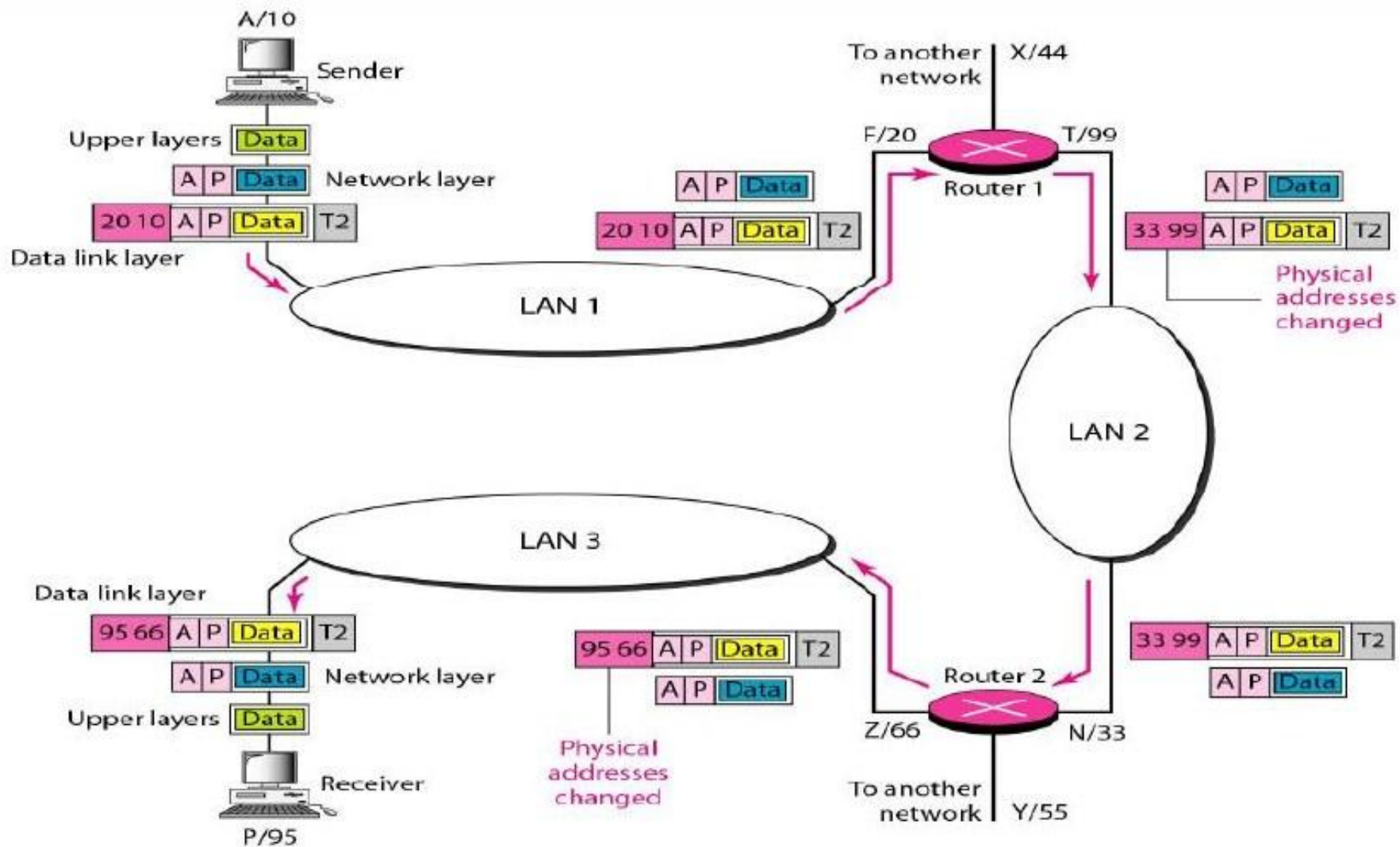


Addresses in TCP/IP – Physical Address

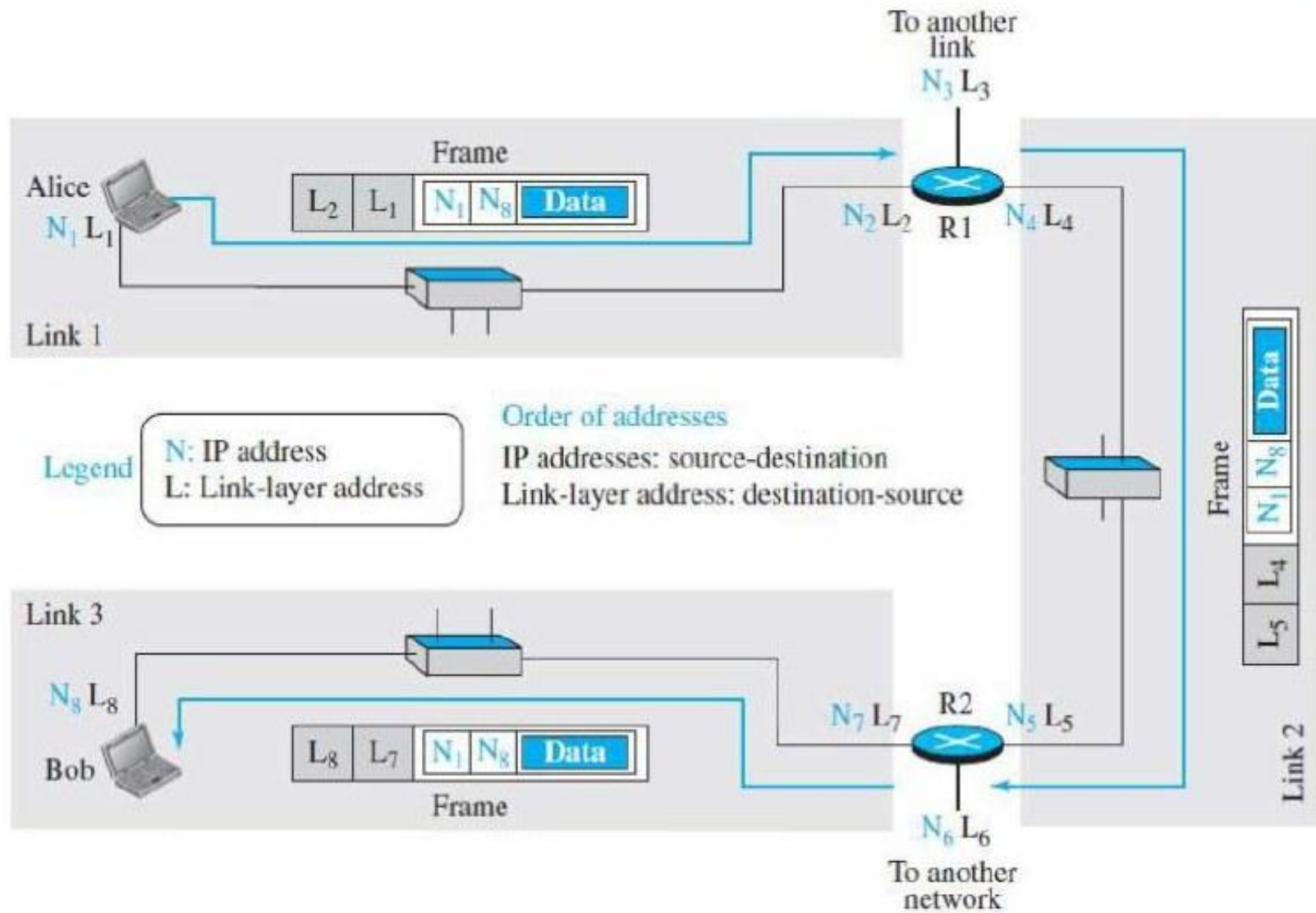
- Physical – 48 bit (6 byte) address
- Defined by LAN or WAN
- Physical address is written as 12 hexadecimal digits
- Each byte (2 hexadecimal digits) is separated by a colon, as shown below:

07:01:02:01:2C:4B
6-byte (12 hexadecimal digits)

Logical Address – Example



IP addresses and link-layer addresses in a small internet



Note

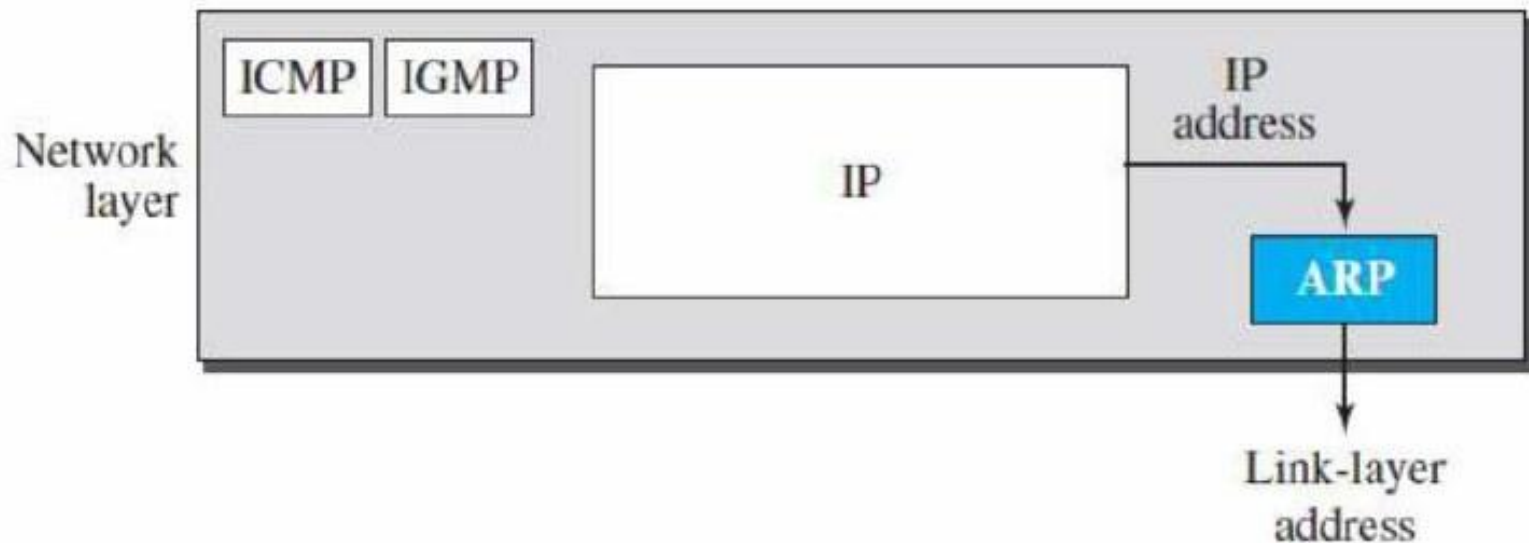
The physical addresses will change from hop to hop; but the logical addresses usually remain the same.

Three Types of addresses

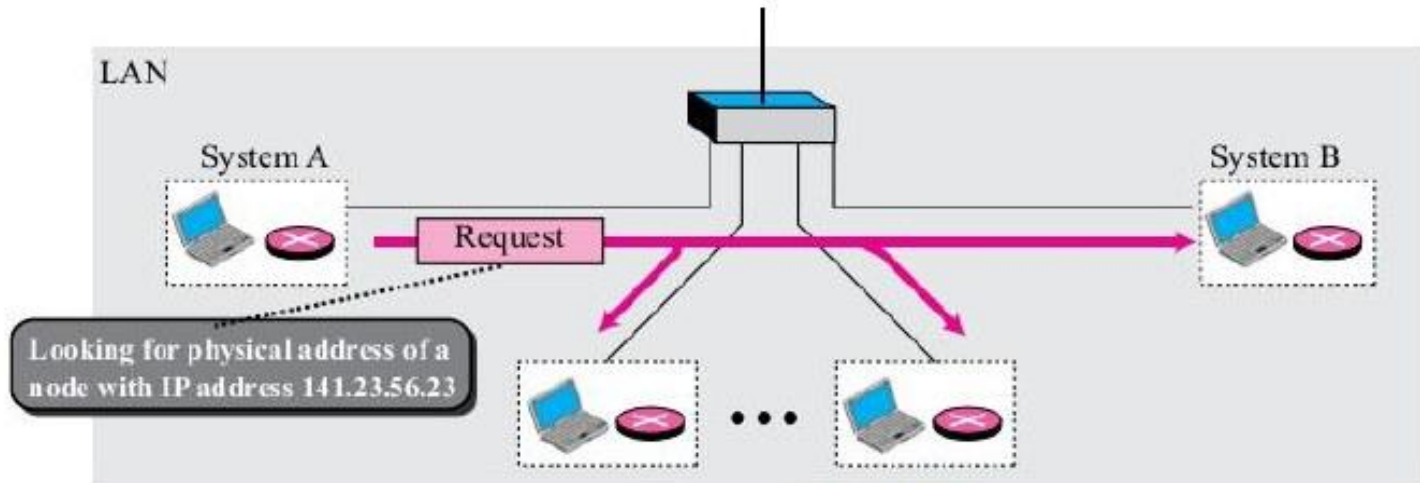
- **Unicast Address**
 - **Multicast Address**
 - **Broadcast Address**
-

ARP

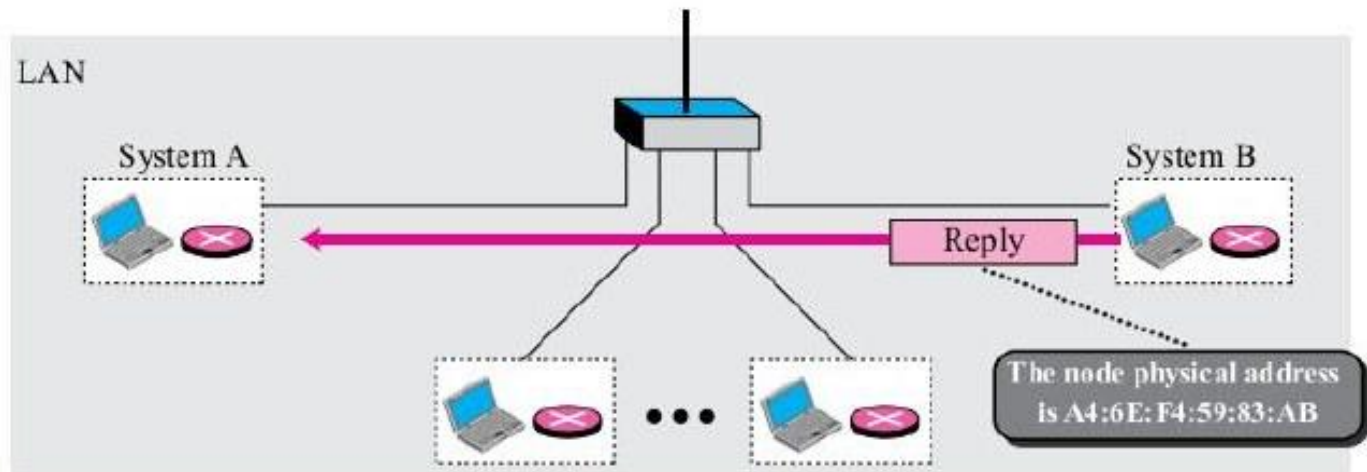
Position of ARP in TCP/IP protocol suite



ARP Operation

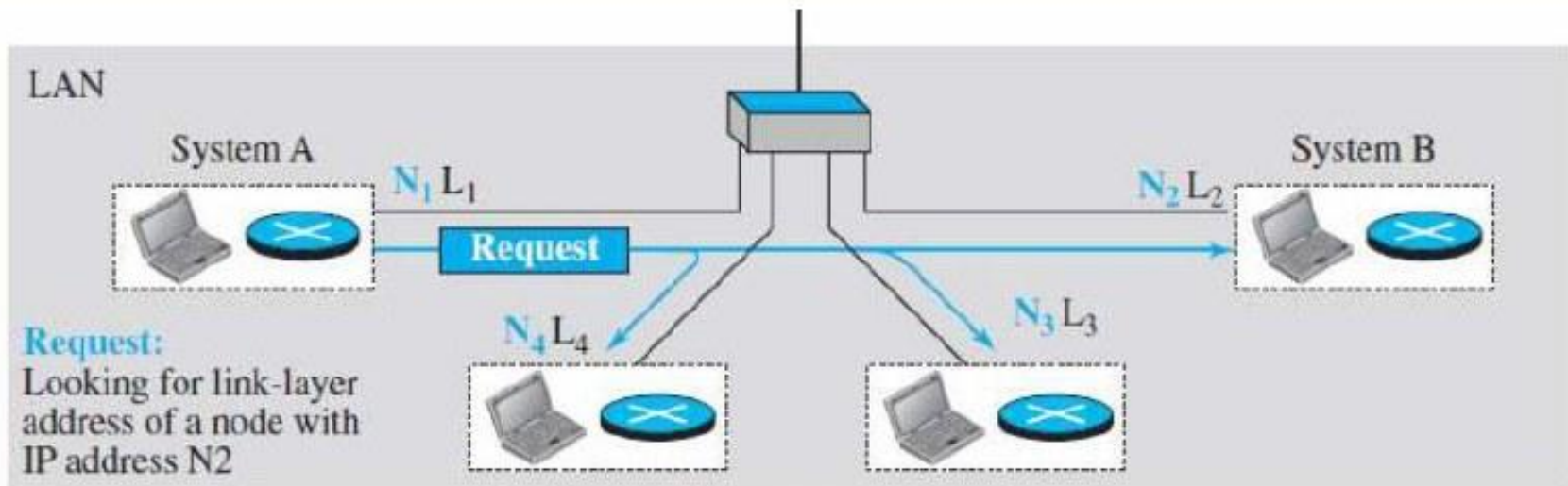


a. ARP request is broadcast

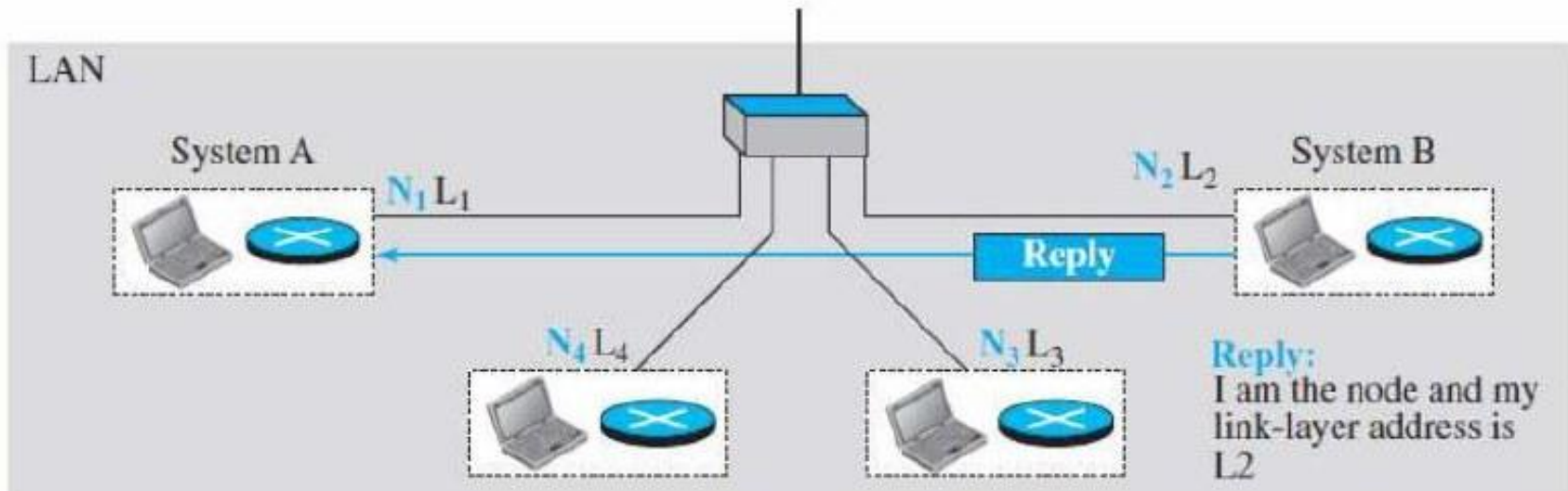


b. ARP reply is unicast

ARP Operation



a. ARP request is broadcast



b. ARP reply is unicast

Note 1

**ARP Request is Broadcast;
ARP Reply is Unicast**

Note 2

**ARP can be useful if the ARP reply is
cached (kept in cache memory for a
while).**

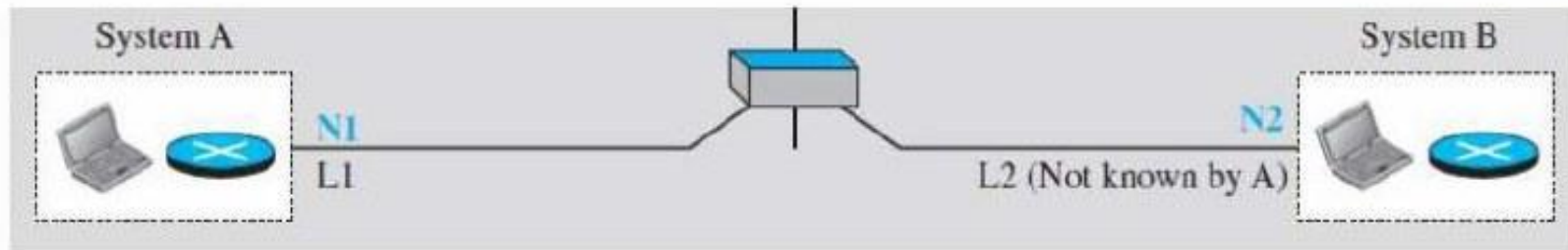
ARP Packet

0	8	16	31
Hardware Type		Protocol Type	
Hardware length	Protocol length	Operation Request:1, Reply:2	
Source hardware address			
Source protocol address			
Destination hardware address (Empty in request)			
Destination protocol address			

Hardware: LAN or WAN protocol

Protocol: Network-layer protocol

An ARP request and reply



ARP request

0x0001		0x0800
0x06	0x04	0x0001
L1 N1 All 0s N2		



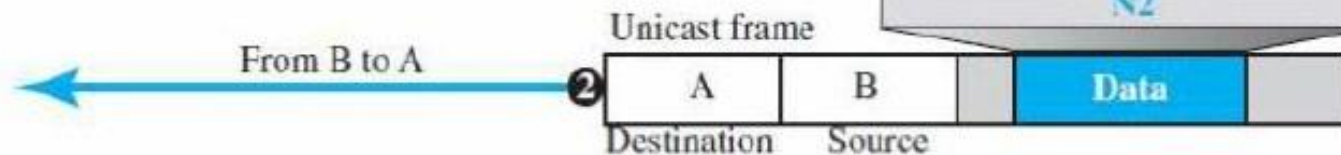
Destination Source.

From A to B

M : Broadcast address

ARP reply

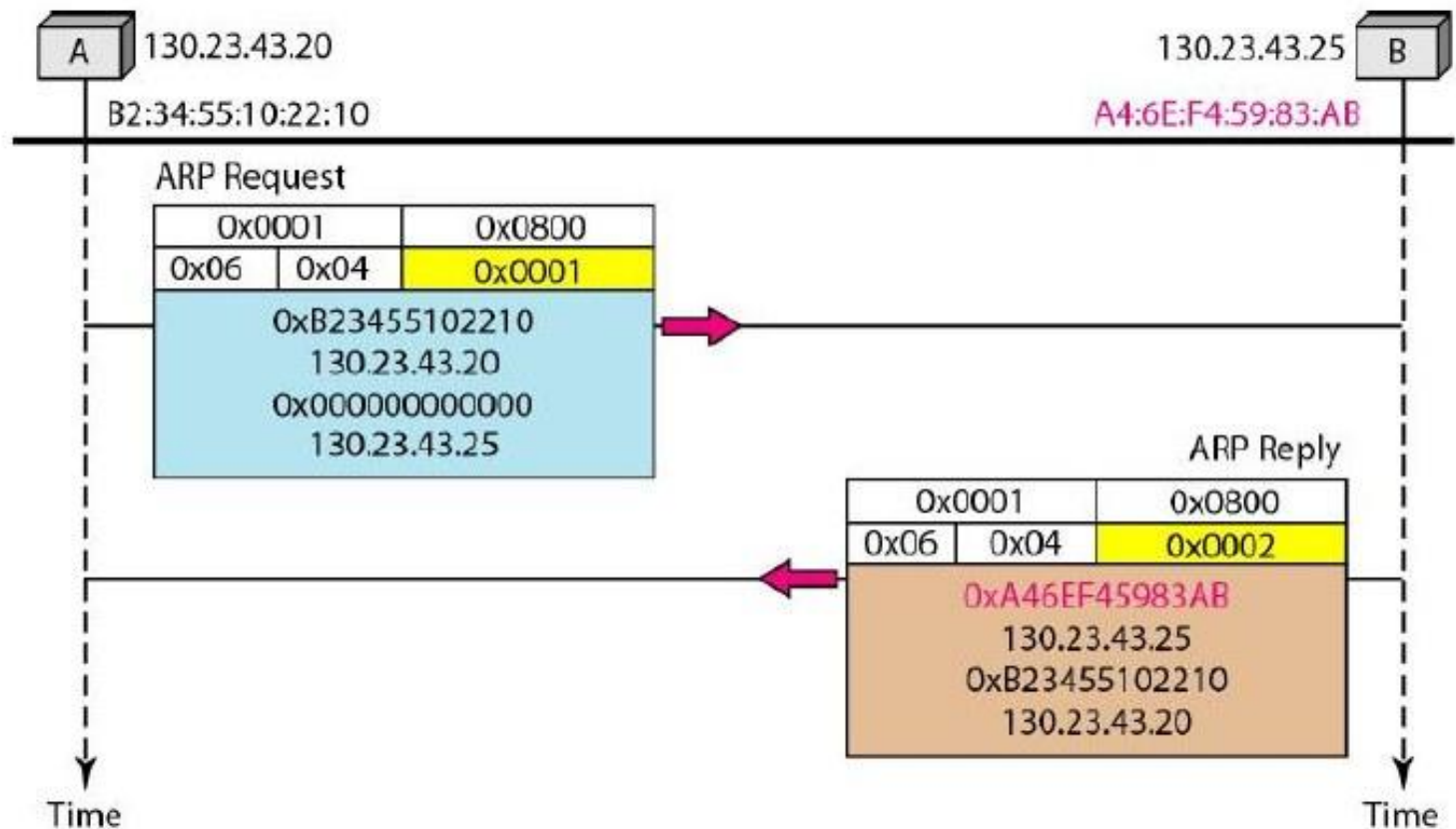
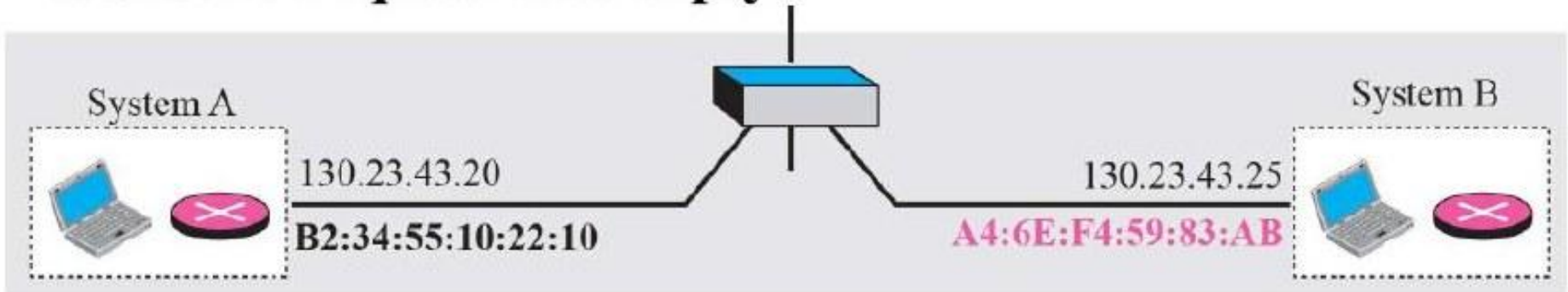
0x0001		0x0800
0x06	0x04	0x0002
L2 N2 L1 N2		



From B to A

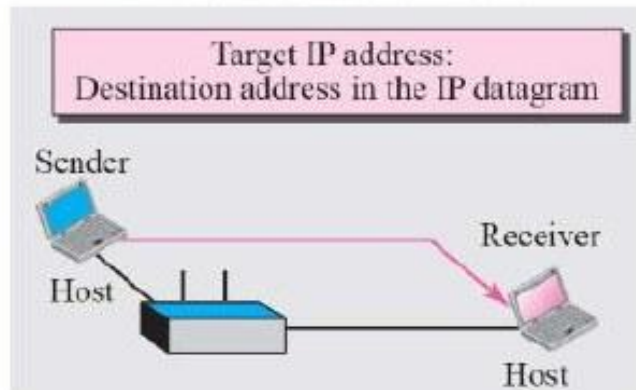
Unicast frame

An ARP request and reply

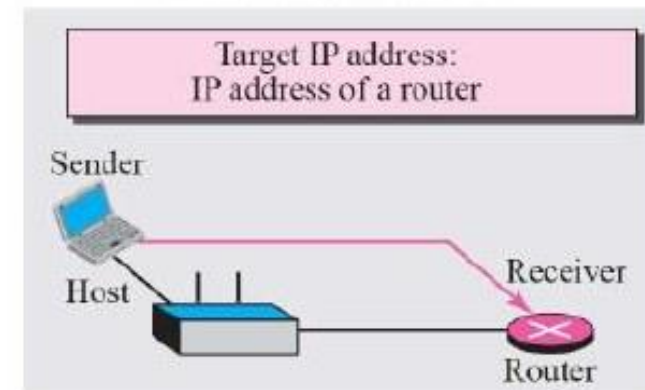


Four cases using ARP

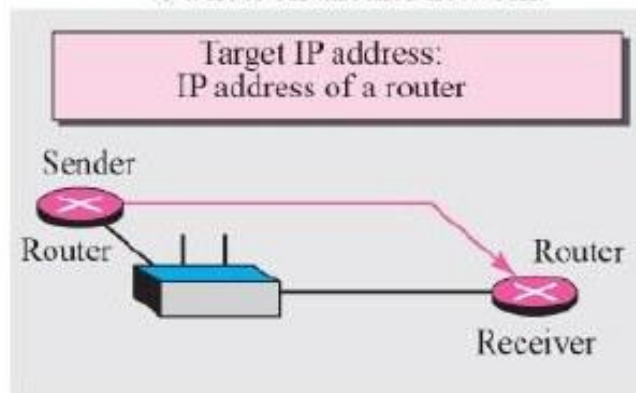
Case 1: A host has a packet to send to a host on the same network.



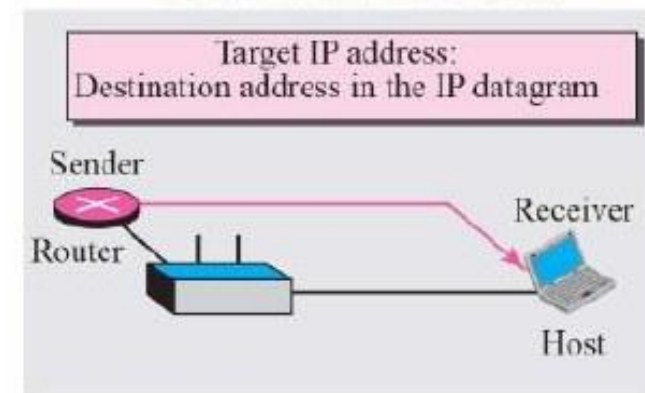
Case 2: A host has a packet to send to a host on another network.



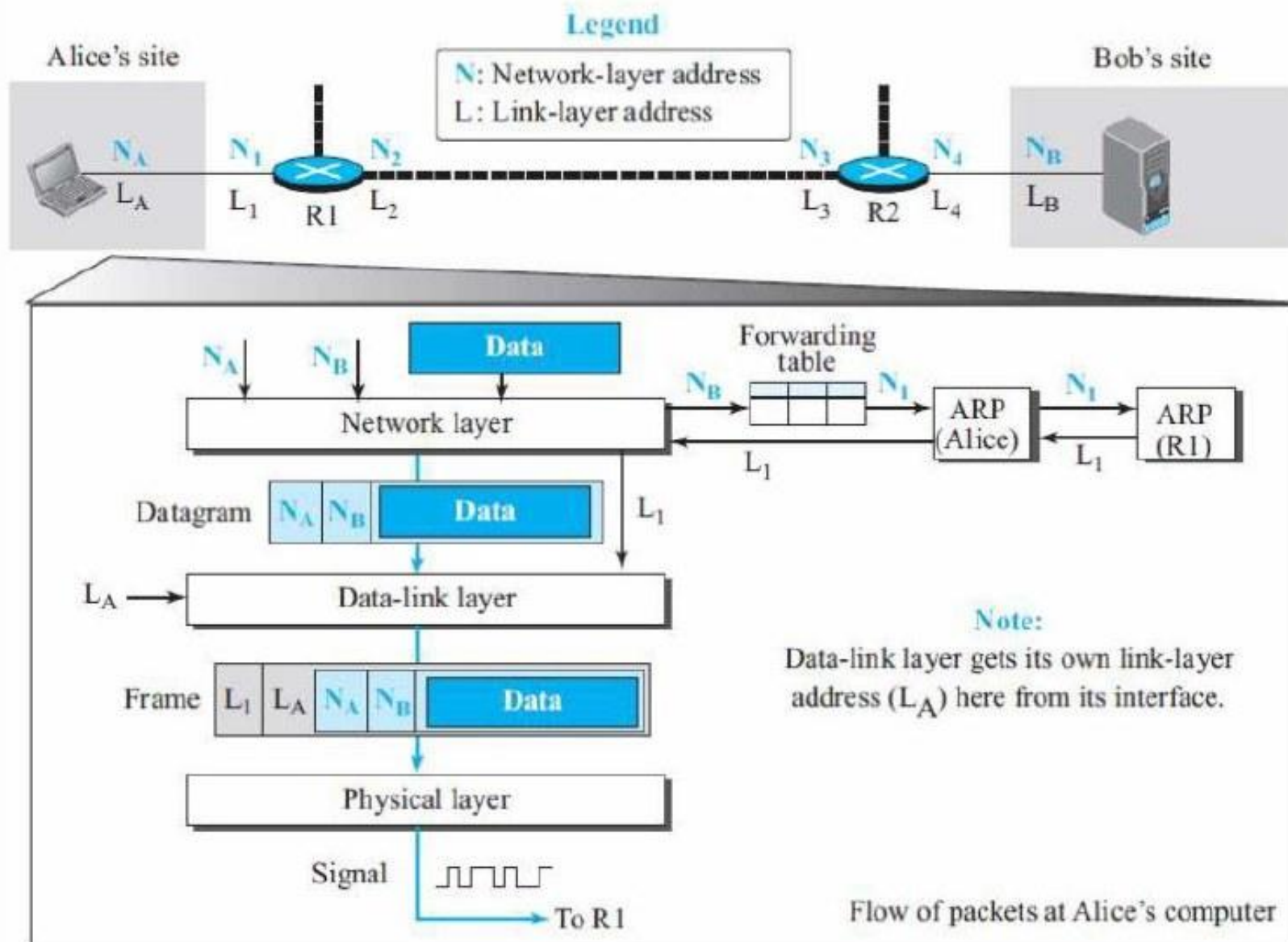
Case 3: A router has a packet to send to a host on another network.



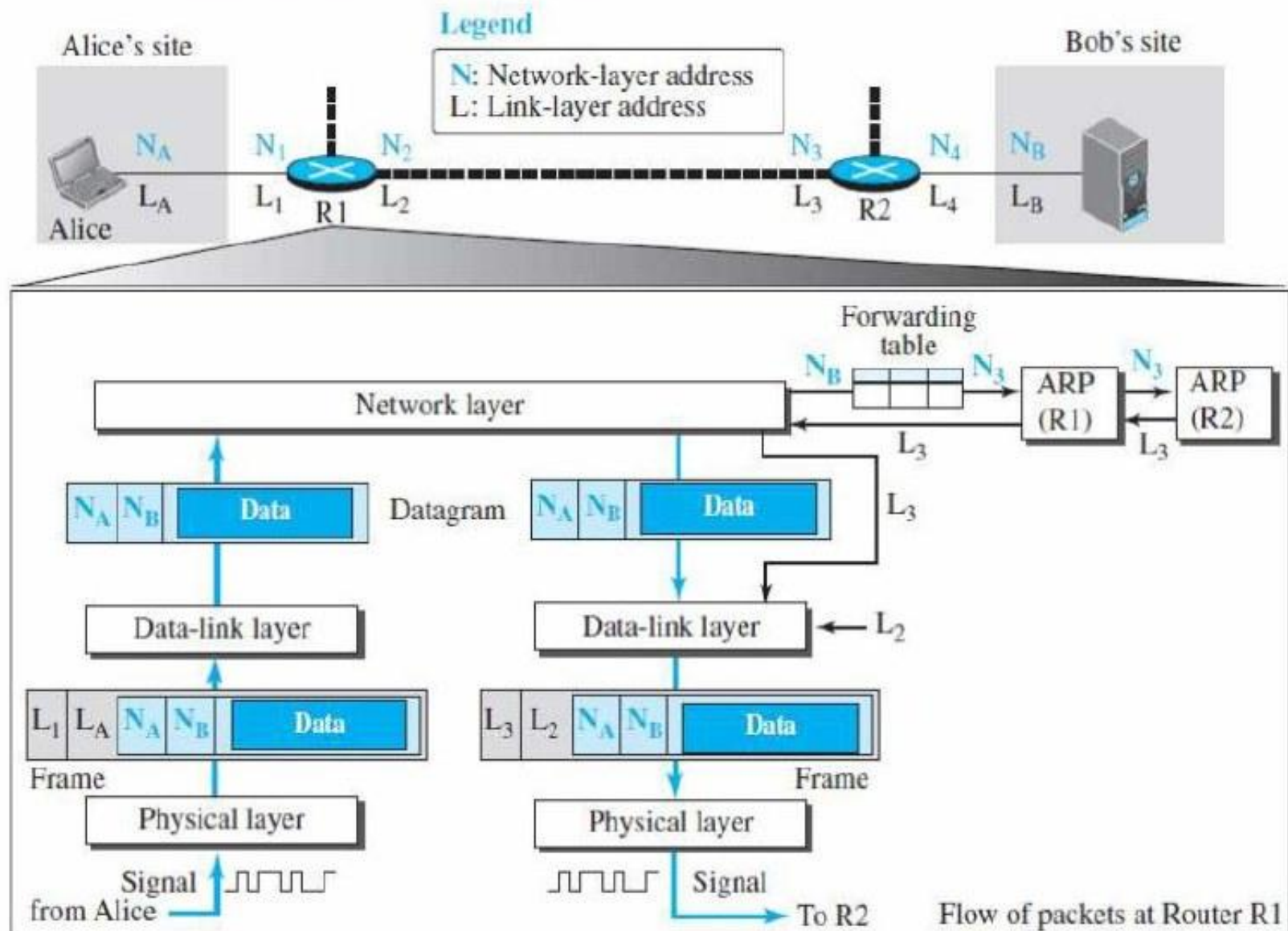
Case 4: A router has a packet to send to a host on the same network.



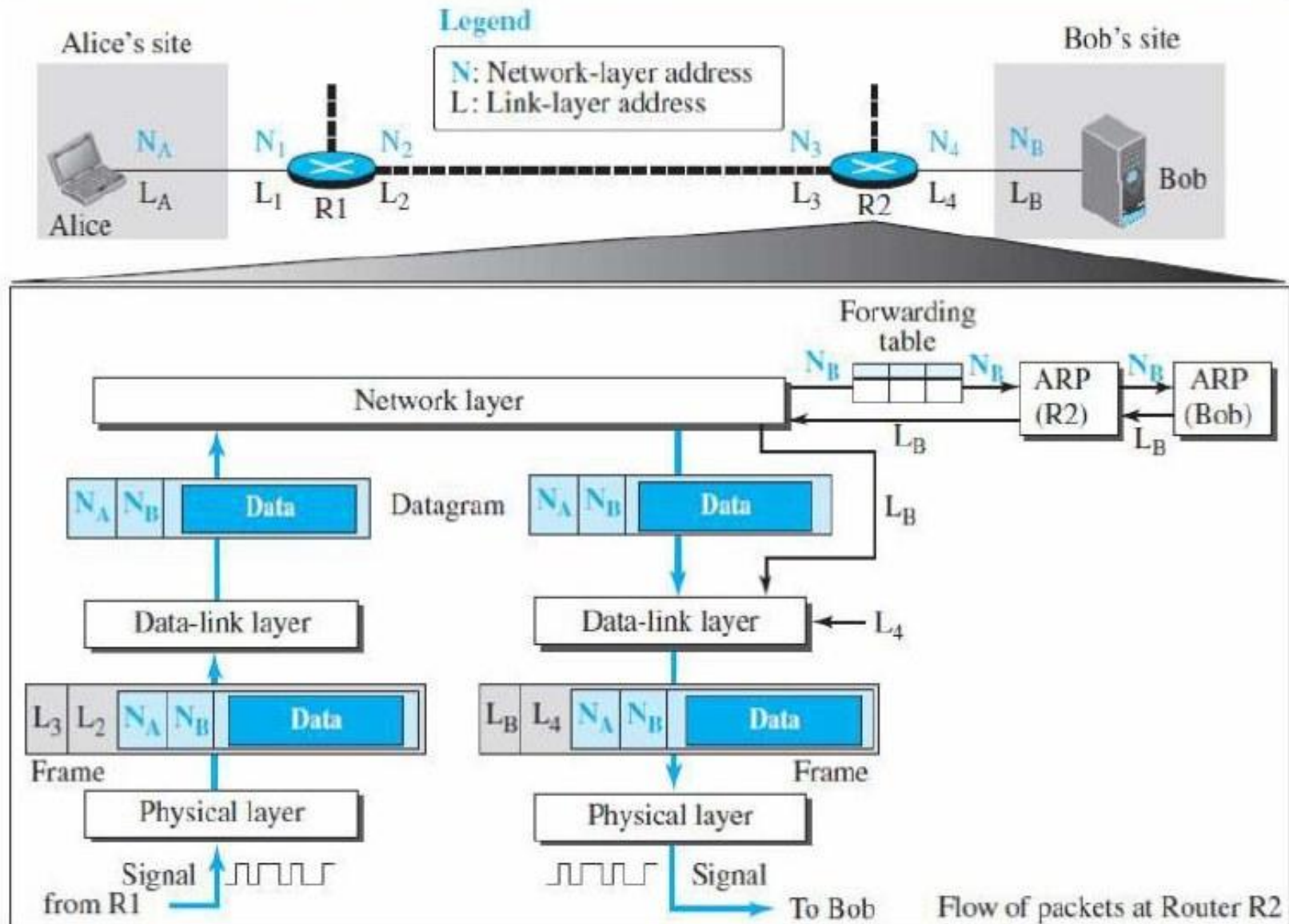
Flow of packets at Alice's computer



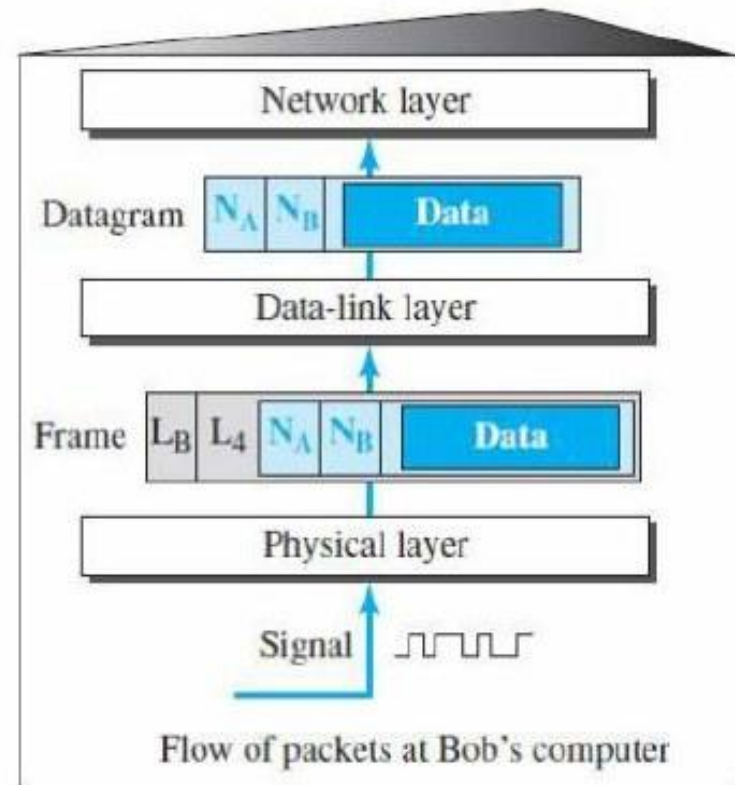
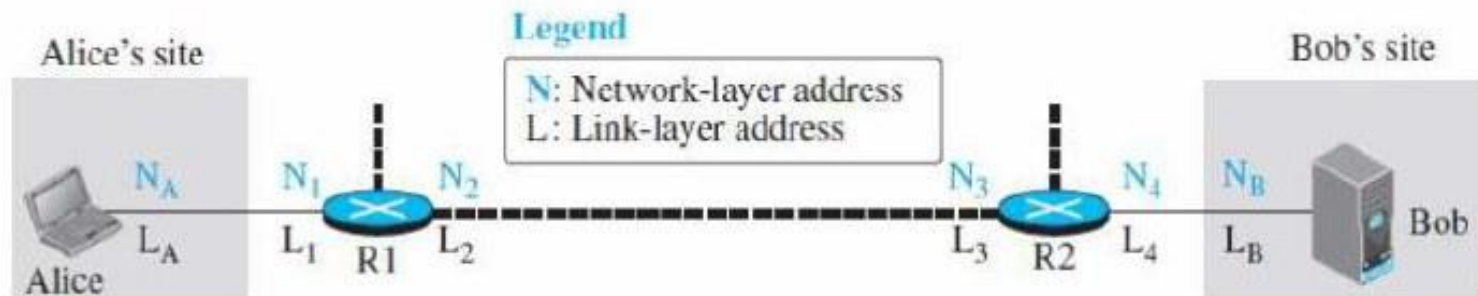
Flow of activities at router R1



Activities at router R2.



Activities at Bob's site

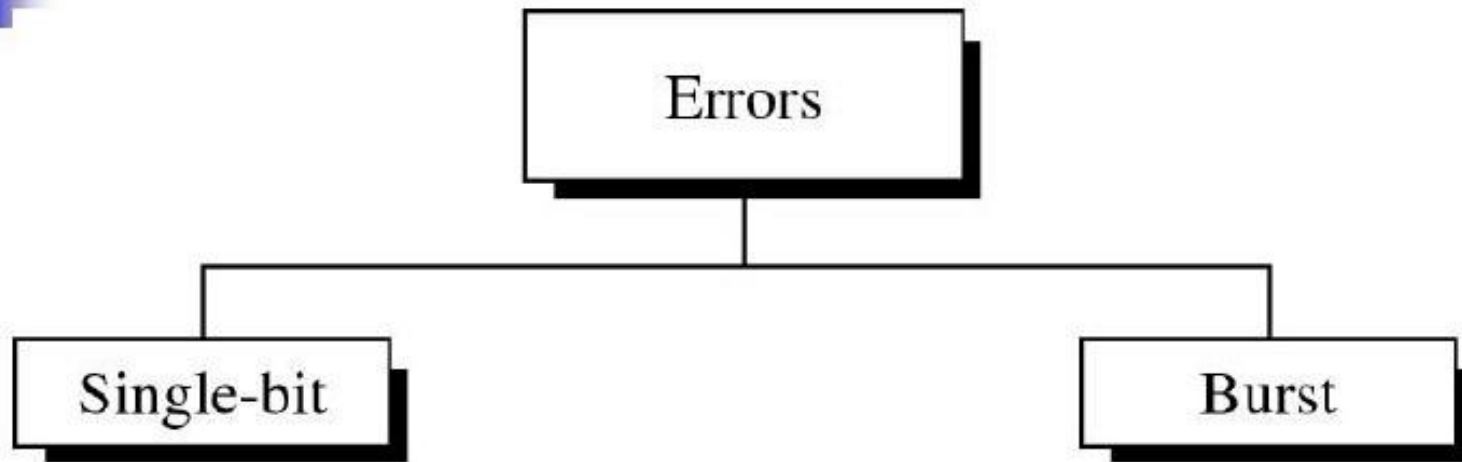


Error Detection and Correction

Note

**Data can be corrupted
during transmission.**

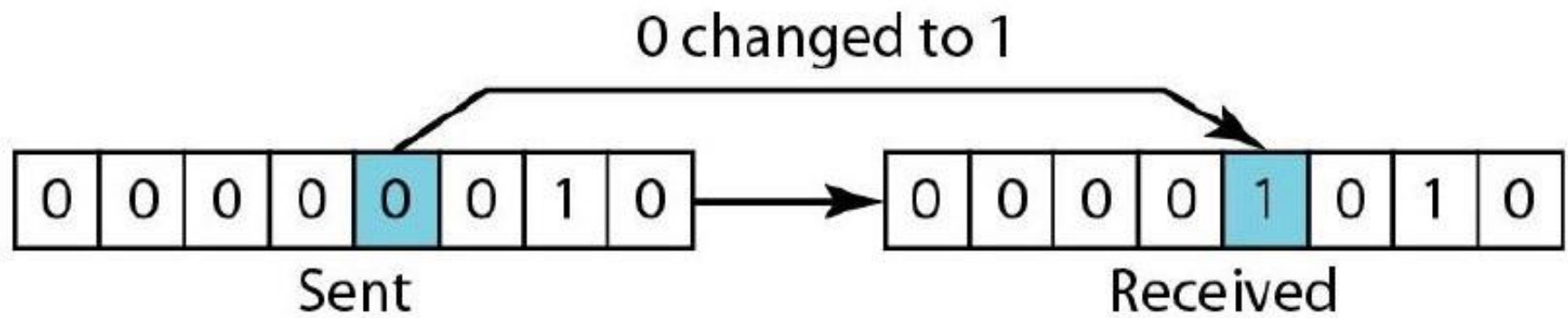
**Some applications require that
errors be detected and corrected.**



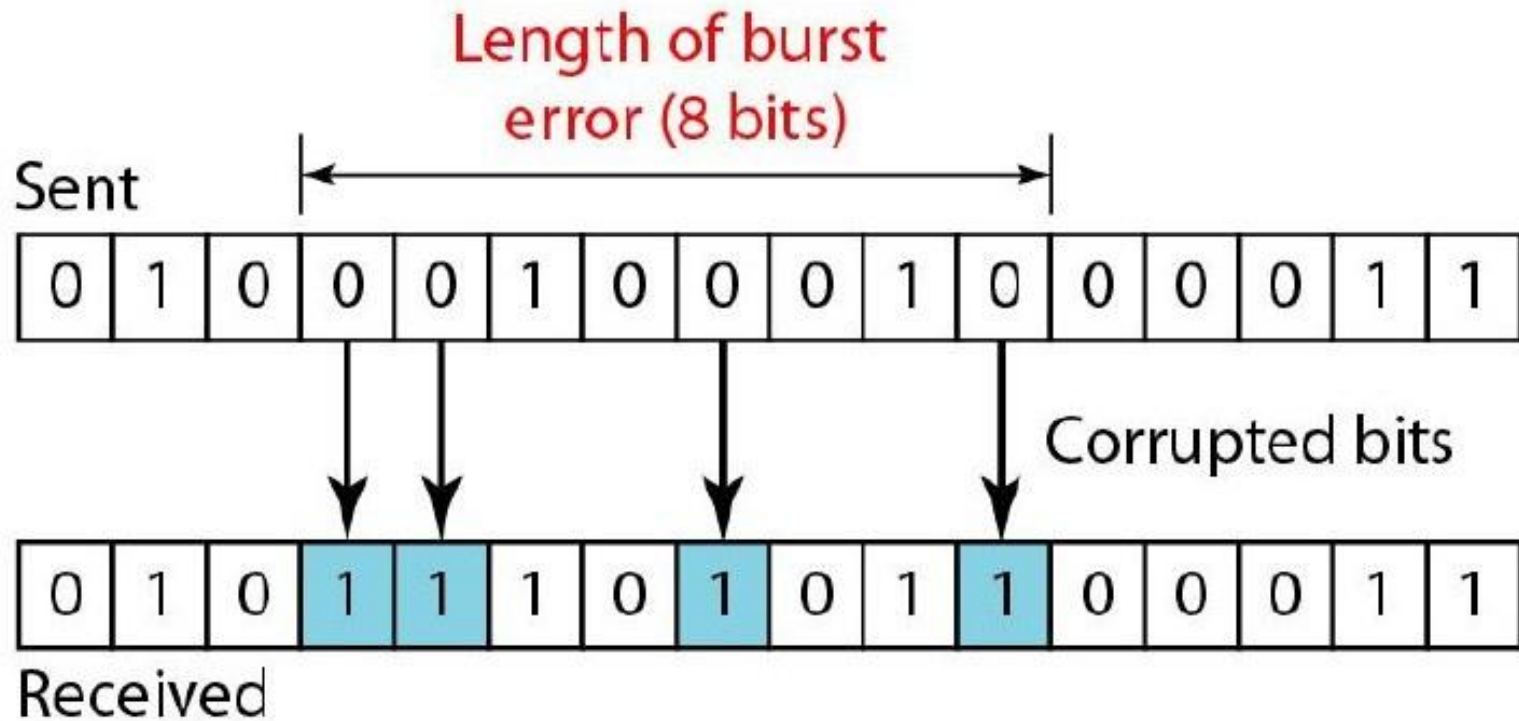
In a single-bit error, only 1 bit in the data unit has changed.

A burst error means that 2 or more bits in the data unit have changed.

Single-bit error



Burst error of length 8



Redundancy

Error detection or correction uses the concept of redundancy, which means adding extra bits for detecting and correcting errors at the destination

Note

To detect or correct errors, we need to send extra (redundant) bits with data.

Error Detection Vs Error Correction

Error Detection

- ✓ We need to look if error has occurred or not
- ✓ Answer is simple yes or no
- ✓ Single bit error and burst error are same

Error Correction

- ✓ Need to know exact number of bits corrupted and their location in the message
- ✓ Number of errors and message size are important factors

Correction is more difficult than detection

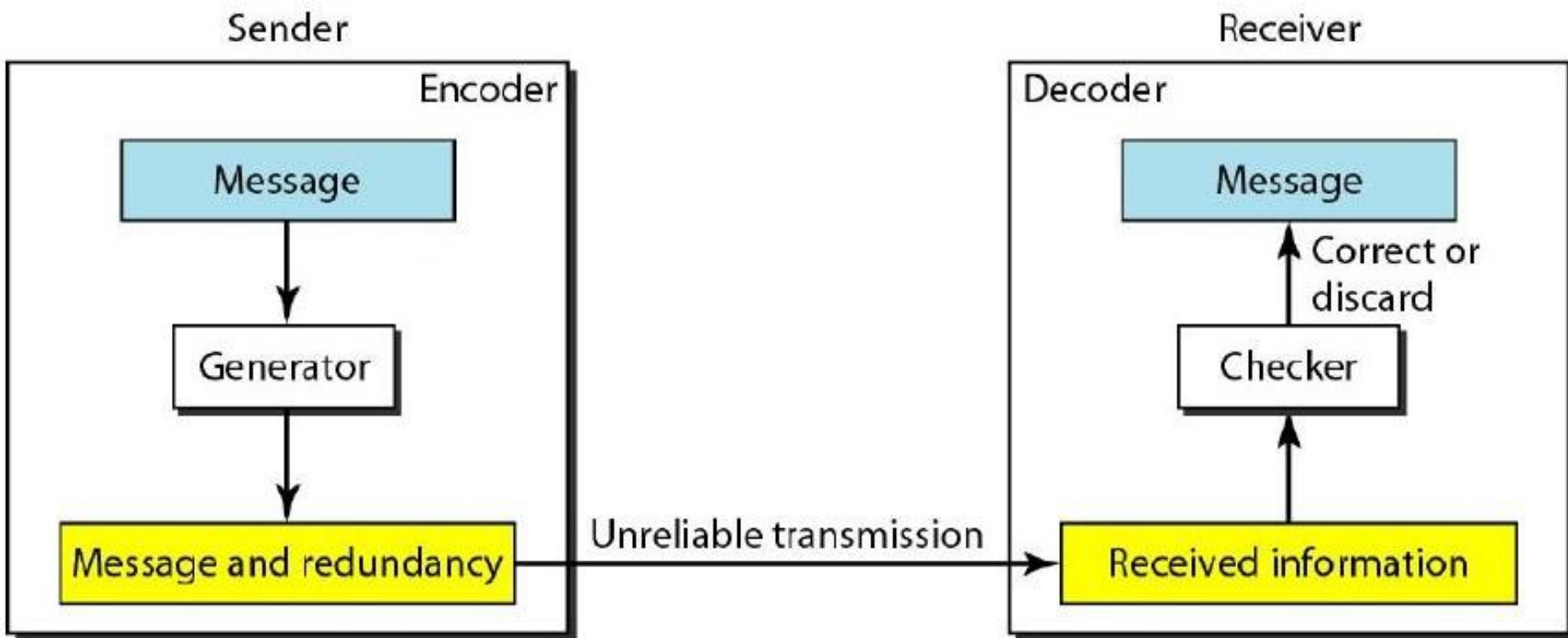
Two techniques of Error correction
Forward error correction and Retransmission

Coding

- ✓ Redundancy is achieved through various coding schemes
- ✓ Sender adds redundant bits through a process that creates a relationship between redundant bits and actual data bits
- ✓ Receiver checks the relationship between two sets of bits to detect or correct errors
- ✓ Two categories of coding schemes – Block coding and convolution coding

In this book and as per the syllabus, we concentrate on block codes and its variants only

The structure of encoder and decoder



Modular Arithmetic

In modulo- N arithmetic, we use only the integers in the range 0 to $N - 1$, inclusive.

Modulo 2 arithmetic

Modulo 2 arithmetic uses 0 and 1

Addition and Subtraction in modulo 2 arithmetic is same and is equivalent to XOR operation

$$0 \oplus 0 = 0 \qquad 1 \oplus 1 = 0$$

a. Two bits are the same, the result is 0.

$$0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1$$

b. Two bits are different, the result is 1.

$$\begin{array}{rcccccc} & & 1 & 0 & 1 & 1 & 0 \\ \oplus & 1 & 1 & 1 & 0 & 0 & \\ \hline & 0 & 1 & 0 & 1 & 0 & \end{array}$$

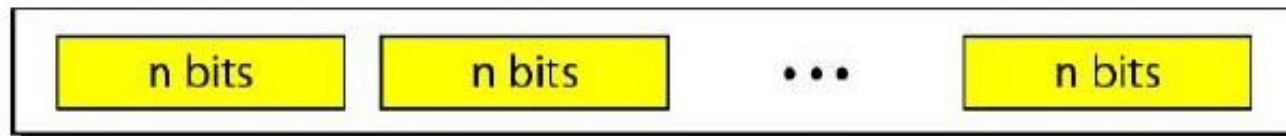
c. Result of XORing two patterns

BLOCK CODING

In block coding, we divide our message into blocks, each of k bits, called **datawords**. We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called **codewords**.



2^k Datawords, each of k bits



2^n Codewords, each of n bits (only 2^k of them are valid)



Example 10.1

In this coding scheme, $k = 4$ and $n = 5$.

We have :

$2^k = 16$ datawords,

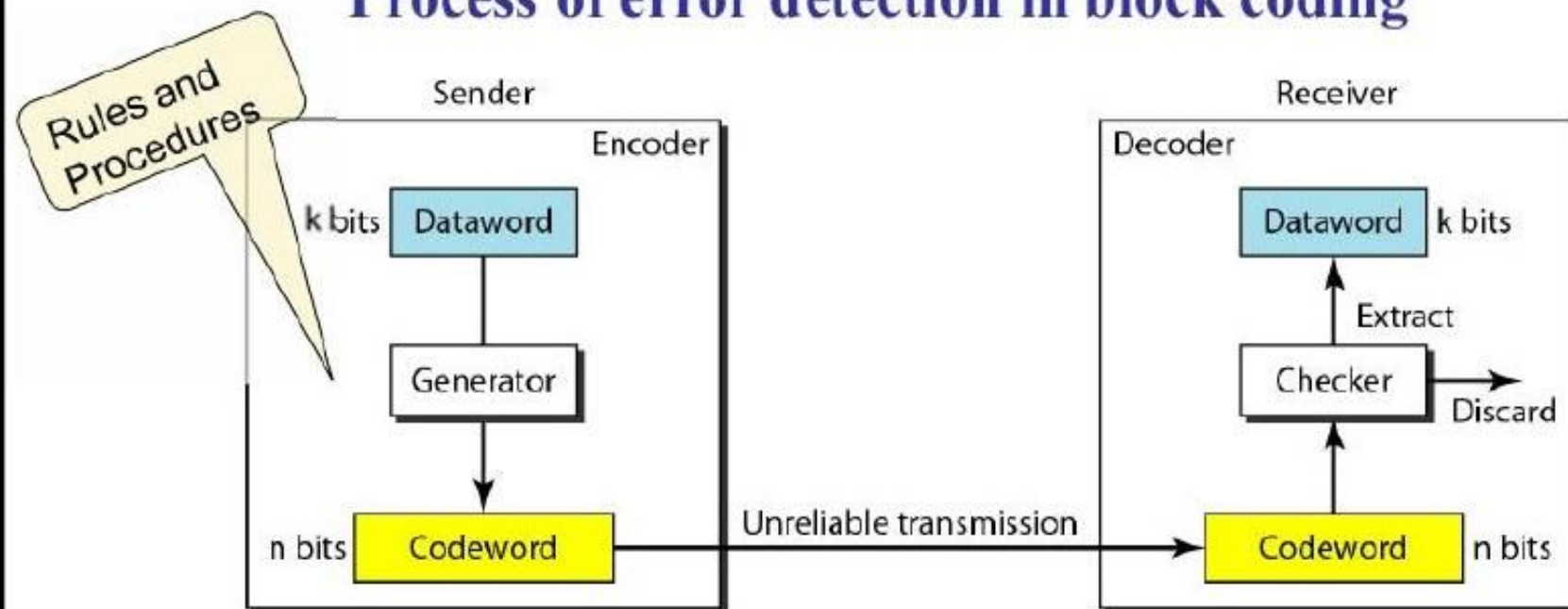
$2^n = 32$ codewords.

We see that 16 out of 32 codewords are used for message transfer and the rest are either used for other purposes or unused.

Error Detection

- How can the errors be detected by using block coding?
- If the following two conditions are met, the receiver can detect a change in the original codeword
 - The receiver has a list of a valid codewords
 - The original codeword has changed to an invalid one

Process of error detection in block coding



Example 10.2

Let us assume that $k = 2$ and $n = 3$. Following Table 10.1 shows the list of datawords and codewords.

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

- 1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.***

Example 10.2 (continued)

- 2. The codeword is corrupted during transmission, and 111 is received. This is not a valid codeword and is discarded.*
- 3. The codeword is corrupted during transmission, and 000 is received. This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.*

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Hamming Distance

The Hamming distance between two words is the number of differences between corresponding bits

- ✓ Hamming distance between two words x and y as $d(x,y)$
- ✓ Can be easily found by applying XOR operation on the two words and count the number of 1s in the result, the value is greater than ZERO

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

Example 10.4

Let us find the Hamming distance between two pairs of words.

1. The Hamming distance $d(000, 011)$ is 2 because

$$000 \oplus 011 \text{ is } 011 \text{ (two 1s)}$$

2. The Hamming distance $d(10101, 11110)$ is 3 because

$$10101 \oplus 11110 \text{ is } 01011 \text{ (three 1s)}$$

Example

Find the minimum Hamming distance of the coding scheme in the following table:

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Solution

We first find all Hamming distances.

$$\begin{array}{llll} d(000, 011) = 2 & d(000, 101) = 2 & d(000, 110) = 2 & d(011, 101) = 2 \\ d(011, 110) = 2 & d(101, 110) = 2 & & \end{array}$$

The $d_{\min}$ in this case is 2.

Example

Find the minimum Hamming distance of the coding scheme in the following table:

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Solution

We first find all Hamming distances.

$d(00000, 01011) = 3$	$d(00000, 10101) = 3$	$d(00000, 11110) = 4$
$d(01011, 10101) = 4$	$d(01011, 11110) = 3$	$d(10101, 11110) = 3$

The d_{min} in this case is 3.

Three parameters

Any coding scheme needs to have three parameters:

- ✓ The codeword size n
- ✓ The dataword size k
- ✓ Minimum hamming distance d_{min}

Coding scheme C is written as $C(n,k)$ with d_{min}

Example: First coding scheme is $C(3,2)$ with $d_{min} = 2$

Note

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{min} = s + 1$.



Example 10.9

A code scheme has a Hamming distance $d_{\min} = 4$. What is the error detection capability of this scheme?

Solution

*This code guarantees the detection of up to **three** errors ($s = 3$)*

LINEAR BLOCK CODES

Almost all block codes used today belong to a subset called **linear block codes**. A linear block code is a code in which the exclusive OR (addition modulo-2) of two valid codewords creates another valid codeword.

Some Linear Block Codes

- ✓ Parity Check Code
- ✓ Hamming Code

Note

In a linear block code, the exclusive OR (XOR) of any two valid codewords creates another valid codeword.

Note

**A parity-check code is a
single-bit error-detecting
code in which
 $n = k + 1$ with $d_{\min} = 2$.**

Figure 10.10 *Encoder and decoder for simple parity-check code*

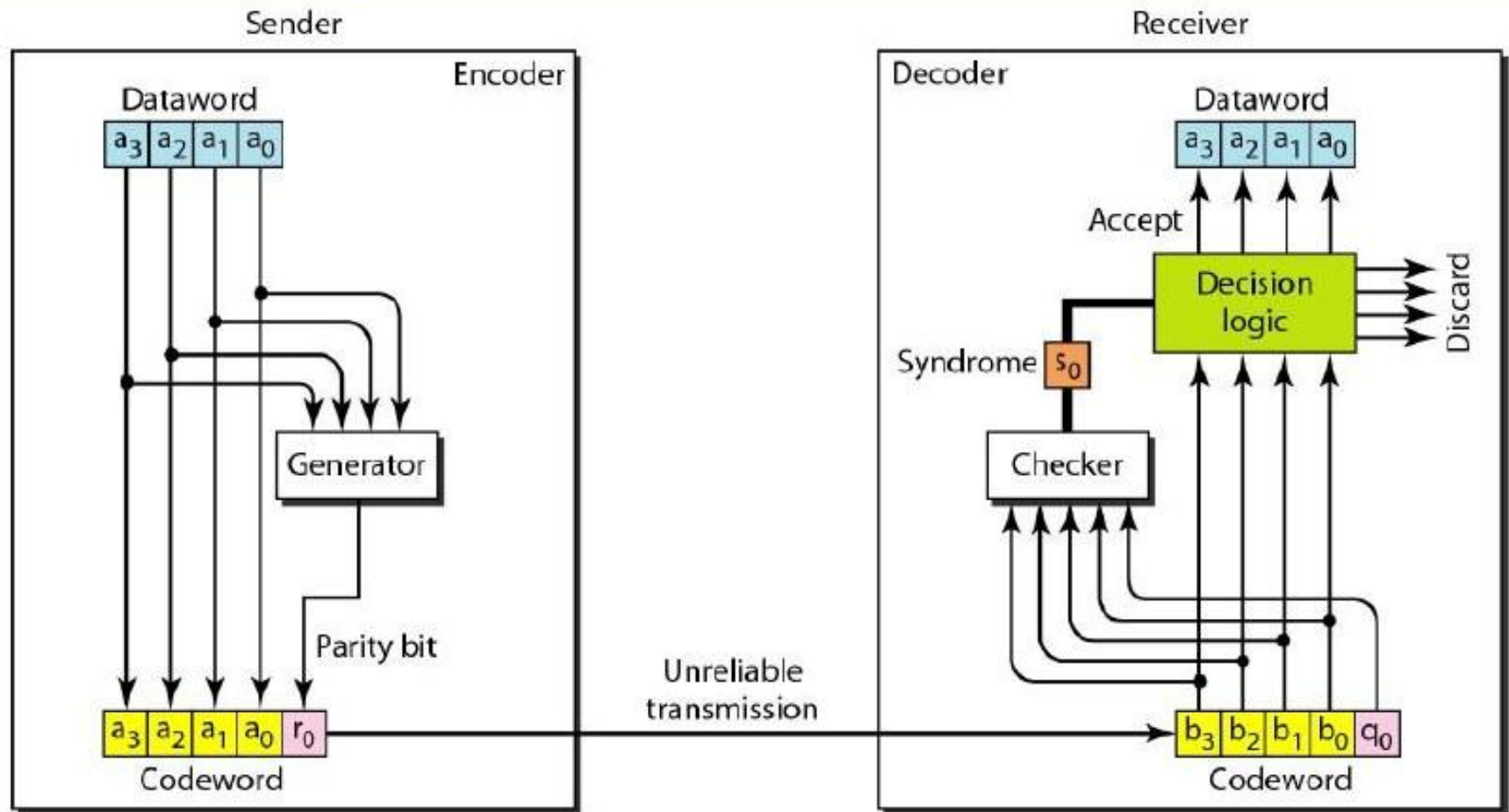


Table 10.3 *Simple parity-check code C(5, 4)*

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

$$r_0 = a_3 + a_2 + a_1 + a_0 \text{ (modulo-2)}$$

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \text{ (modulo-2)}$$



Example

Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:

- 1. No error occurs; the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.*
- 2. One single-bit error changes a_1 . The received codeword is 10011. The syndrome is 1. No dataword is created.*
- 3. One single-bit error changes r_0 . The received codeword is 10110. The syndrome is 1. No dataword is created.*

Example 10.12 (continued)

- 4. An error changes r_0 and a second error changes a_3 . The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value.*
- 5. Three bits— a_3 , a_2 , and a_1 —are changed by errors. The received codeword is 01011. The syndrome is 1. The dataword is not created. This shows that the simple parity check, guaranteed to detect one single error, can also find any odd number of errors.*

Note

A simple parity-check code can detect an odd number of errors.

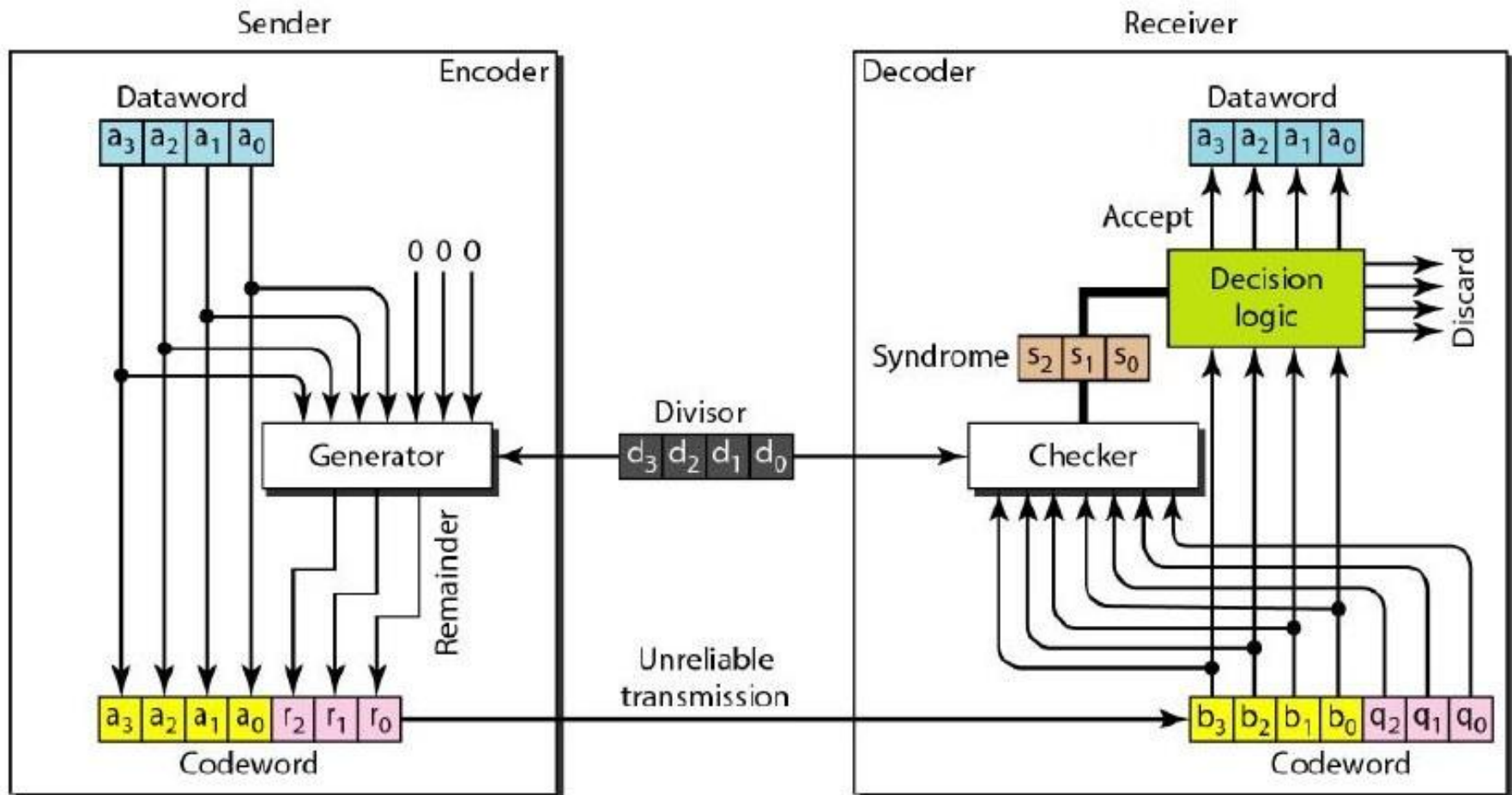
CYCLIC CODES

Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.

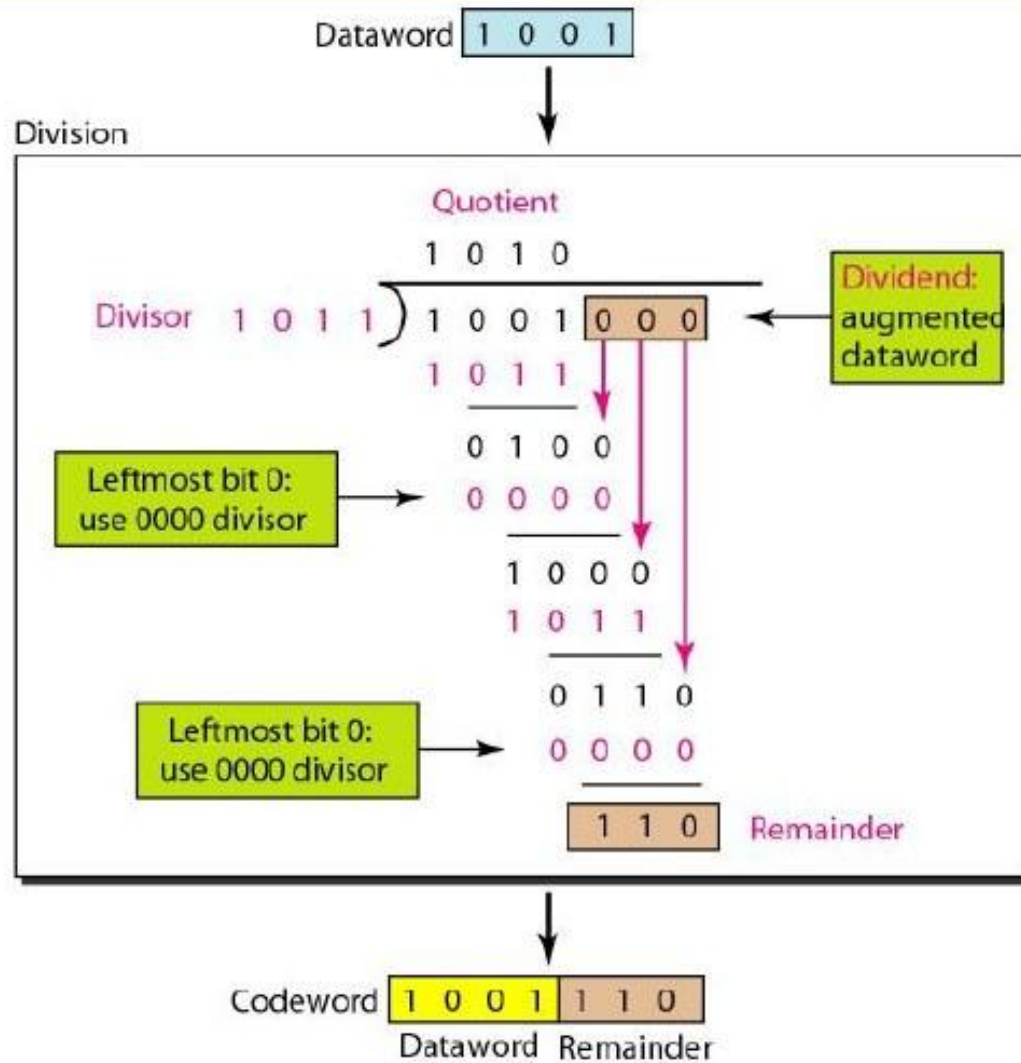
<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

A CRC code with $C(7, 4)$

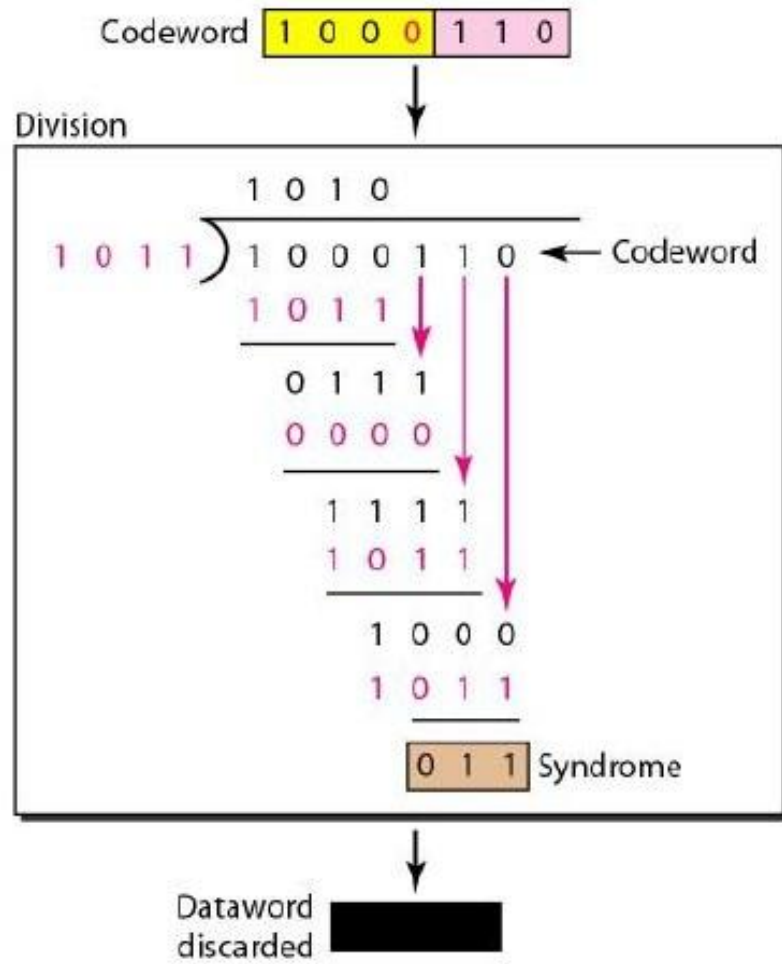
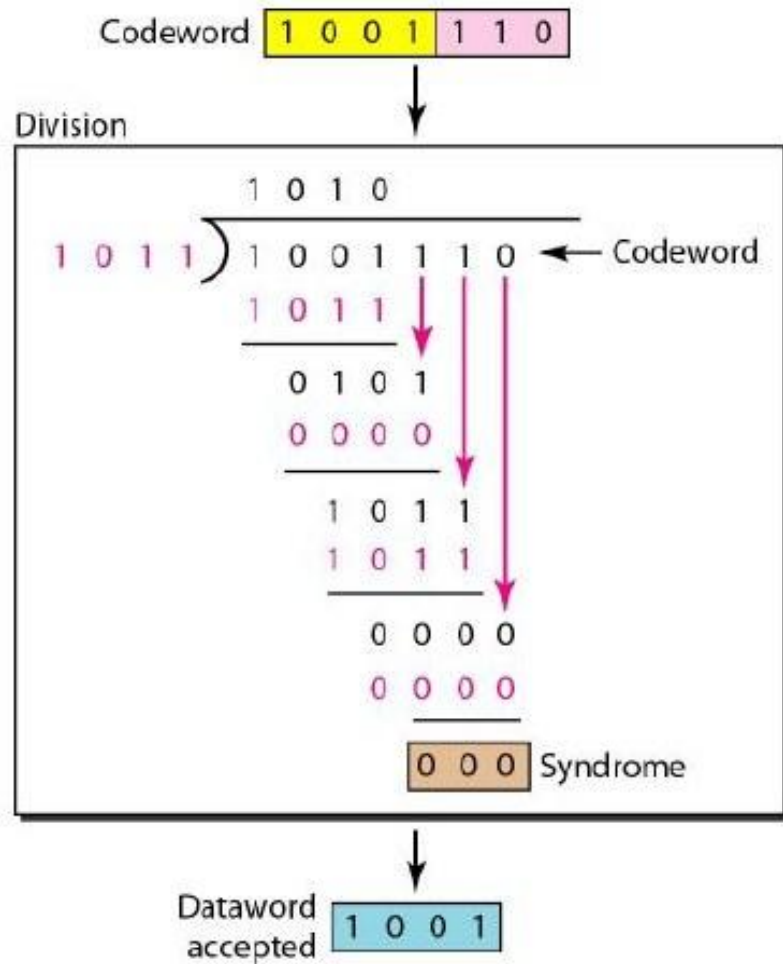
CRC encoder and decoder



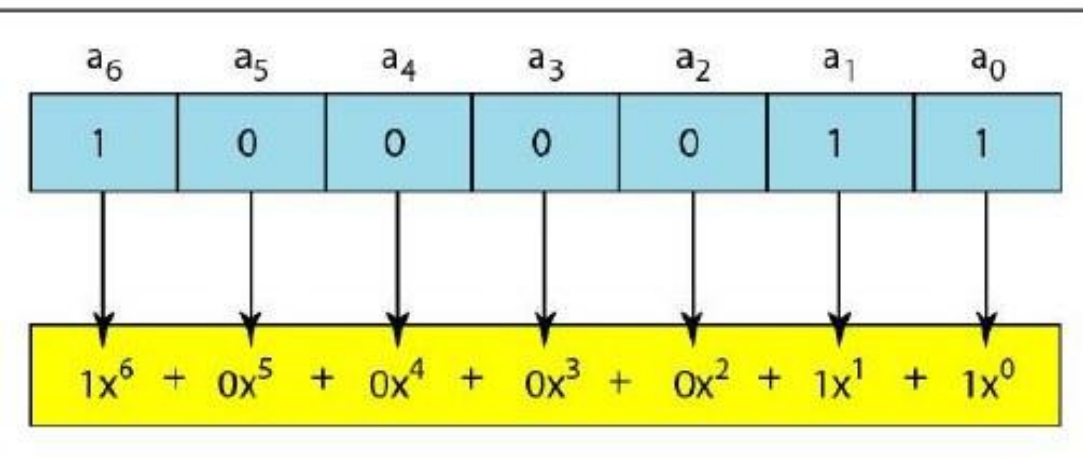
Division in CRC encoder



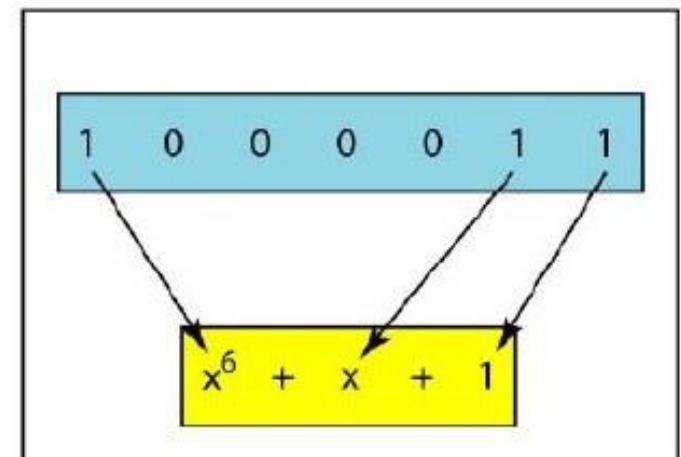
Division in the CRC decoder for two cases



A polynomial to represent a binary word

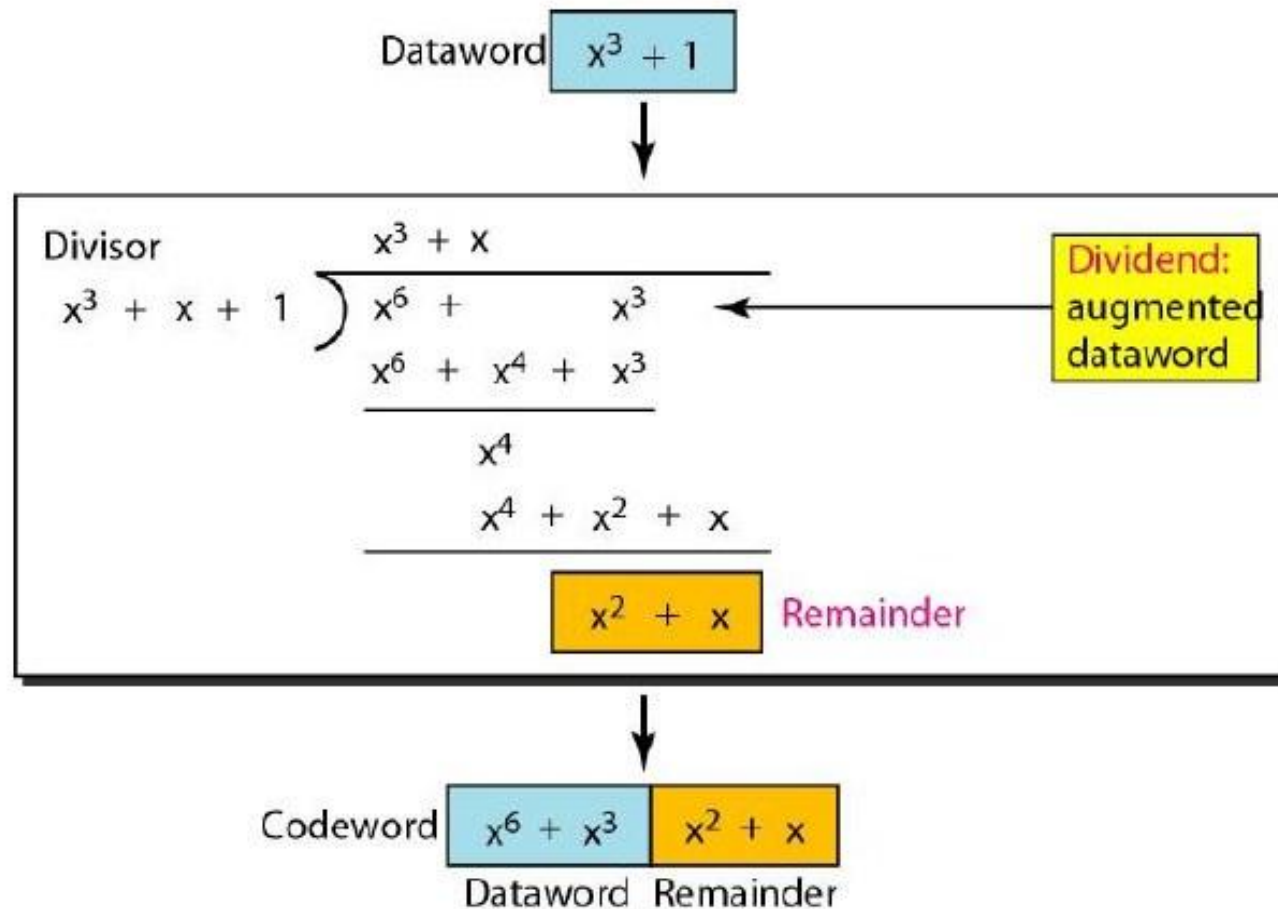


a. Binary pattern and polynomial



b. Short form

CRC division using polynomials



Note

The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

10-5 CHECKSUM

The last error detection method we discuss here is called the checksum. The checksum is used in the Internet by several protocols although not at the data link layer. However, we briefly discuss it here to complete our discussion on error checking

Topics discussed in this section:

Idea

One's Complement

Internet Checksum



Example

Suppose our data is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the data are not accepted.



Example

*We can make the job of the receiver easier if we send the negative (complement) of the sum, called the **checksum**. In this case, we send (7, 11, 12, 0, 6, **-36**). The receiver can add all the numbers received (including the checksum). If the result is 0, it assumes no error; otherwise, there is an error.*



Example

*How can we represent the number 21 in **one's complement arithmetic** using only four bits?*

Solution

*The number 21 in binary is 10101 (it needs five bits). We can wrap the leftmost bit and add it to the four rightmost bits. We have $(0101 + 1) = 0110$ or **6**.*



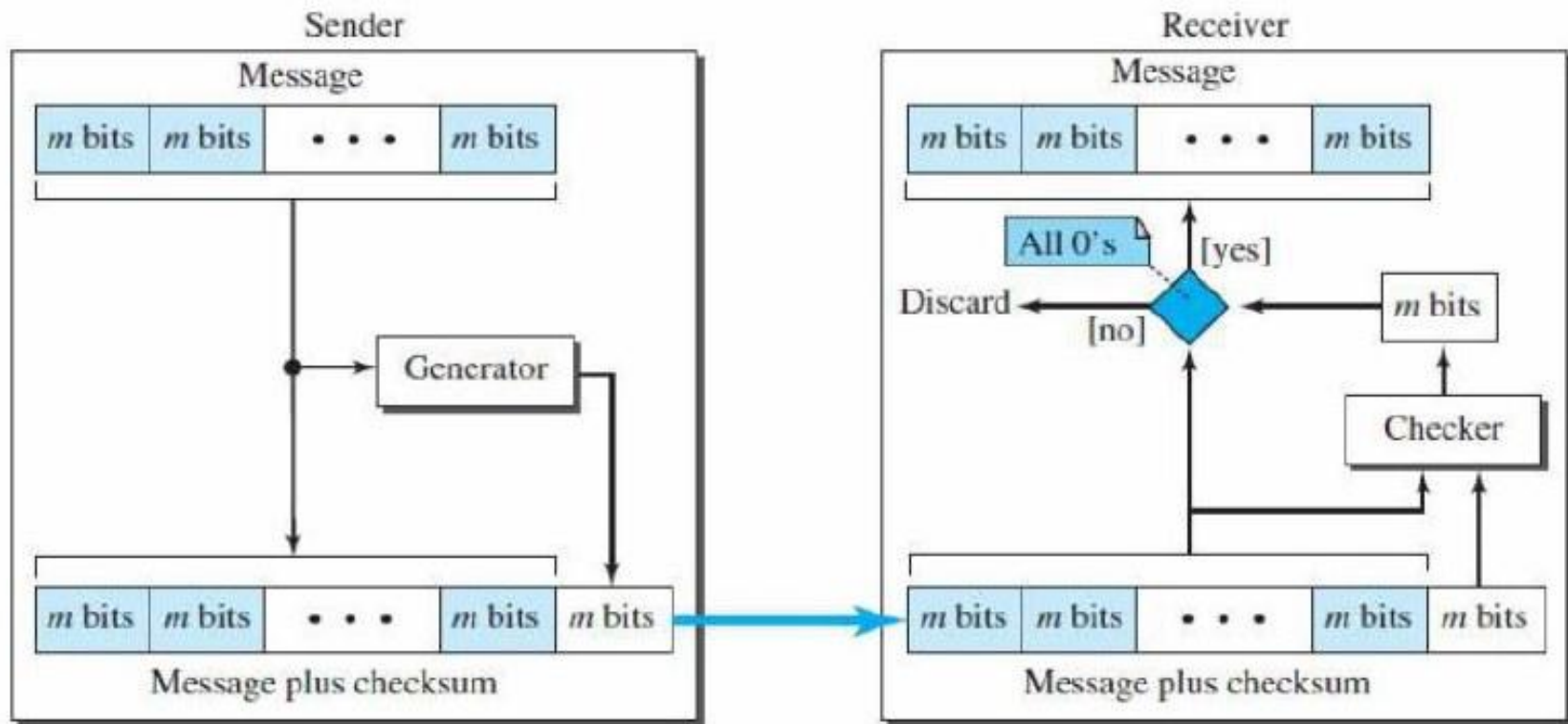
Example

How can we represent the number -6 in one's complement arithmetic using only four bits?

Solution

In one's complement arithmetic, the negative or complement of a number is found by inverting all bits. Positive 6 is 0110; negative 6 is 1001. If we consider only unsigned numbers, this is 9. In other words, the complement of 6 is 9. Another way to find the complement of a number in one's complement arithmetic is to subtract the number from $2^n - 1$ ($16 - 1$ in this case).

Checksum



Example

Sender site

	7
	11
	12
	0
	6
	0
Sum	→ 36
Wrapped sum	→ 6
Checksum	→ 9

7, 11, 12, 0, 6, 9

Packet

Receiver site

	7
	11
	12
	0
	6
	9
Sum	→ 45
Wrapped sum	→ 15
Checksum	→ 0

1	0	0	1	0	0	36
└───┬───┘						
└───┬───┘						10

0	1	1	0			6
1	0	0	1			9

Details of wrapping
and complementing

1	0	1	1	0	1	45
└───┬───┘						
└───┬───┘						10

1	1	1	1			15
0	0	0	0			0

Details of wrapping
and complementing

Table 10.5 *Procedure to calculate the traditional checksum*

<i>Sender</i>	<i>Receiver</i>
1. The message is divided into 16-bit words. 2. The value of the checksum word is initially set to zero. 3. All words including the checksum are added using one's complement addition. 4. The sum is complemented and becomes the checksum. 5. The checksum is sent with the data.	1. The message and the checksum are received. 2. The message is divided into 16-bit words. 3. All words are added using one's complement addition. 4. The sum is complemented and becomes the new checksum. 5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Example 10.23

Let us calculate the checksum for a text of 8 characters ("Forouzan"). The text needs to be divided into 2-byte (16-bit) words. We use ASCII (see Appendix A) to change each byte to a 2-digit hexadecimal number. For example, F is represented as 0x46 and o is represented as 0x6F. Figure 10.25 shows how the checksum is calculated at the sender and receiver sites. In part a of the figure, the value of partial sum for the first column is 0x36. We keep the rightmost digit (6) and insert the leftmost digit (3) as the carry in the second column. The process is repeated for each column. Note that if there is any corruption, the checksum recalculated by the receiver is not all 0s. We leave this an exercise.

Figure 10.25 *Example 10.23*

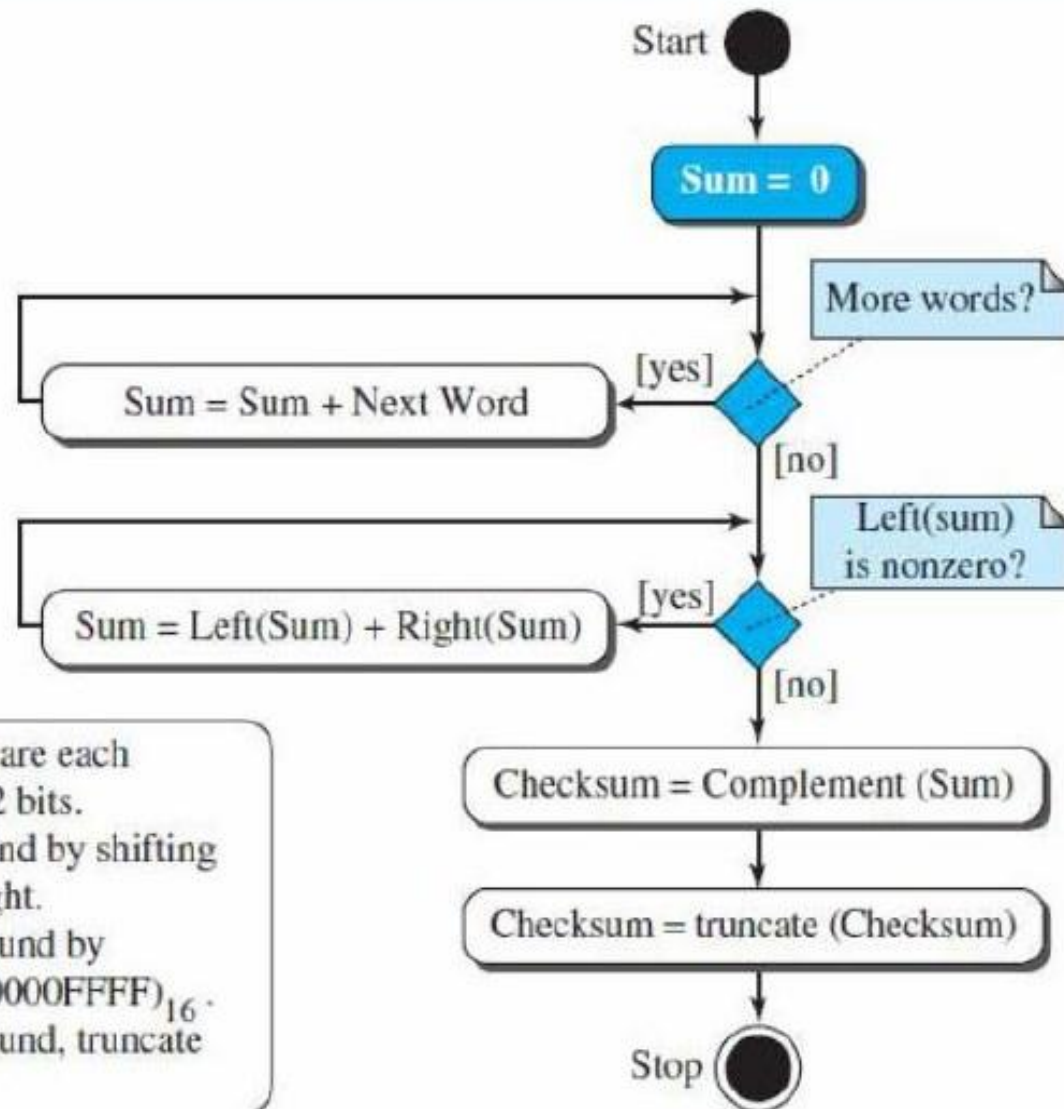
1	0	1	3	Carries	
4	6	6	F	(Fo)	
7	2	6	F	(ro)	
7	5	7	A	(uz)	
6	1	6	E	(an)	
0	0	0	0		Checksum (initial)
8	F	C	6		Sum (partial)
8	F	C	7		Sum
7	0	3	8		Checksum (to send)

a. Checksum at the sender site

1	0	1	3	Carries	
4	6	6	F	(Fo)	
7	2	6	F	(ro)	
7	5	7	A	(uz)	
6	1	6	E	(an)	
7	0	3	8		Checksum (received)
F	F	F	E		Sum (partial)
F	F	F	F		Sum
0	0	0	0		Checksum (new)

a. Checksum at the receiver site

Algorithm to calculate a traditional checksum



Notes:

- Word and Checksum are each 16 bits, but Sum is 32 bits.
- Left(Sum) can be found by shifting Sum 16 bits to the right.
- Right(Sum) can be found by ANDing Sum with $(0000FFFF)_{16}$.
- After Checksum is found, truncate it to 16 bits.

Exercise Problems - Checksum

1. Calculate the Internet check sum for the text
“ISE DEPARTMENT” at sender and receiver side
(assume A=41, D=44, E=45, I=49, M=4D, N=4E, P=50,
R=52, S=53, T=54, space=32)

Solution to Problem 1

Sender				
2	3	1	1	
	4	9	5	3
	4	5	3	2
	4	4	4	5
	5	0	4	1
	5	2	5	4
	4	D	4	5
	4	E	5	4
	0	0	0	0
	1	0	F	8
				2
	1	0	F	A
	E	F	0	5

IS
E Space
DE
PA
RT
ME
NT
Initial Checksum
Partial Sum
Wrapping
Wrapped Sum
Checksum to send

Solution to Problem 1

Receiver				
2	3	1	1	
	4	9	5	3
	4	5	3	2
	4	4	4	5
	5	0	4	1
	5	2	5	4
	4	D	4	5
	4	E	5	4
	E	F	0	5
	F	F	F	D
				2
	F	F	F	F
	0	0	0	0

IS
E Space
DE
PA
RT
ME
NT
Received Checksum
Partial Sum
Wrapping
Wrapped Sum
New Checksum

Exercise Problems - Checksum

2. Calculate the Internet check sum for the text
“**INFORMATIONSCIENCE**” at sender and receiver side
(consider A=41, C=43, E=45, F=46, I=49, M=4D, N=4E,
O=4F, R=52, S=53, T=54)

Exercise Problems - Checksum

3. A sender needs to send the four data items 0x3456, 0xABCC, 0x02BC and 0xEEEE

- a) Find the checksum at the sender site
 - b) Find the checksum at the receiver site, if there is no error
 - c) Find the checksum at the receiver site, if the second data item is changed to 0xABCE
 - d) Find the checksum at the receiver site, if the second data item is changed to 0xABCE and the third data item is changed to 0x02BA
-

Solution to Problem 3

a. Checksum at the sender site

1	2	2	2	Carries
3	4	5	6	
A	B	C	C	
0	2	B	C	
E	E	E	E	
0	0	0	0	Checksum (initial)
D	1	C	C	Sum (partial)
			1	
D	1	C	D	Sum
2	E	3	2	Checksum (to send)

b. Checksum at the receiver site (no error)

1	2	2	2	Carries
3	4	5	6	
A	B	C	C	
0	2	B	C	
E	E	E	E	
2	E	3	2	Checksum (received)
F	F	F	E	Sum (partial)
			1	
F	F	F	F	Sum
0	0	0	0	Checksum

Solution to Problem 3

2	3	3	3	Carries
3	4	5	6	
A	B	C	E	— Error here
0	2	B	C	
E	E	E	E	
2	E	3	2	Checksum (received)
0	0	0	0	Sum (partial)
0	0	0	2	Sum
F	F	F	D	Checksum

c. Checksum at the receiver site (one caught error)

1	2	2	2	Carries
3	4	5	6	
A	B	C	E	————— Error here
0	2	B	A	————— Error here
E	E	E	E	
2	E	3	2	Checksum (received)
F	F	F	E	Sum (partial)
F	F	F	F	Sum
0	0	0	0	Checksum

d. Checksum at the receiver site (two errors, but not caught)

Exercise Problems - CRC

1. Given the data word 10100111 and the divisor 10111
 - a) Show the generation of the codeword at the sender site (using binary division)
 - b) Show the checking of the codeword at the receiver site (assume no error)
-

Solution

Dataword

1 0 1 0 0 1 1 1

1 0 1 1 1

Divisor

1 0 0 1 1 1 1

Quotient

1 0 1 0 0 1 1 1

0 0 1 1 1

0 0 0 0 0

0 1 1 1 1

0 0 0 0 0

1 1 1 1 1

1 0 1 1 1

1 0 0 0 0

1 0 1 1 1

0 1 1 1 0

0 0 0 0 0

1 1 1 0 0

1 0 1 1 1

1 0 1 1 0

1 0 1 1 1

0 0 0 1

Remainder

Codeword

1 0 1 0 0 1 1 1 0 0 0 1

Codeword

1 0 1 0 0 1 1 1 0 0 0 1

1 0 1 1 1

Divisor

1 0 0 1 1 1 1

Quotient

1 0 1 0 0 1 1 1

1 0 1 1 1

0 0 1 1 1

0 0 0 0 0

0 1 1 1 1

0 0 0 0 0

1 1 1 1 1

1 0 1 1 1

1 0 0 0 0

1 0 1 1 1

0 1 1 1 0

0 0 0 0 0

1 1 1 0 0

1 0 1 1 1

1 0 1 1 1

0 0 0 0

Remainder (no error)

Dataword

1 0 1 0 0 1 1 1

Exercise Problems - CRC

2. Given the data word 10100111 and the divisor 10111, do the following using CRC polynomial division

- a) Show the generation of the codeword at the sender site (using binary division)
 - b) Show the checking of the codeword at the receiver site (assume no error)
-

Solution

Dataword $x^7 + x^3 + x^2 + x + 1$

Sender

Divisor	$x^7 + x^4 + x^3 + x + 1$	Quotient
$x^4 + x^2 + x + 1$	$ \begin{array}{r} x^{11} + x^9 + + x^6 + x^5 + x^4 \\ \underline{x^{11} + x^9 + x^8 + x^7} \\ x^8 + x^7 + x^6 + x^5 + x^4 \\ \underline{x^8 + x^6 + x^5 + x^4} \\ x^7 \\ \underline{x^7} + x^5 + x^4 + x^3 \\ x^5 + x^4 + x^3 \\ \underline{x^5 + x^3 + x^2 + x} \\ x^4 + x^2 + x \\ \underline{x^4 + x^2 + x + 1} \\ \text{Remainder } 1 \end{array} $	

Codeword $x^{11} + x^7 + x^6 + x^5 + x^4 + 1$

Codeword $x^{11} + x^7 + x^6 + x^5 + x^4 + 1$

Receiver

Divisor	$x^7 + x^4 + x^3 + x + 1$	Quotient
$x^4 + x^2 + x + 1$	$ \begin{array}{r} x^{11} + x^9 + + x^6 + x^5 + x^4 + 1 \\ \underline{x^{11} + x^9 + x^8 + x^7} \\ x^8 + x^7 + x^6 + x^5 + x^4 \\ \underline{x^8 + x^6 + x^5 + x^4} \\ x^7 \\ \underline{x^7} + x^5 + x^4 + x^3 \\ x^5 + x^4 + x^3 \\ \underline{x^5 + x^3 + x^2 + x} \\ x^4 + x^2 + x + 1 \\ \underline{x^4 + x^2 + x + 1} \\ \text{Remainder } 0 \end{array} $	

Dataword $x^7 + x^3 + x^2 + x + 1$

Data Link Control

FRAMING

*The data link layer needs to pack bits into **frames**, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.*

Topics discussed in this section:

Fixed-Size Framing

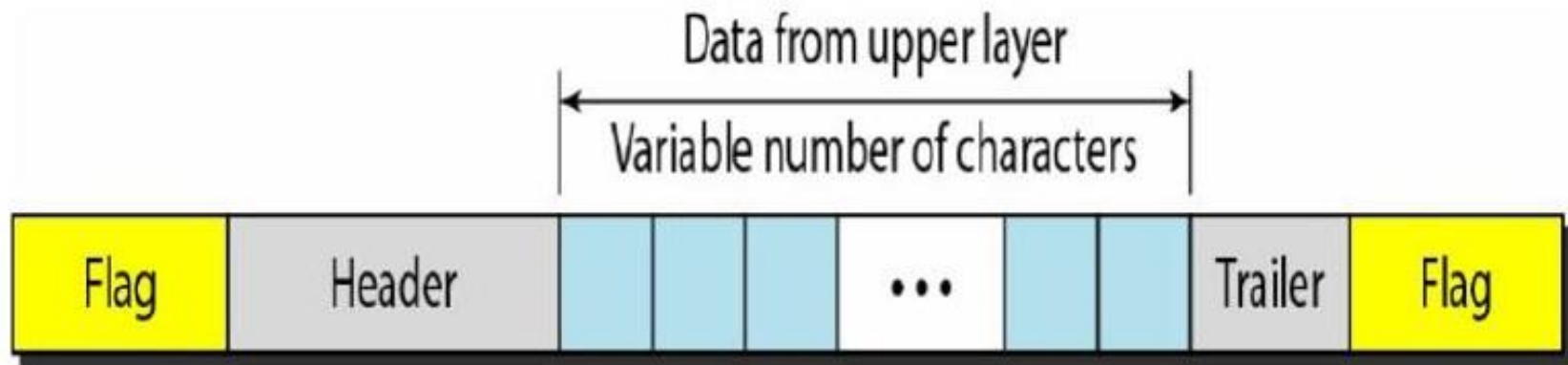
Variable-Size Framing

FRAMING

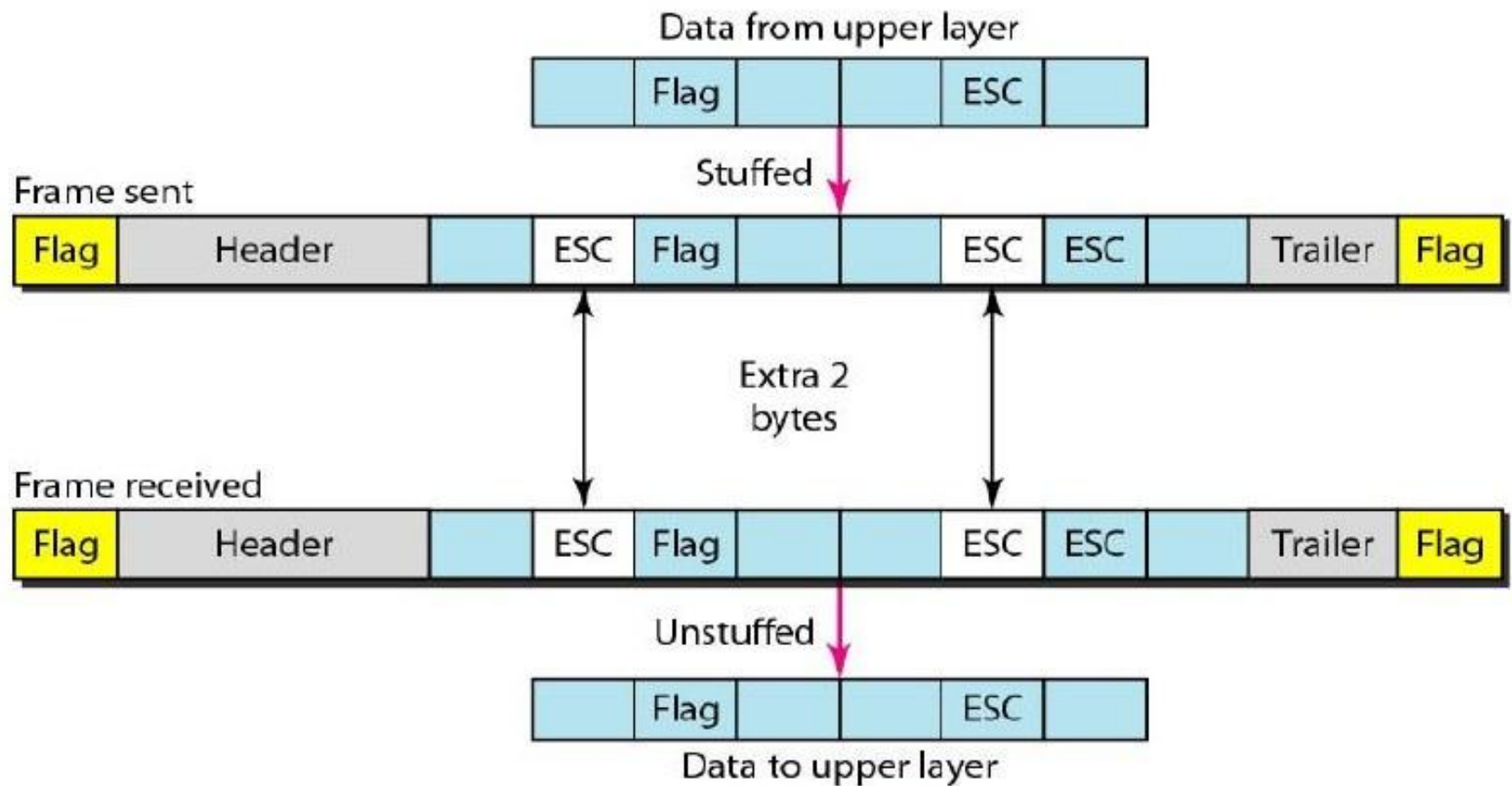
- Fixed - size framing:
 - No delimiter required, size itself separate the frames

 - Variable - size framing
 - Start and end of the frame to be defined
 - Two approaches
 - Character-oriented approach
 - Bit-oriented approach
-

A frame in a character-oriented protocol



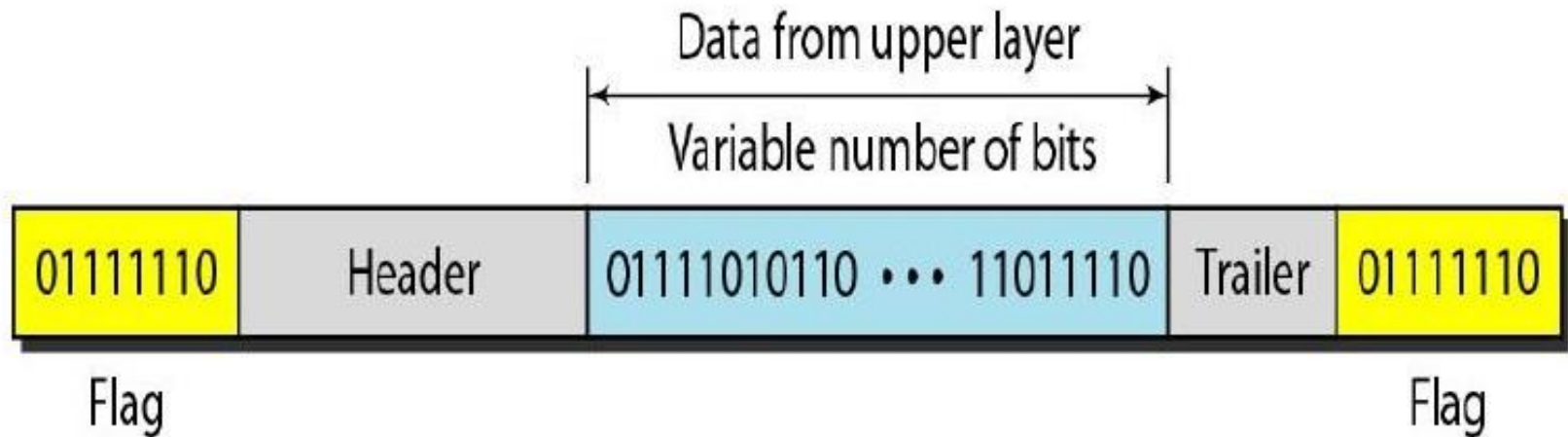
Byte stuffing and unstuffing



Note

Byte stuffing is the process of adding 1 extra byte whenever there is a flag or escape character in the text.

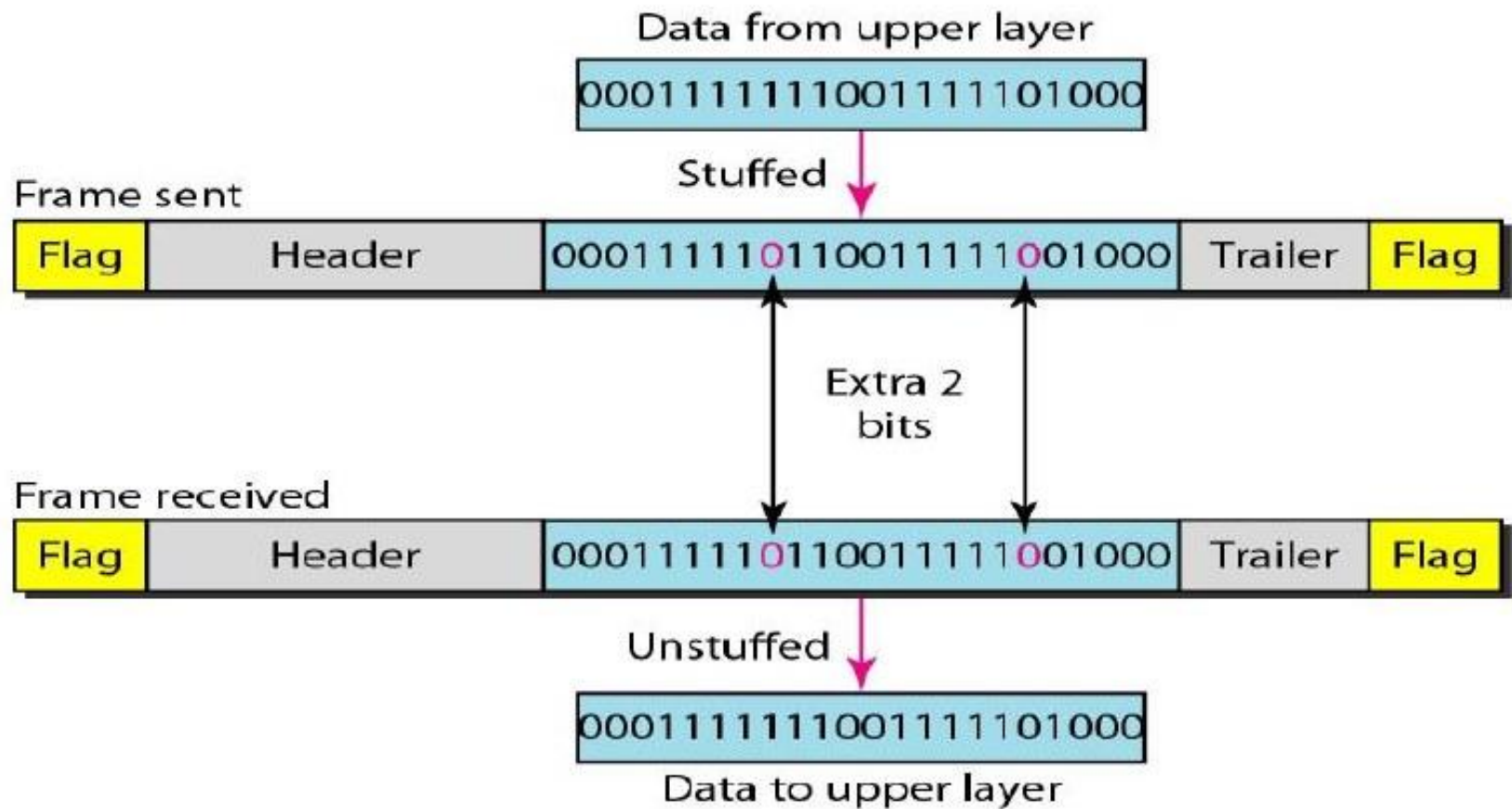
A frame in a bit-oriented protocol



Note

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

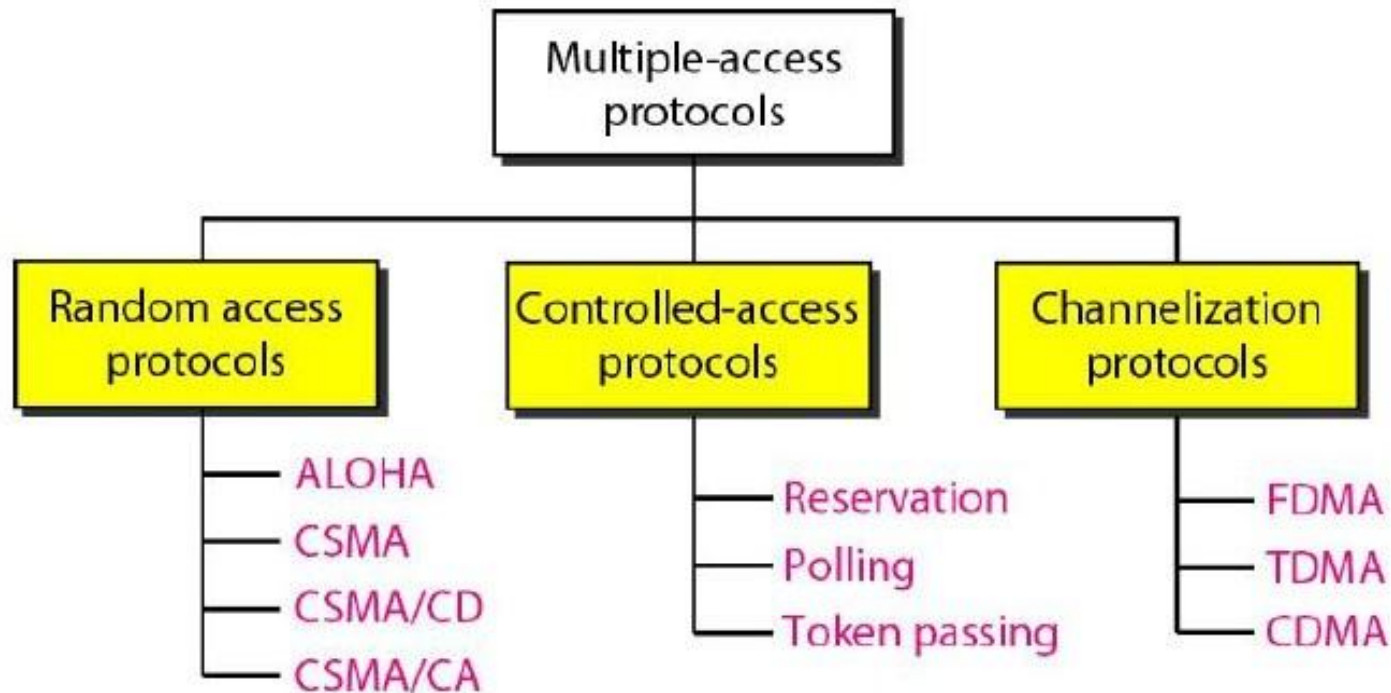
Bit stuffing and unstuffing



Media Access Control

- The transmission medium is shared between the stations
 - When multiple stations send data at the same time, the data from those stations collide and get corrupted or destroyed
 - How to coordinate the access of multiple stations sharing the link???
 - **Solution:** We need a **protocol** to coordinate such transmissions
 - These protocols are called **Medium or Multiple Access Control (MAC) Protocols**
 - What is expected from Multiple Access Protocols:
 - Main task is to minimize the collisions in order to utilize the bandwidth by avoiding the access conflicts
-

Taxonomy of multiple-access protocols



RANDOM ACCESS PROTOCOLS

- Also known as Contention methods.
- In **random access** or **contention** methods, no station is superior to another station and none is assigned the control over another.
- No station permits, or does not permit, another station to send .
- At each instance, a station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send.
- Decision depends on the state of medium – idle or busy.
- The name random access because the stations start the transmission whenever they have the data to send. No rules specify which station should send next.
- The name contention method because the stations need to compete with one another to access the medium.

RANDOM ACCESS PROTOCOLS

➤ If more than one station tries to access the medium, there is an access conflict – collision – and the frames will be either destroyed or modified.

➤ To resolve this, each station follows a procedure that answer the following questions:

- ✓ When can the station access the medium?
- ✓ What can the station do if the medium is busy?
- ✓ How can the station determine the success or failure of the transmission?
- ✓ What can the station do if there is an access conflict?

Topics discussed in this section:

ALOHA – Pure ALOHA and Slotted ALOHA

Carrier Sense Multiple Access

Carrier Sense Multiple Access with Collision Detection

Carrier Sense Multiple Access with Collision Avoidance

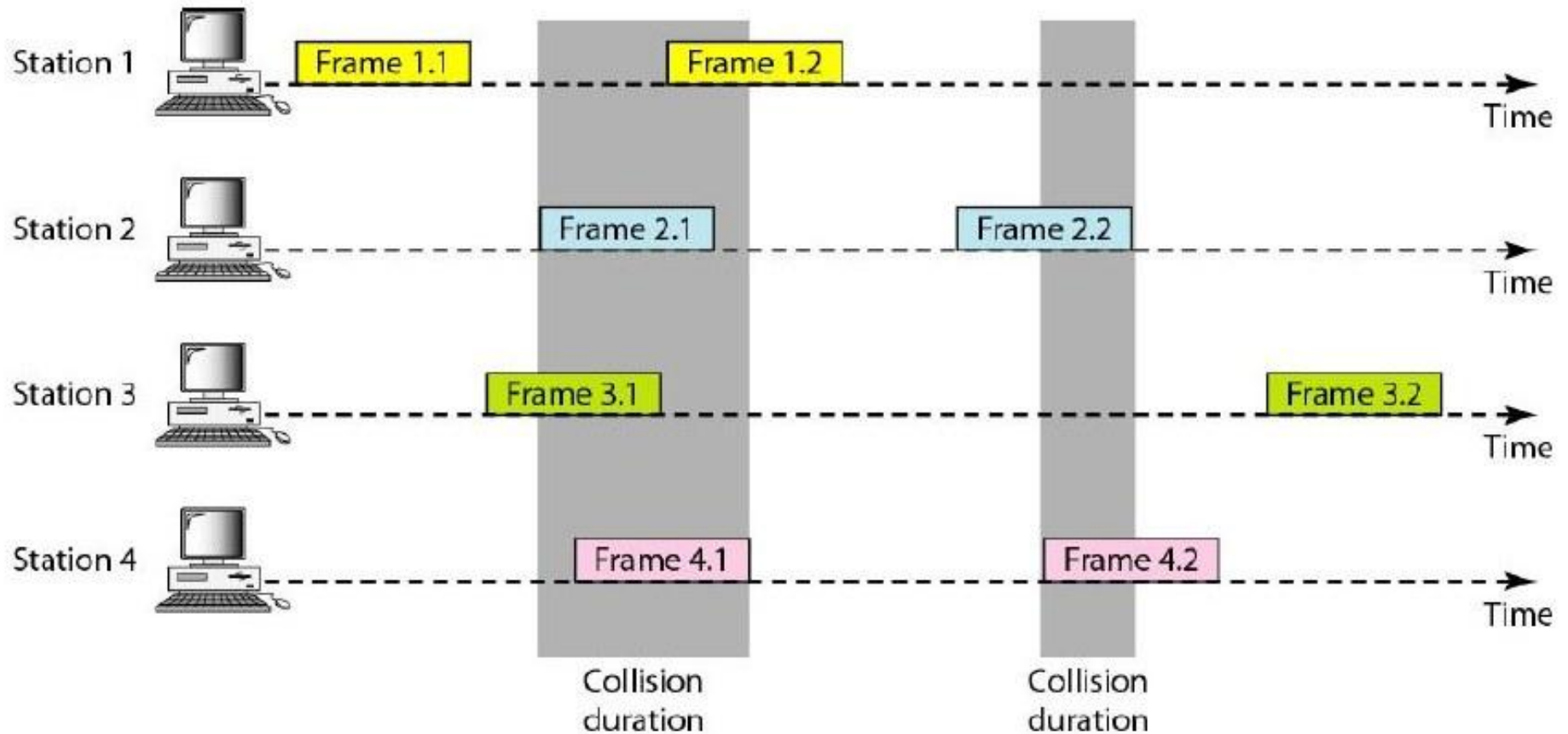
ALOHA

- Earliest random access method
- Developed at the University of Hawaii in early 1970
- Two variants:
 - Pure ALOHA
 - Slotted ALOHA

Pure ALOHA

- Simple and elegant protocol
- Idea is that each station sends a frame whenever it has a frame to send
- However, because of channel sharing, there is a possibility of collision between frames from different stations

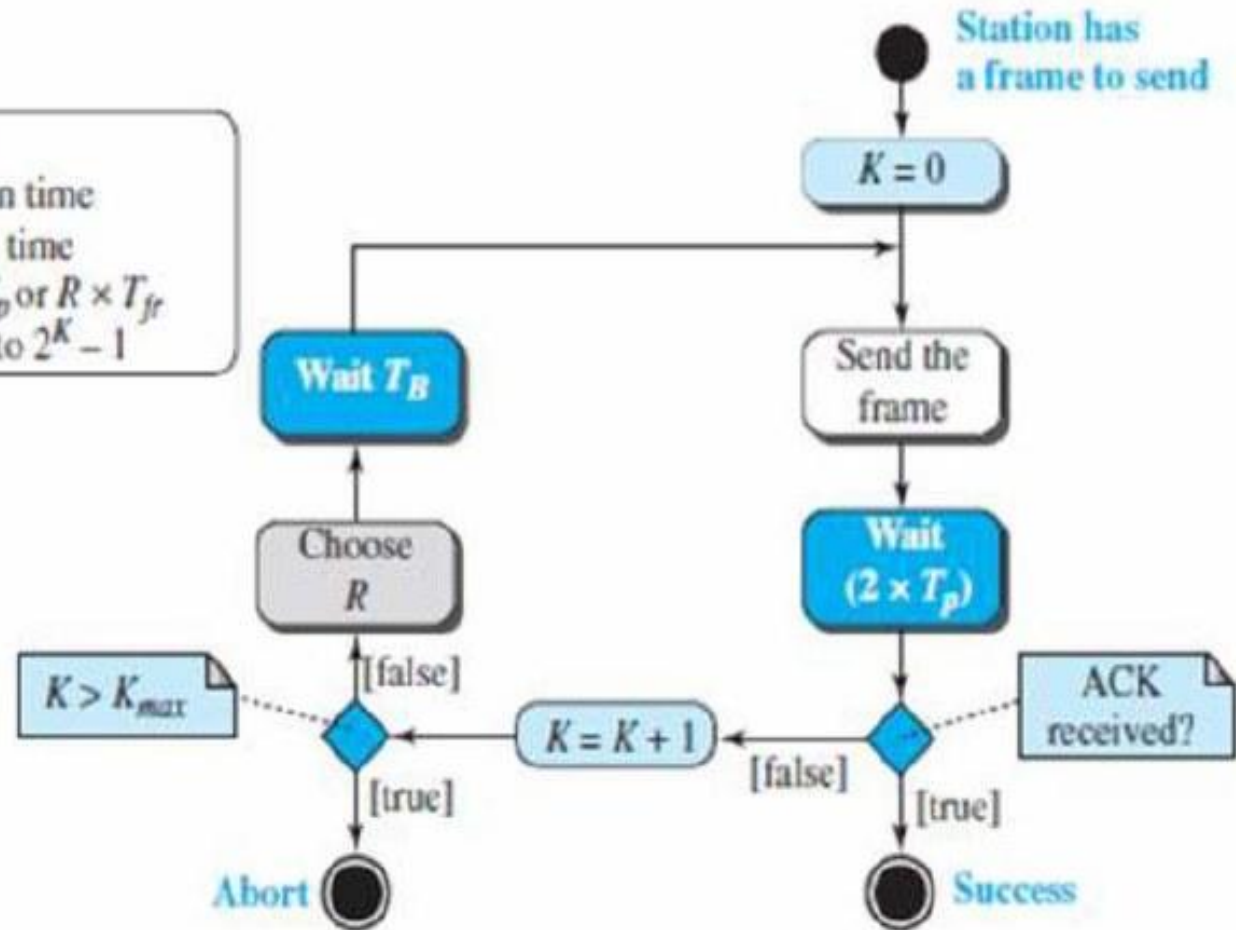
Frames in a pure ALOHA network



Procedure for pure ALOHA protocol

Legend

K : Number of attempts
 T_p : Maximum propagation time
 T_{fr} : Average transmission time
 T_B : (Backoff time): $R \times T_p$ or $R \times T_{fr}$
 R : (Random number): 0 to $2^K - 1$



Example 12.1

The stations on a wireless ALOHA network are a maximum of 600 km apart. If we assume that signals propagate at 3×10^8 m/s, we find

$$T_p = (600 \times 10^3) / (3 \times 10^8) = 2 \text{ ms.}$$

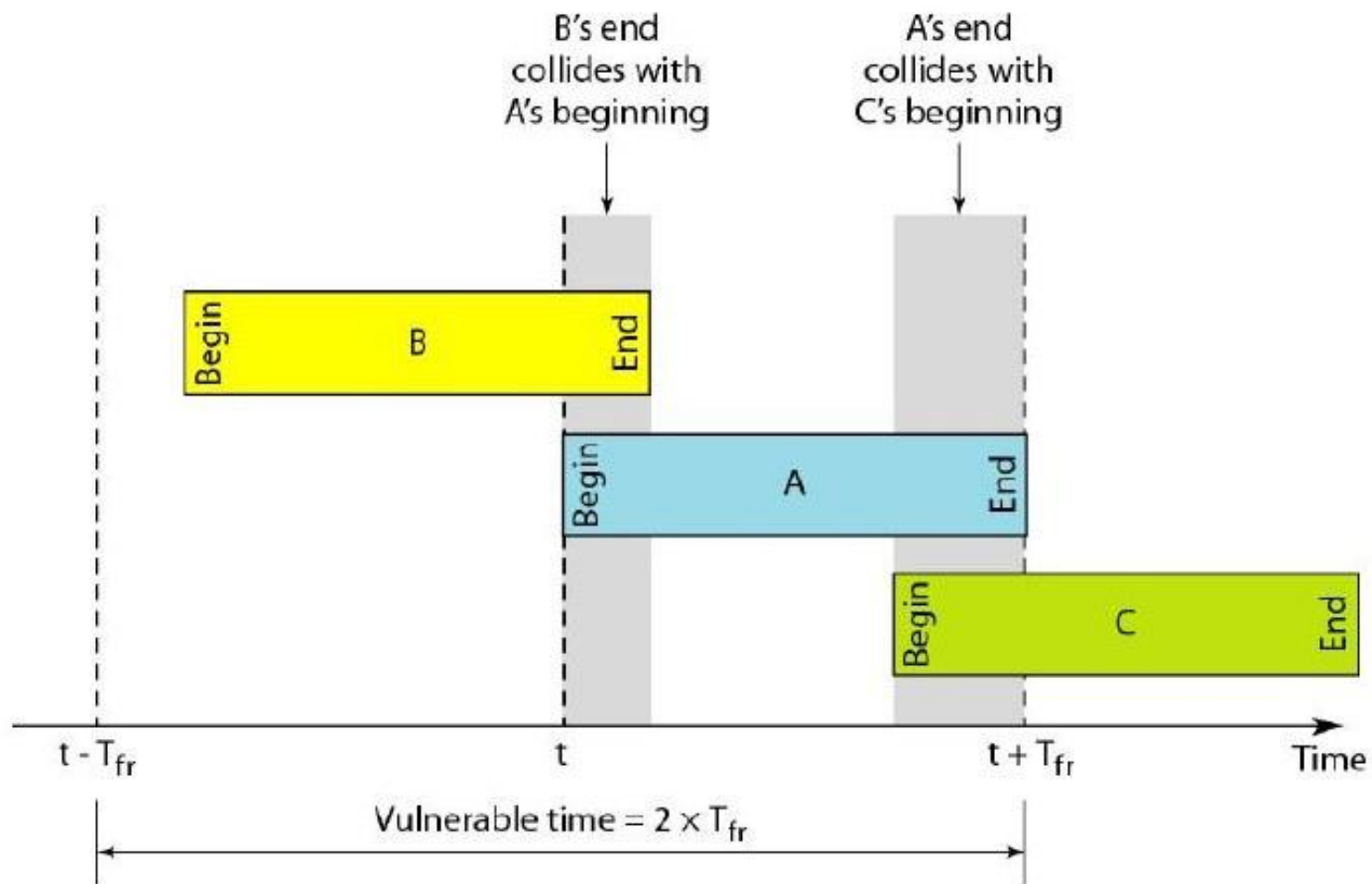
Now we can find the value of T_B for different values of K .

- a. For $K = 1$, the range is $\{0, 1\}$. The station needs to generate a random number with a value of 0 or 1. This means that T_B is either 0 ms (0×2) or 2 ms (1×2), based on the outcome of the random variable.*

Example 12.1 (continued)

- b.** For $K = 2$, the range is $\{0, 1, 2, 3\}$. This means that T_B can be 0, 2, 4, or 6 ms, based on the outcome of the random variable.*
- c.** For $K = 3$, the range is $\{0, 1, 2, 3, 4, 5, 6, 7\}$. This means that T_B can be 0, 2, 4, $\dots$, 14 ms, based on the outcome of the random variable.*

Vulnerable time for pure ALOHA protocol



Example 12.2

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. Calculate the vulnerable time?

Solution

Average frame transmission time

$$T_{fr} = 200 \text{ bits} / 200 \text{ kbps} \text{ or } 1 \text{ ms.}$$

$$\text{The vulnerable time} = 2 \times T_{fr} = 2 \times 1 \text{ ms} = 2 \text{ ms.}$$

Note

The throughput for pure ALOHA is

$$S = G \times e^{-2G} .$$

The maximum throughput

$$S_{\max} = 0.184 \text{ when } G = (1/2).$$

S = Throughput

G = Average number of frames generated by a station during one frame transmission time (Number of frames generated in $1 T_{fr}$)

Example 12.3

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system (all stations together) produces

- a.** 1000 frames per second **b.** 500 frames per second*
- c.** 250 frames per second.*

Solution

The frame transmission time (T_{fr}) is 200/200 kbps or 1 ms.

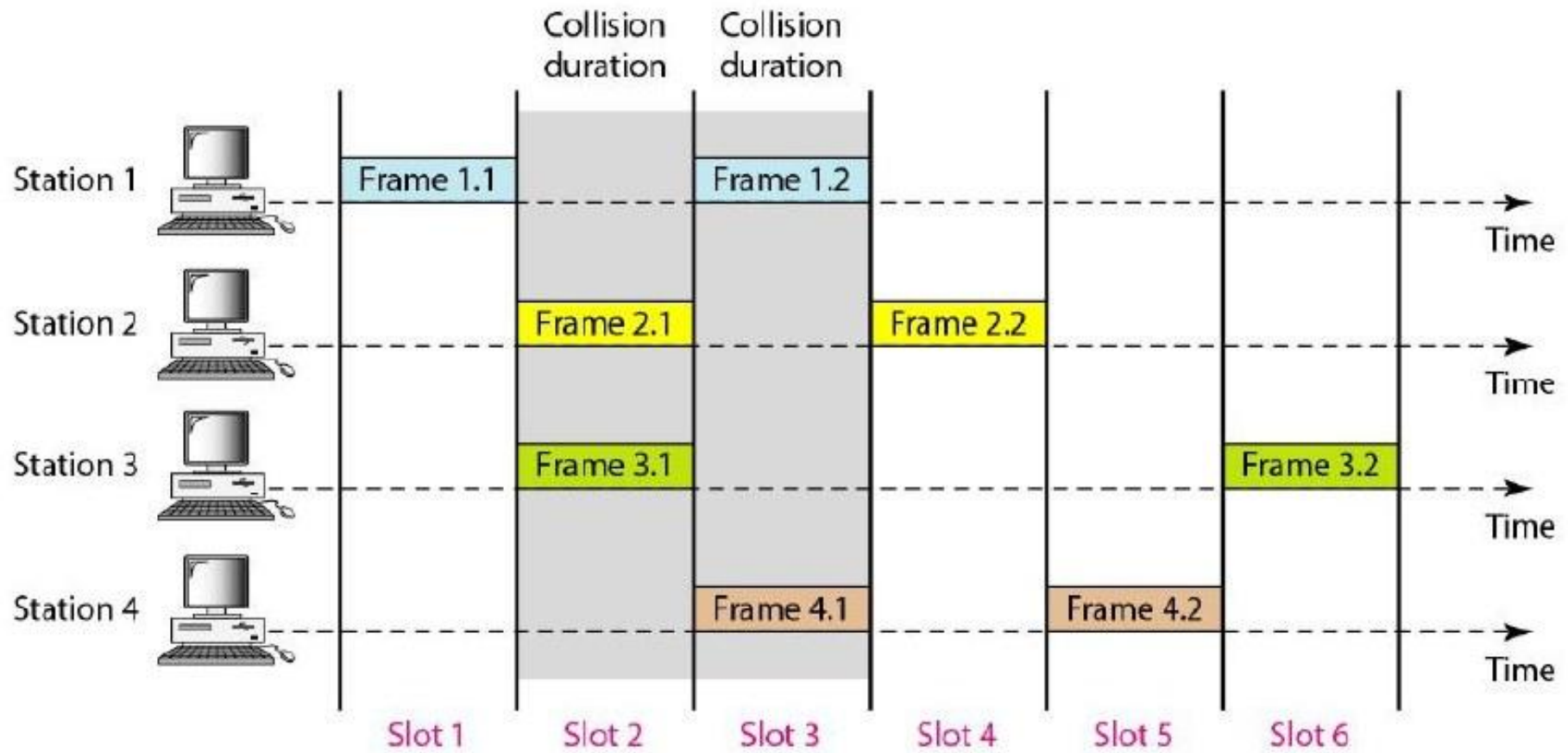
- a.** If the system creates 1000 frames per second, this is 1 frame per millisecond. The load is 1. In this case $S = G \times e^{-2G}$ or $S = 0.135$ (13.5 percent). This means that the throughput is $1000 \times 0.135 = 135$ frames. Only 135 frames out of 1000 will probably survive.*

Example 12.3 (continued)

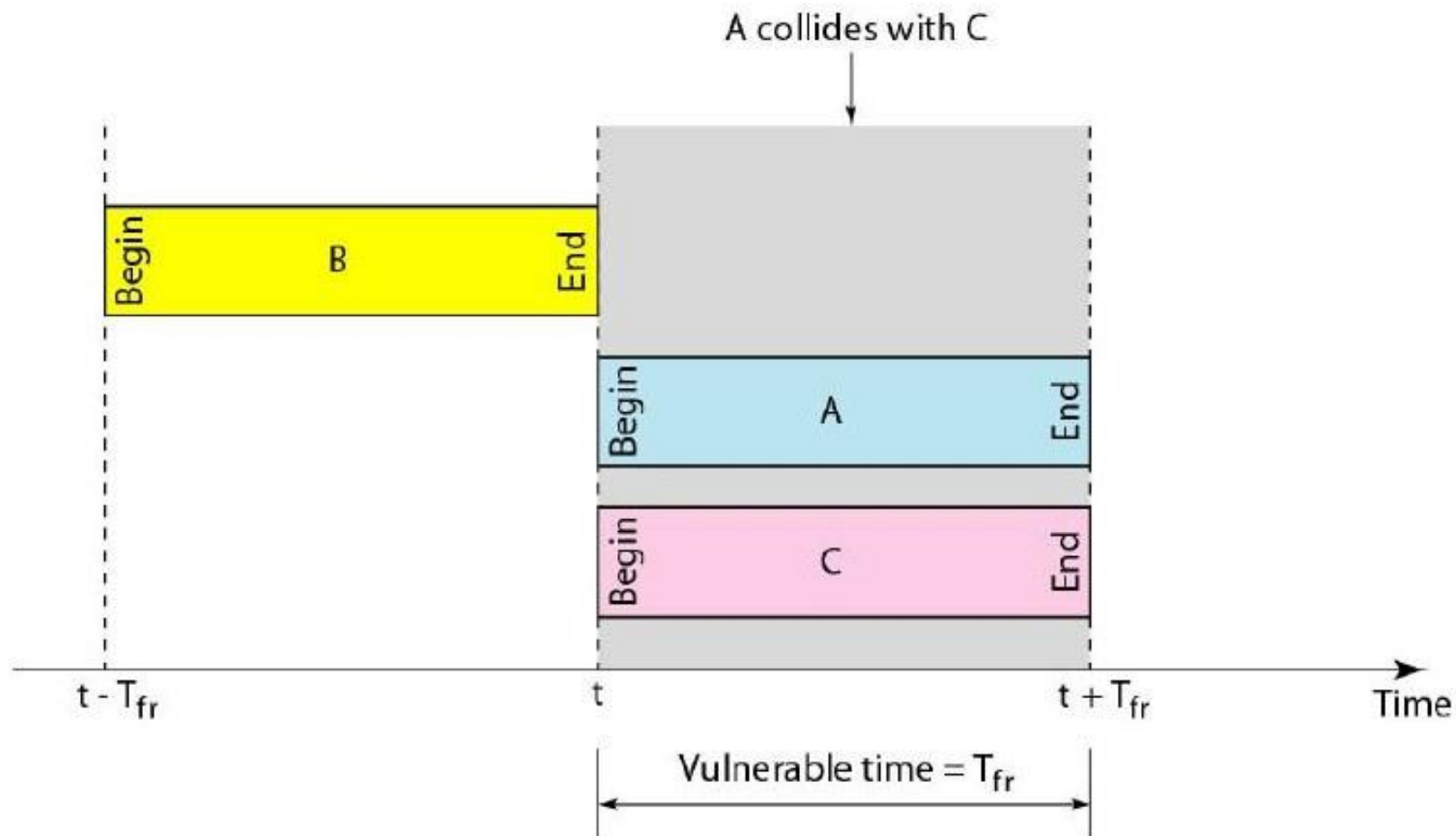
- b. If the system creates 500 frames per second, this is (1/2) frame per millisecond. The load is (1/2). In this case $S = G \times e^{-2G}$ or $S = 0.184$ (18.4 percent). This means that the throughput is $500 \times 0.184 = 92$ and that only 92 frames out of 500 will probably survive. Note that this is the maximum throughput case, percentagewise.*
- c. If the system creates 250 frames per second, this is (1/4) frame per millisecond. The load is (1/4). In this case $S = G \times e^{-2G}$ or $S = 0.152$ (15.2 percent). This means that the throughput is $250 \times 0.152 = 38$. Only 38 frames out of 250 will probably survive.*

Slotted ALOHA

Frames in a slotted ALOHA network



Vulnerable time for slotted ALOHA protocol



Note

The throughput for slotted ALOHA is

$$S = G \times e^{-G}.$$

The maximum throughput

$$S_{\max} = 0.368 \text{ when } G = 1.$$

S = Throughput

G = Average number of frames generated by a station during one frame transmission time (Number of frames generated in $1 T_{fr}$)

Example 12.4

A slotted ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system (all stations together) produces

- a. 1000 frames per second b. 500 frames per second*
- c. 250 frames per second.*

Solution

The frame transmission time (T_{fr}) is 200/200 kbps or 1 ms.

- a. If the system creates 1000 frames per second, this is 1 frame per millisecond. The load is 1. In this case $S = G \times e^{-G}$ or $S = 0.368$ (36.8 percent). This means that the throughput is $1000 \times 0.0368 = 368$ frames. Only 386 frames out of 1000 will probably survive.*

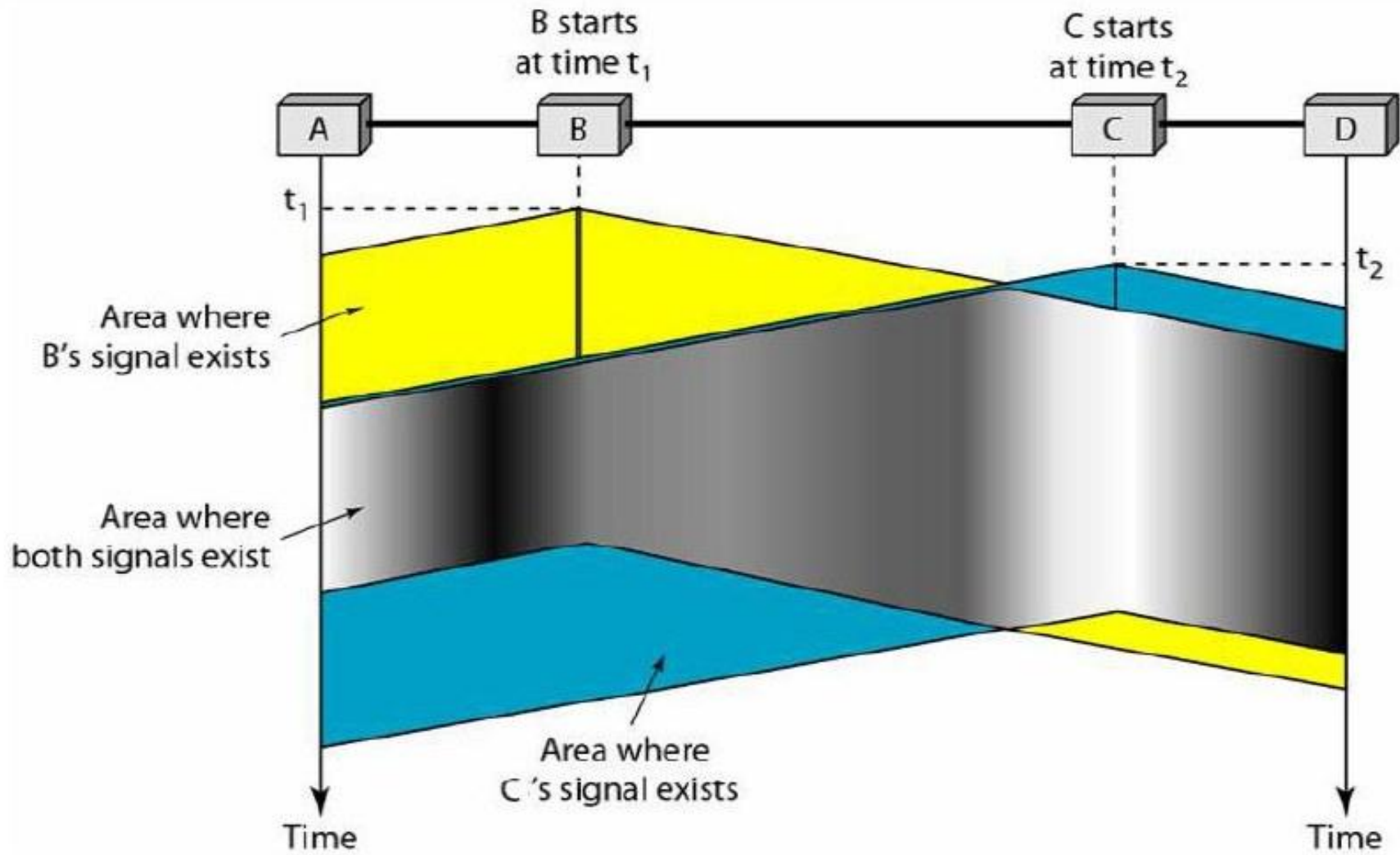
Example 12.4 (continued)

- b.** *If the system creates 500 frames per second, this is (1/2) frame per millisecond. The load is (1/2). In this case $S = G \times e^{-G}$ or $S = 0.303$ (30.3 percent). This means that the throughput is $500 \times 0.303 = 151$. Only 151 frames out of 500 will probably survive.*
- c.** *If the system creates 250 frames per second, this is (1/4) frame per millisecond. The load is (1/4). In this case $S = G \times e^{-G}$ or $S = 0.195$ (19.5 percent). This means that the throughput is $250 \times 0.195 = 49$. Only 49 frames out of 250 will probably survive.*

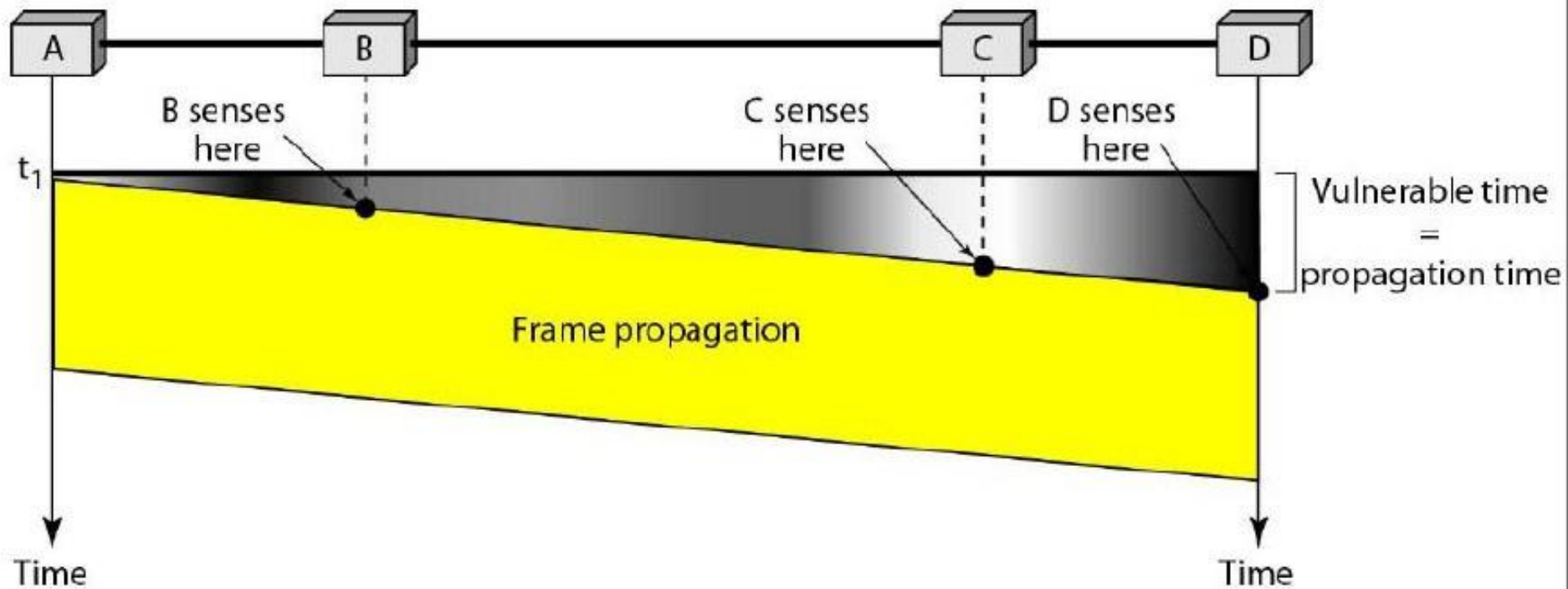
Carrier Sense Multiple Access - CSMA

- To minimize the chance of collision and increase the performance, CSMA method was developed
- Chance of collision can be reduced if a station senses the medium before trying to use it
- CSMA requires that each station first sense the medium to check its state (busy or idle) before sending the frame
- Based on the principle – “sense before transmit” or (listen before talk”
- Reduces the possibility of collision but cannot eliminate it
- Two variants:
 - CSMA /CD
 - CSMA /CA

Space/time model of the collision in CSMA

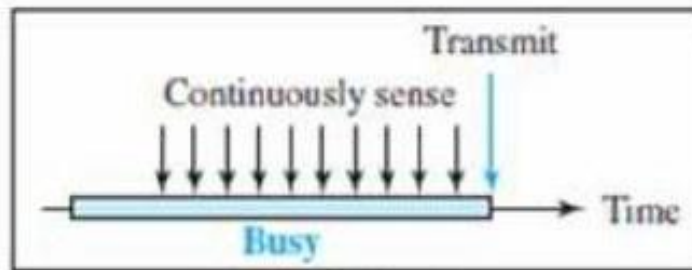


Vulnerable time in CSMA

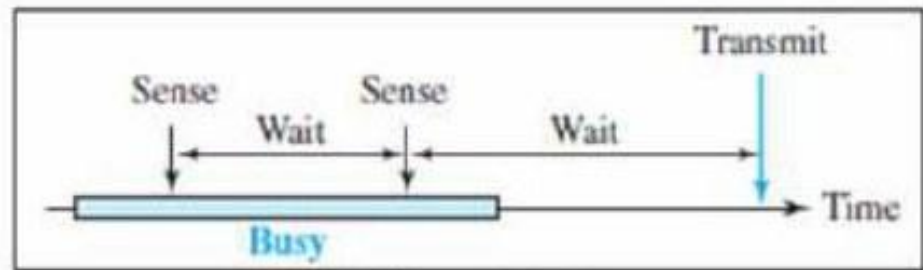


Behavior of three persistence methods

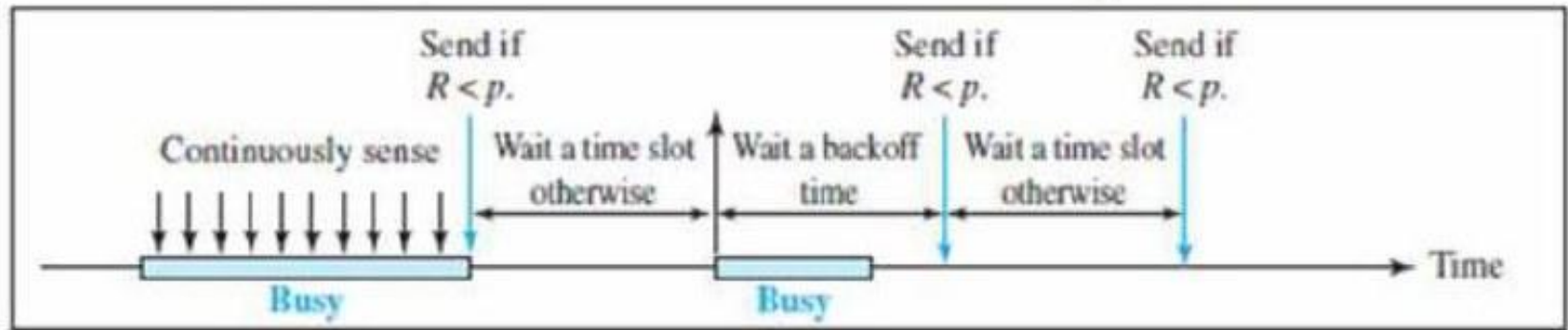
- What should a station do if the channel is busy??
- What should a station do if the channel is idle??
- Answered by persistence methods
- There are 3 persistence methods defined by CSMA



a. 1-Persistent

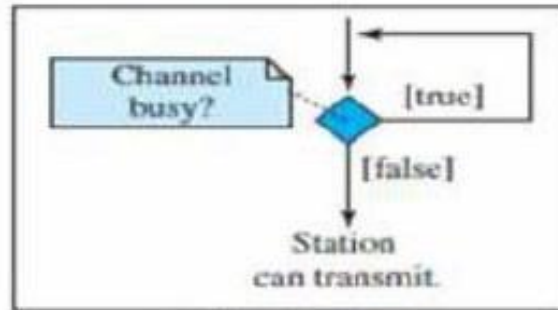


b. Nonpersistent

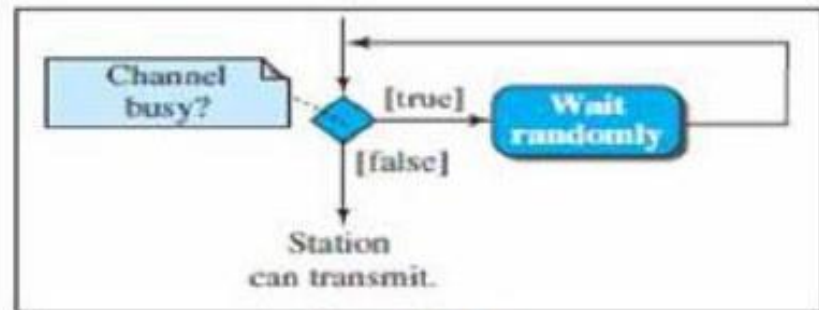


c. p-Persistent

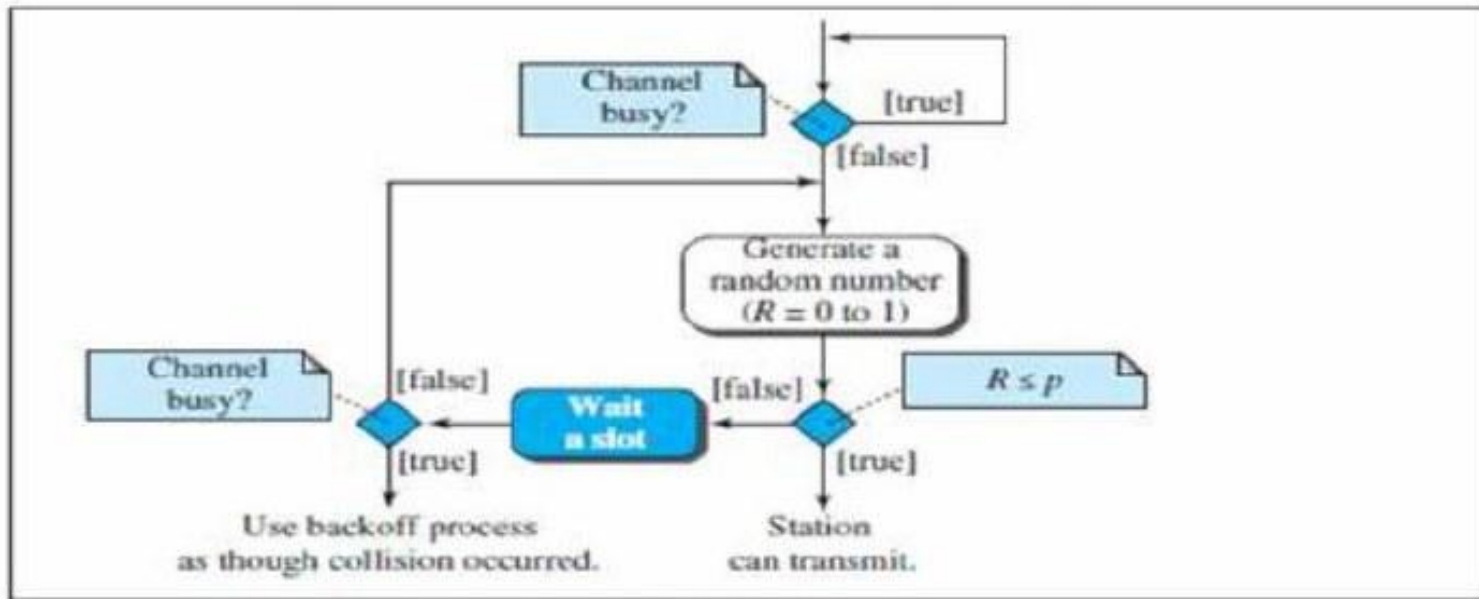
Flow diagram for three persistence methods



a. 1-Persistent



b. Nonpersistent

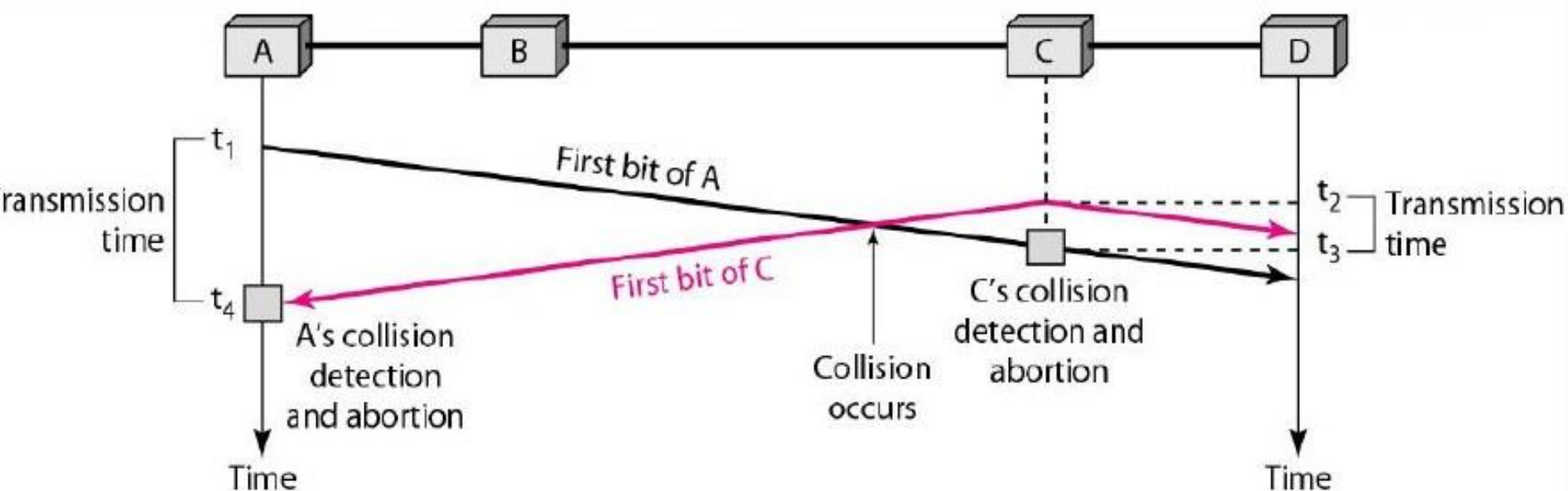


c. p-Persistent

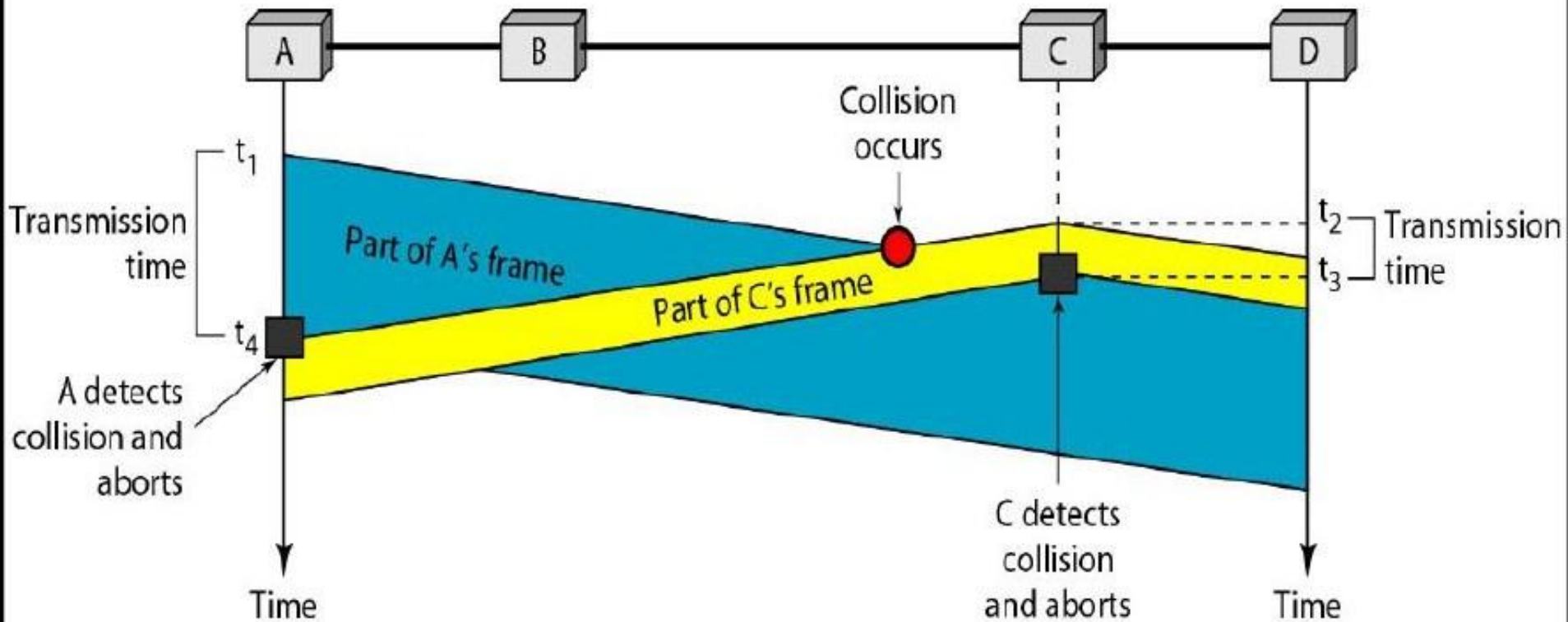
CSMA / CD

- CSMA does not specify the procedure following a collision
- CSMA / CD augments the algorithm to handle the collision
- In case of collision, the current transmission is aborted and the frame is resent

Collision of the first bit in CSMA/CD



Collision and abortion in CSMA/CD



Minimum Frame Size

- There is a restriction the frame size in CSMA / CD
- Minimum frame size = Bandwidth X T_{fr}
- $T_{fr} = 2 \times T_p$ for CSMA / CD

Example 12.5

A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including the delays in the devices and ignoring the time needed to send a jamming signal, as we see later) is $25.6 \mu\text{s}$, what is the minimum size of the frame?

Solution

The frame transmission time is $T_{fr} = 2 \times T_p = 51.2 \mu\text{s}$. This means, in the worst case, a station needs to transmit for a period of $51.2 \mu\text{s}$ to detect the collision. The minimum size of the frame is $10 \text{ Mbps} \times 51.2 \mu\text{s} = 512$ bits or 64 bytes. This is actually the minimum size of the frame for Standard Ethernet.

Another Example

Calculate the minimum frame length in network using CSMA/CD method for channel access having a bandwidth of 20Mbps, and signal speed same as light with end to end stations 500km apart.

Solution

$$\text{Light speed} = 3 \times 10^8 \text{ m/s} = 3 \times 10^5 \text{ km/s} = 300000 \text{ km/s}$$

$$T_p = \text{distance} / \text{speed} = 500\text{km} / (300000 \text{ km/s}) = 1.66\text{ms}$$

$$T_{fr} = 2 \times T_p = 2 \times 1.66\text{ms} = 3.32\text{ms}$$

$$\begin{aligned} \text{Minimum frame size} &= \text{bandwidth} \times T_{fr} \\ &= 20\text{Mbps} \times 3.32\text{ms} \\ &= 2.5\text{MBytes} \times 0.00332 \\ &= 0.008\text{MBytes} = 8000\text{bytes} \end{aligned}$$

Flow diagram for the CSMA/CD

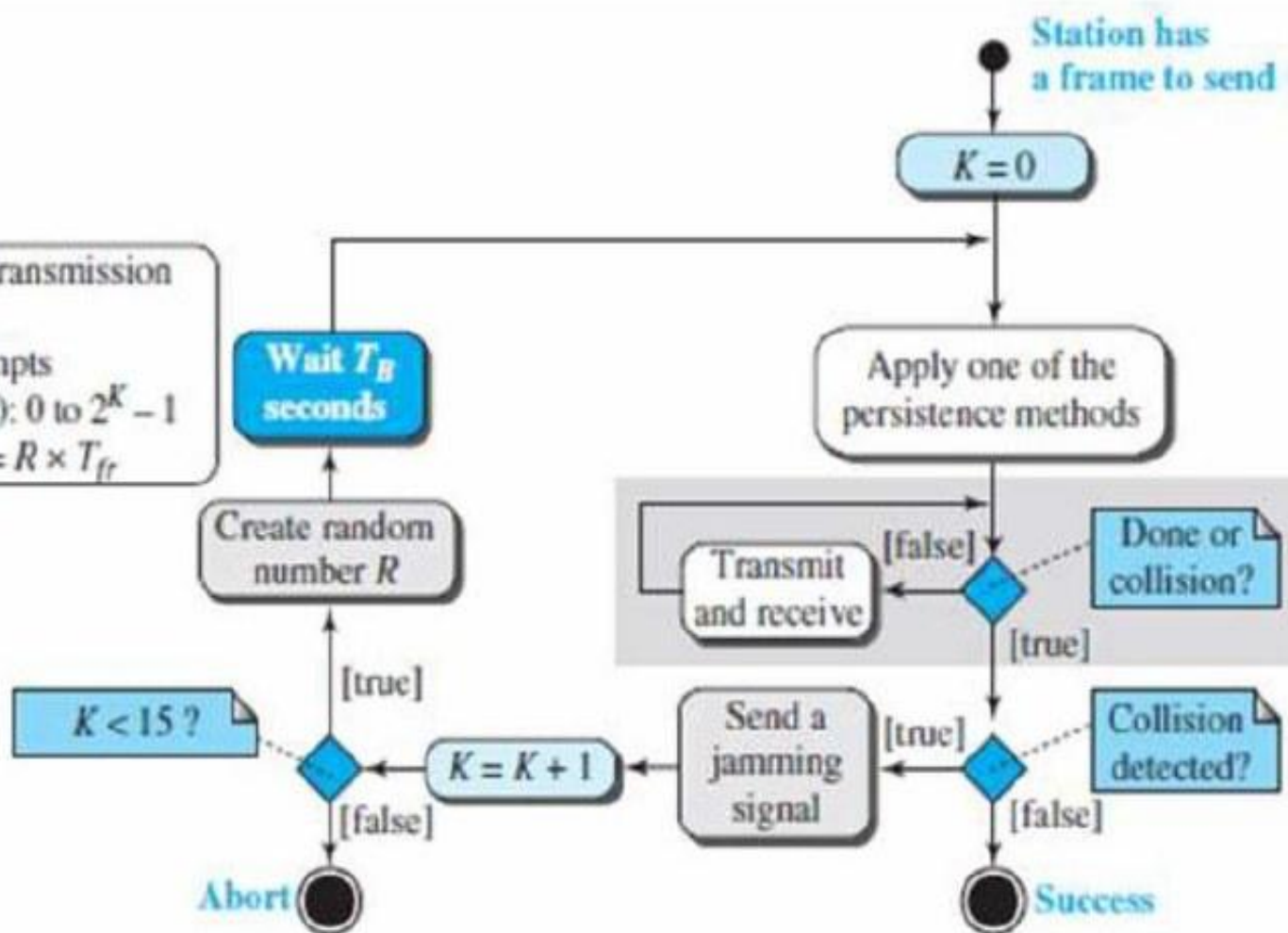
Legend

T_{fr} : Frame average transmission time

K : Number of attempts

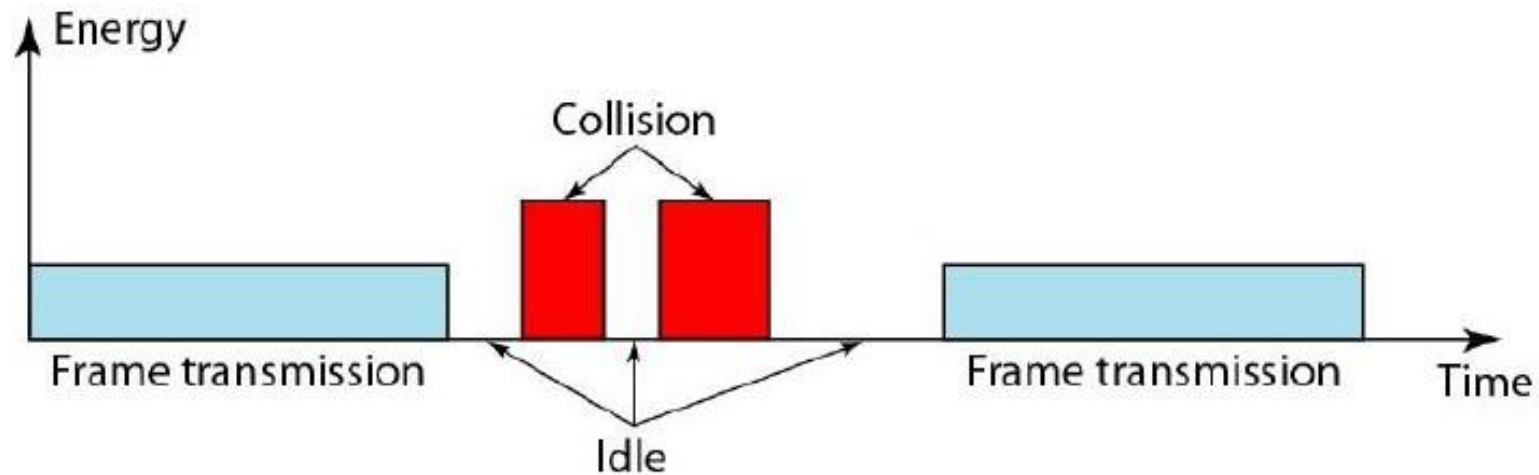
R : (random number): 0 to $2^K - 1$

T_B : (Backoff time) = $R \times T_{fr}$



Energy level during transmission, idleness, or collision

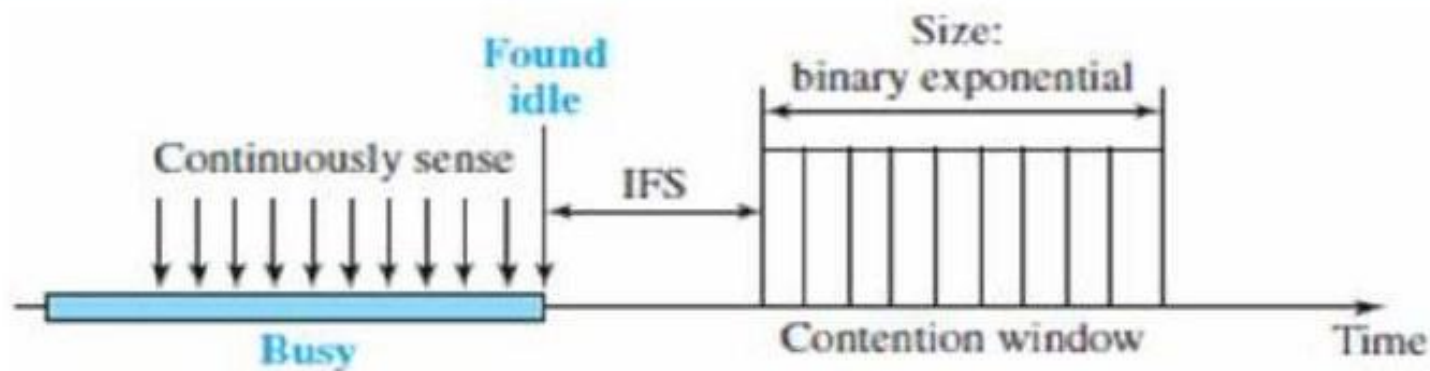
- Level of energy in a channel has three values
- Zero: channel is idle
- Normal: station has successfully captured the channel and is transmitting its frame
- Abnormal: there is collision and the energy level is twice the normal level



CSMA / CA

- Carrier sense multiple access with collision avoidance
- Uses three strategies
 - Interframe Space (IFS)
 - Contention Window
 - Acknowledgements

Contention window



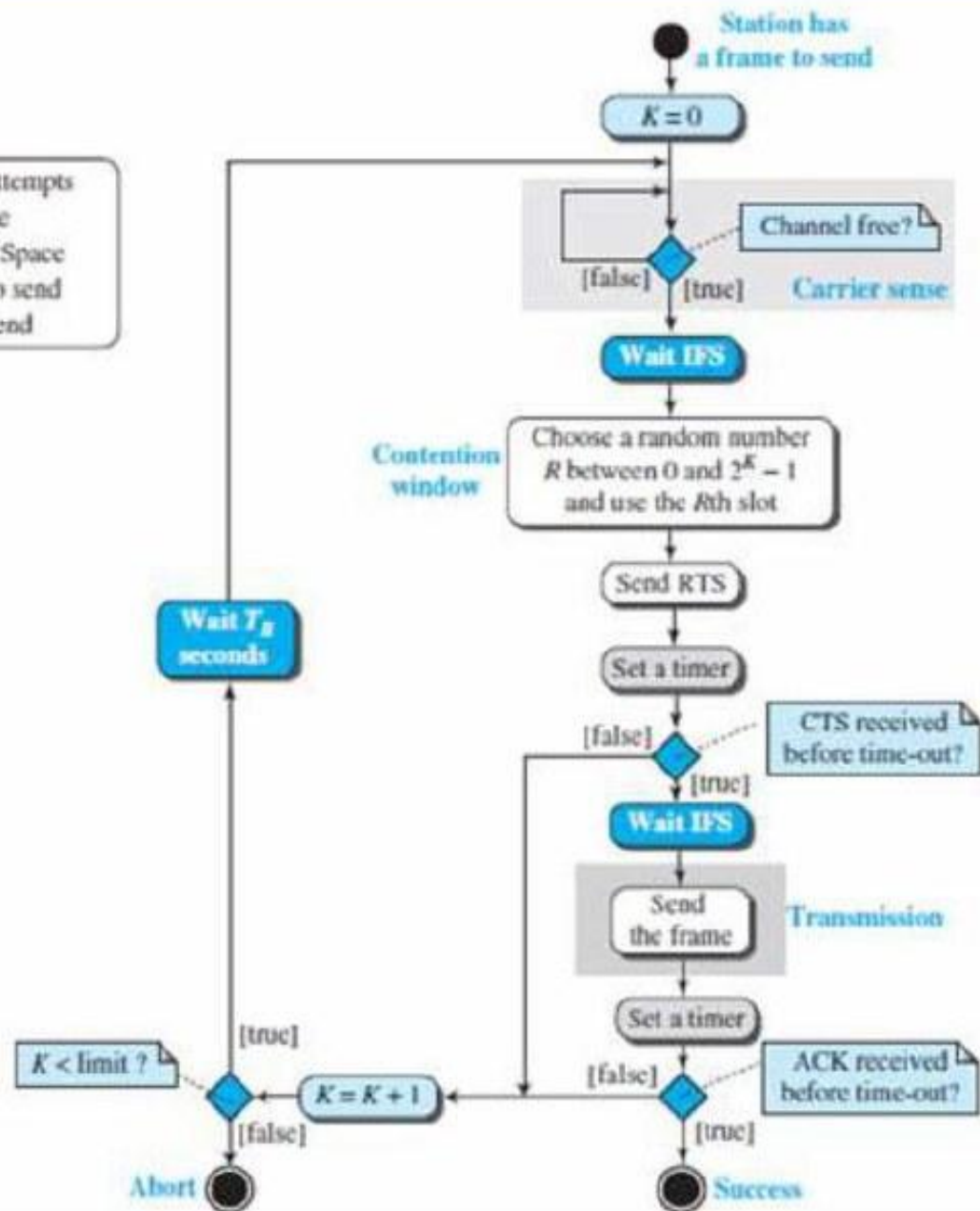
Note

In CSMA/CA, the IFS can also be used to define the priority of a station or a frame.

Flow diagram for CSMA/CA

Legend

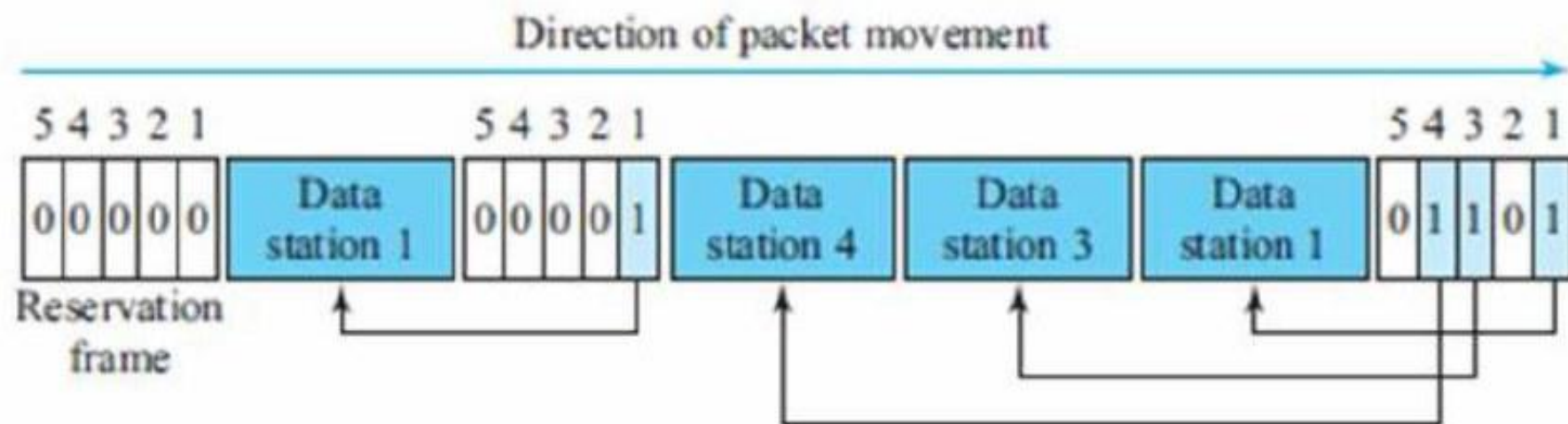
K : Number of attempts
 T_B : Backoff time
 IFS: Interframe Space
 RTS: Request to send
 CTS: Clear to send
 CTS: Clear to send



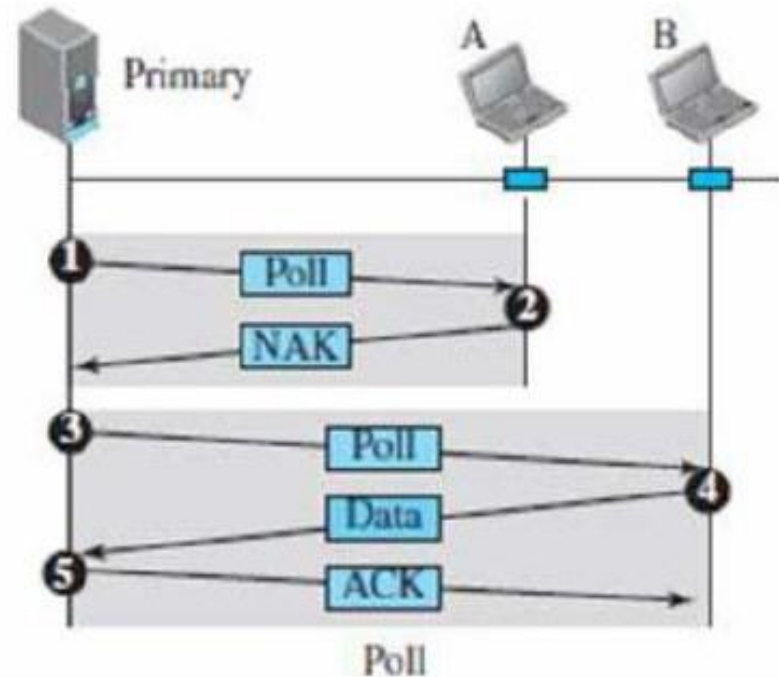
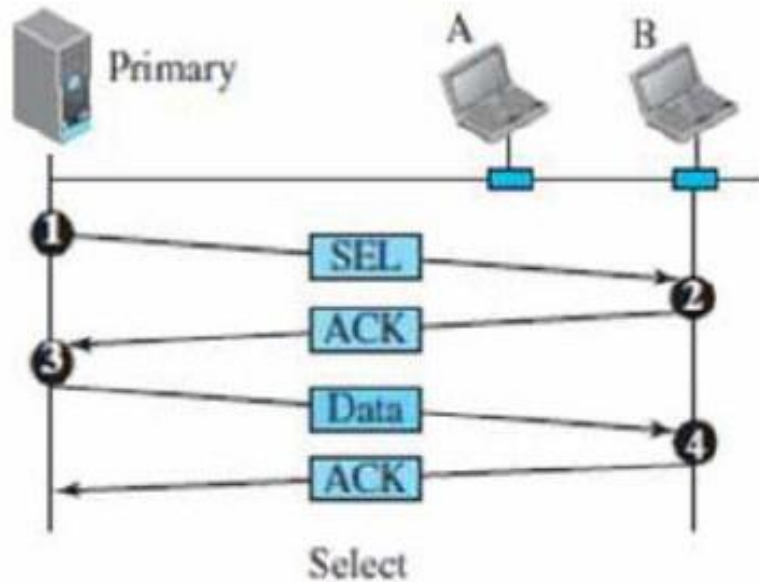
CONTROLLED ACCESS

- In **controlled access**, the stations consult one another to find which station has the right to send.
- A station cannot send unless it has been authorized by other stations.
- Three controlled-access methods.
 - ✓ **Reservation**
 - ✓ **Polling**
 - ✓ **Token Passing**

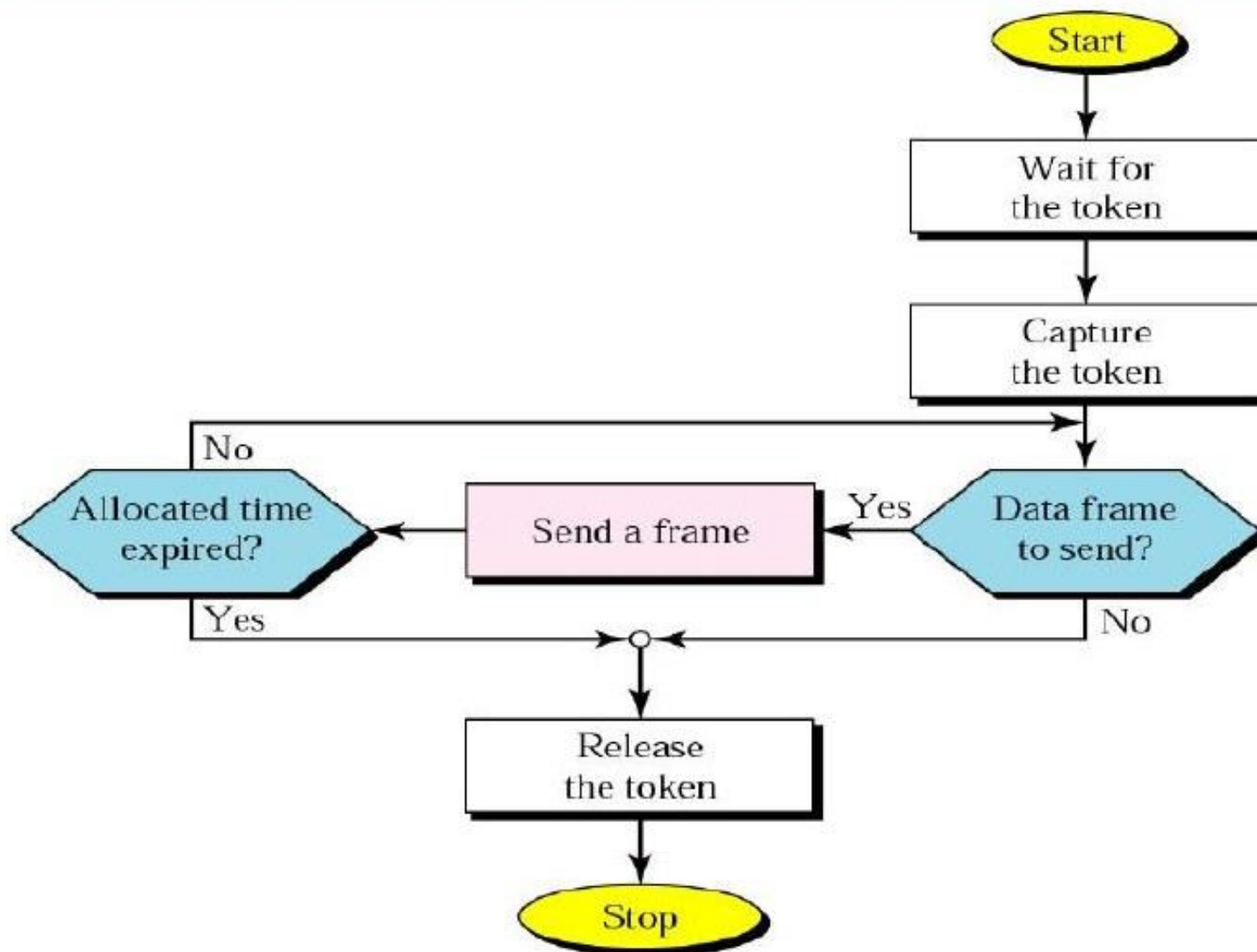
Reservation access method



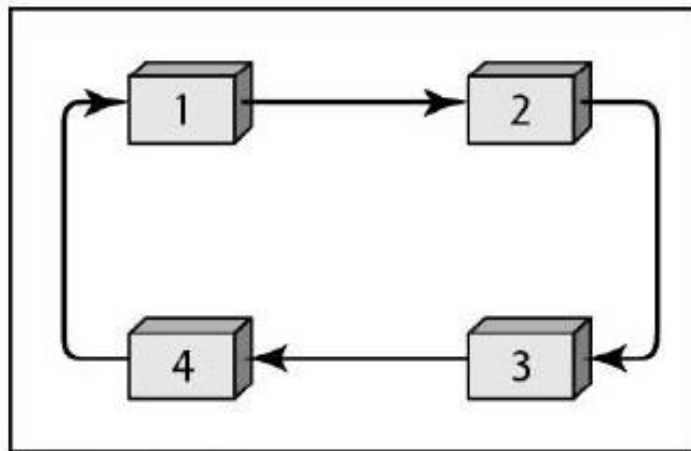
Select and poll functions in polling access method



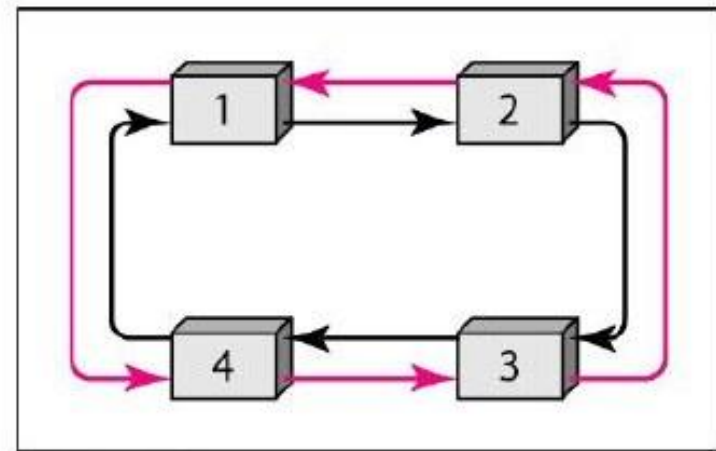
Procedure in token-passing access method



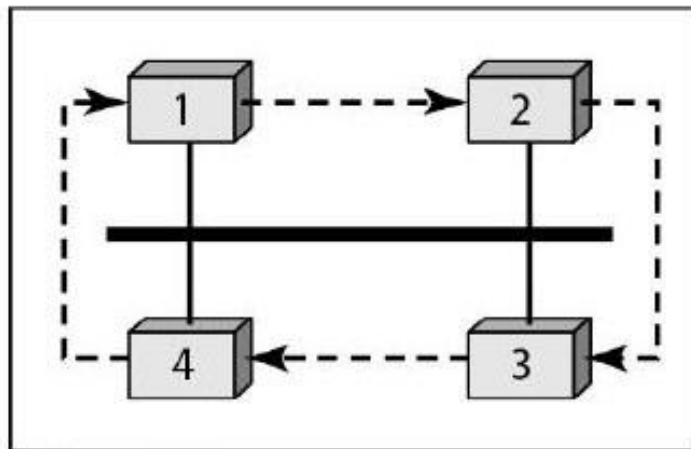
Logical ring and physical topology in token-passing access method



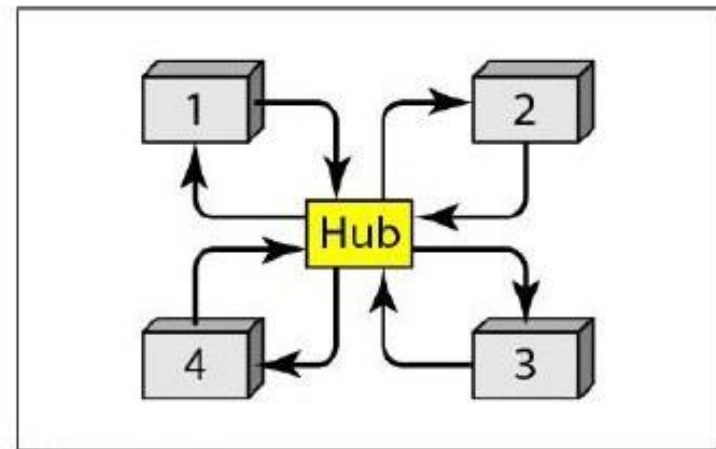
a. Physical ring



b. Dual ring



c. Bus ring



d. Star ring

CHANNELIZATION

Channelization is a multiple-access method in which the available bandwidth of a link is shared in time, frequency, or through code, between different stations.

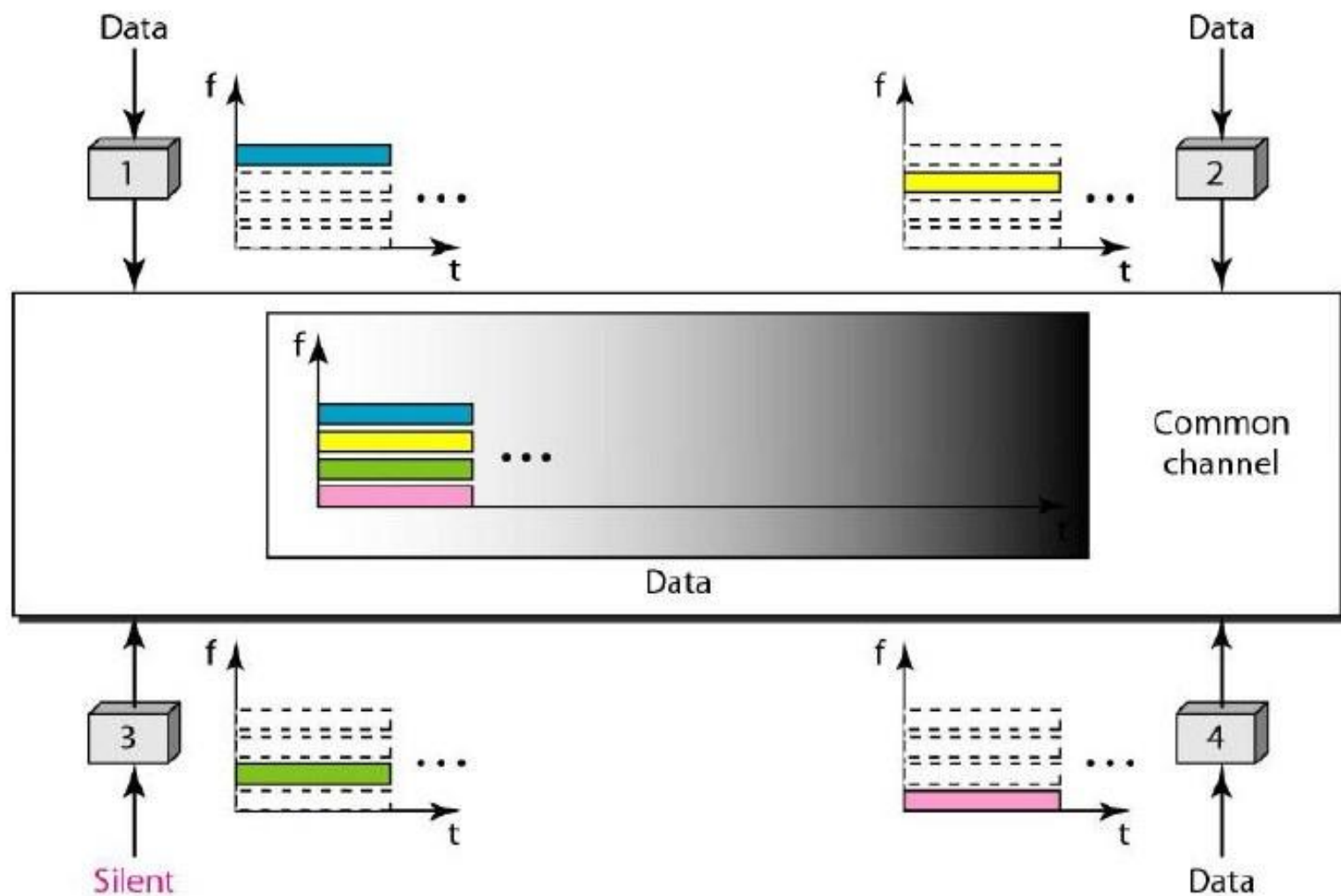
Topics discussed in this section:

Frequency-Division Multiple Access (FDMA)

Time-Division Multiple Access (TDMA)

Code-Division Multiple Access (CDMA)

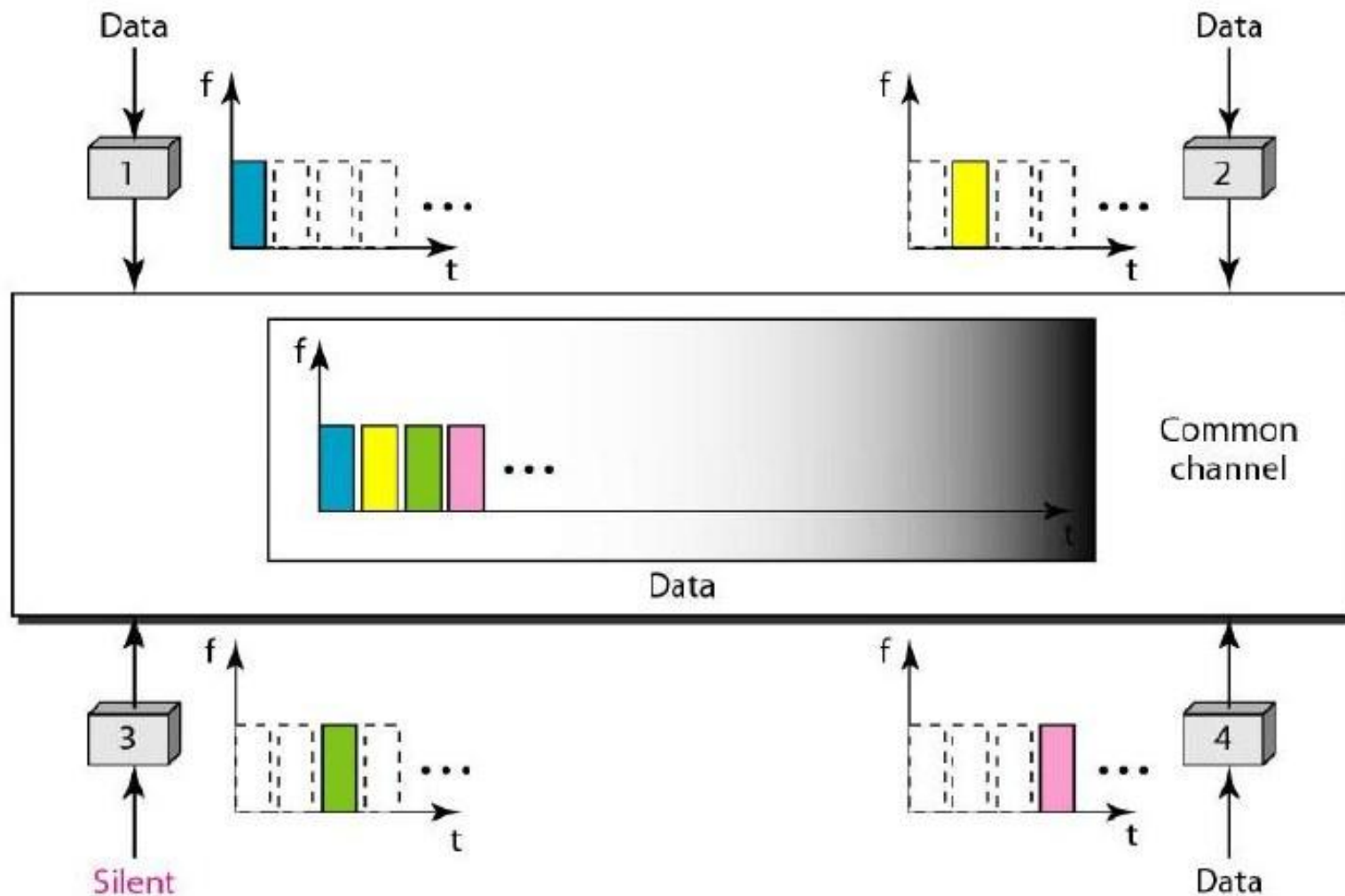
Frequency-division multiple access (FDMA)



Note

In FDMA, the available bandwidth of the common channel is divided into bands that are separated by guard bands.

Time-division multiple access (TDMA)



Note

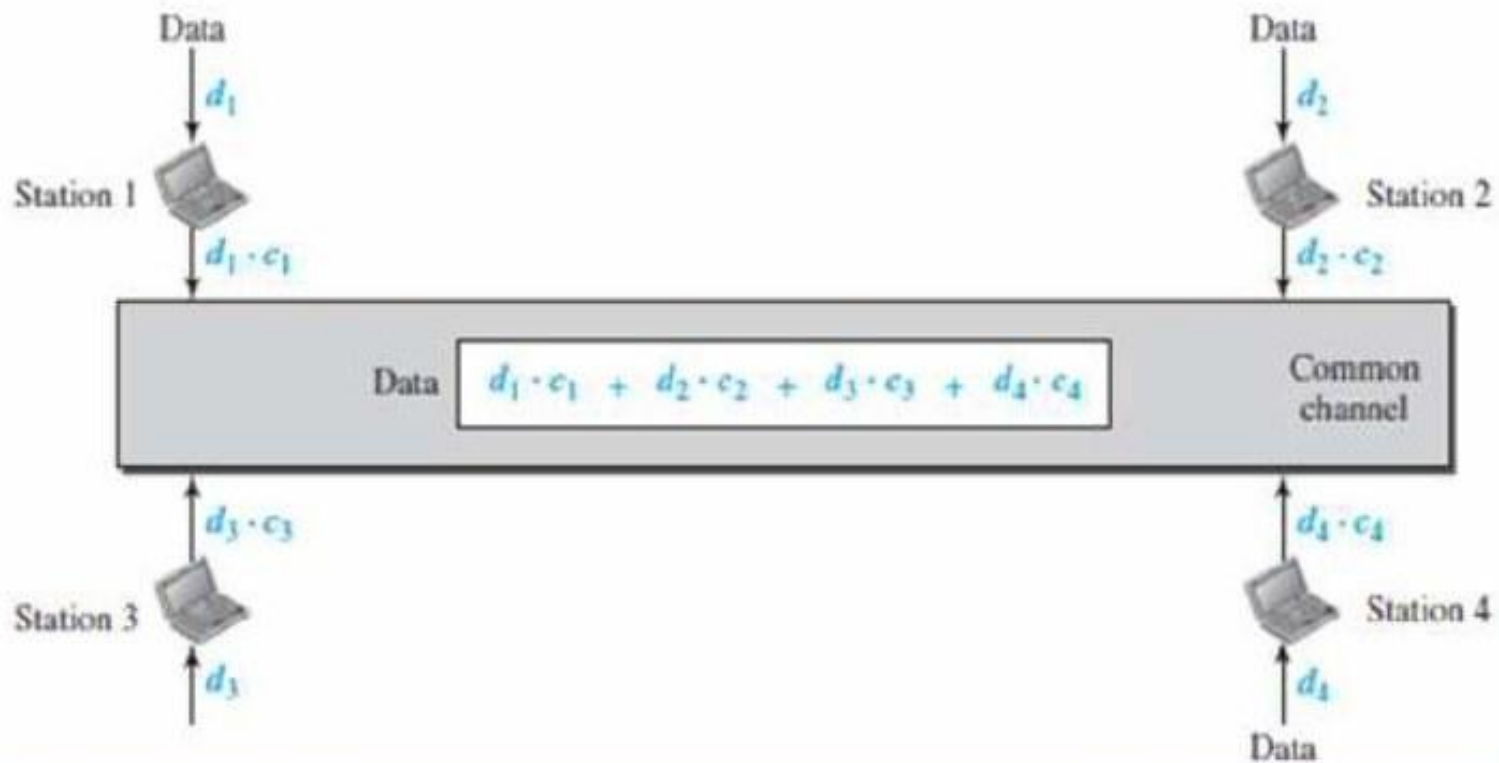
In TDMA, the bandwidth is just one channel that is timeshared between different stations.



Note

In CDMA, one channel carries all transmissions simultaneously.

Simple idea of communication with code



$$\begin{aligned} \text{data} &= (d_1 \cdot c_1 + d_2 \cdot c_2 + d_3 \cdot c_3 + d_4 \cdot c_4) \cdot c_1 \\ &= d_1 \cdot c_1 \cdot c_1 + d_2 \cdot c_2 \cdot c_1 + d_3 \cdot c_3 \cdot c_1 + d_4 \cdot c_4 \cdot c_1 = 4 \times d_1 \end{aligned}$$

Chip sequences

- Also known as orthogonal sequences
- Has 5 major properties
- Generated using Walsh Table (a 2-dimensional table with equal number of rows and columns) with a specific rule to follow

C_1

[+1 +1 +1 +1]

C_2

[+1 -1 +1 -1]

C_3

[+1 +1 -1 -1]

C_4

[+1 -1 -1 +1]

5 major properties of chip sequence

1. Each sequence is made of N elements, where N is the number of stations.
2. If we multiply a sequence by a number, every element in the sequence is multiplied by that element. This is called multiplication of a sequence by a scalar. For example,

$$2 \bullet [+1 +1 -1 -1] = [+2 +2 -2 -2]$$

3. If we multiply two equal sequences, element by element, and add the results, we get N , where N is the number of elements in each sequence. This is called the *inner product* of two equal sequences. For example,

$$[+1 +1 -1 -1] \bullet [+1 +1 -1 -1] = 1 + 1 + 1 + 1 = 4$$

4. If we multiply two different sequences, element by element, and add the results, we get 0. This is called the *inner product* of two different sequences. For example,

$$[+1 +1 -1 -1] \bullet [+1 +1 +1 +1] = 1 + 1 - 1 - 1 = 0$$

5. Adding two sequences means adding the corresponding elements. The result is another sequence. For example,

$$[+1 +1 -1 -1] + [+1 +1 +1 +1] = [+2 +2 \ 0 \ 0]$$

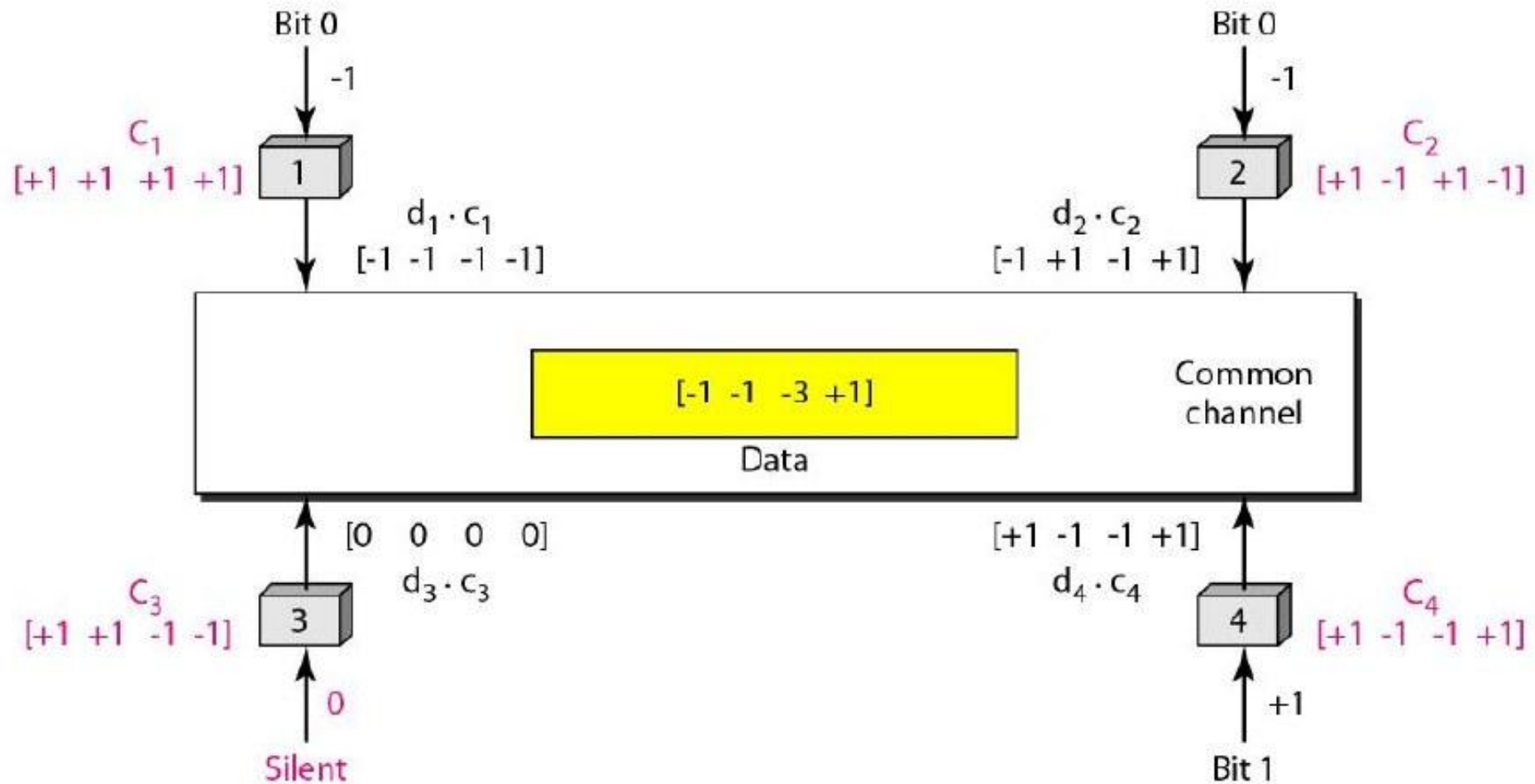
Data representation in CDMA

Data bit 0 $\longrightarrow$ -1

Data bit 1 $\longrightarrow$ +1

Silence $\longrightarrow$ 0

Sharing channel in CDMA



Digital signal created by four stations in CDMA

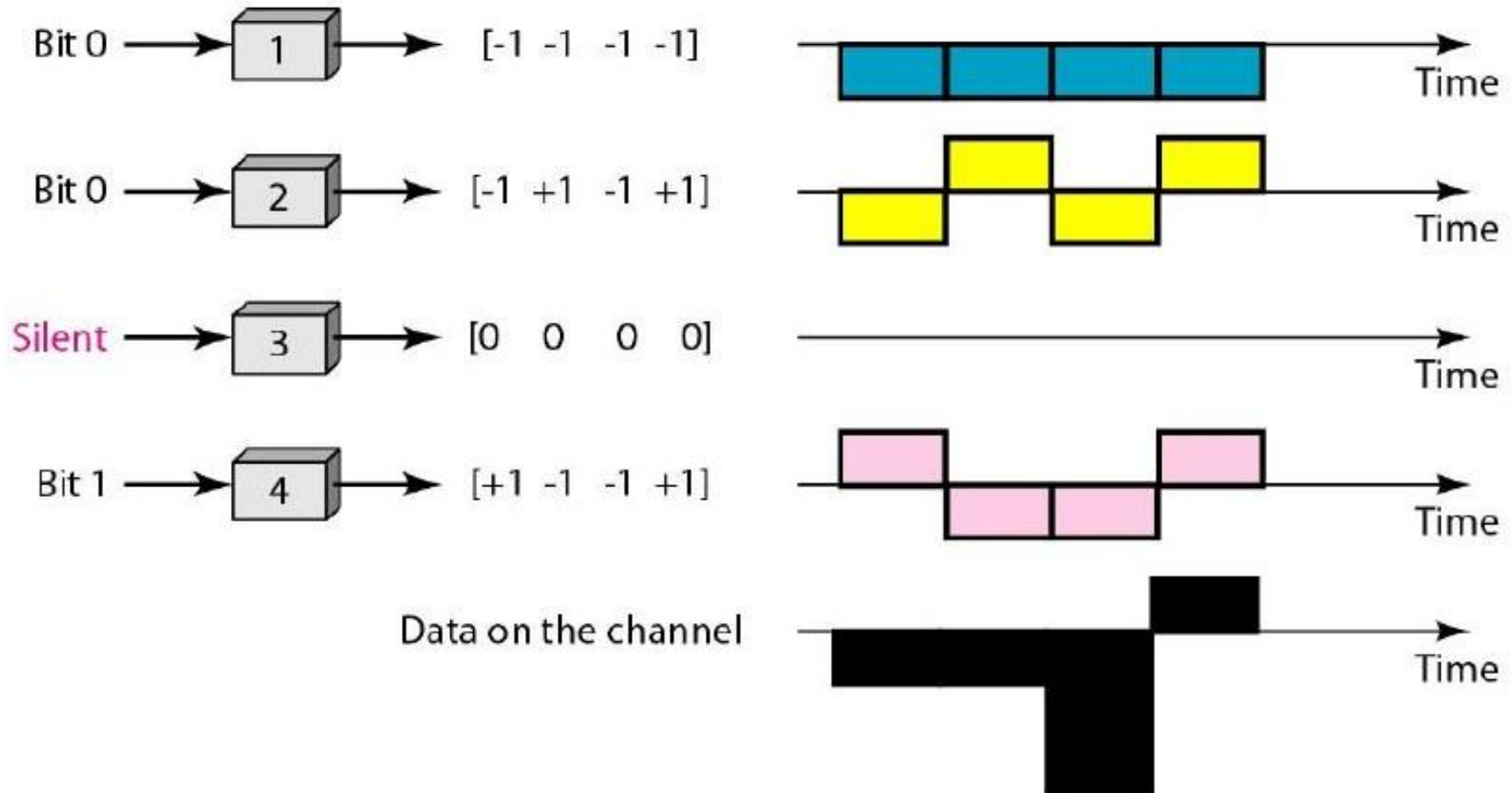


Figure 12.29 *General rule and examples of creating Walsh tables*

$$W_1 = \begin{bmatrix} +1 \end{bmatrix} \qquad W_{2N} = \begin{bmatrix} W_N & W_N \\ W_N & \overline{W_N} \end{bmatrix}$$

a. Two basic rules

$$W_1 = \begin{bmatrix} +1 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix}$$

$$W_4 = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

b. Generation of W_1 , W_2 , and W_4



Note

The number of sequences in a Walsh table needs to be $N = 2^m$.



Example 12.6

Find the chips for a network with

a. Two stations

b. Four stations

Solution

We can use the rows of W_2 and W_4 in Figure 12.29:

a. For a two-station network, we have

$$[+1 \ +1] \text{ and } [+1 \ -1].$$

b. For a four-station network we have

$$[+1 \ +1 \ +1 \ +1], [+1 \ -1 \ +1 \ -1], \\ [+1 \ +1 \ -1 \ -1], \text{ and } [+1 \ -1 \ -1 \ +1].$$



Example 12.7

What is the number of sequences if we have 90 stations in our network?

Solution

The number of sequences needs to be 2^m . We need to choose $m = 7$ and $N = 2^7$ or 128. We can then use 90 of the sequences as the chips.

Example 12.8

Prove that a receiving station can get the data sent by a specific sender if it multiplies the entire data on the channel by the sender's chip code and then divides it by the number of stations.

Solution

Let us prove this for the first station, using our previous four-station example. We can say that the data on the channel

$$D = (d_1 \square c_1 + d_2 \square c_2 + d_3 \square c_3 + d_4 \square c_4).$$

The receiver which wants to get the data sent by station 1 multiplies these data by c_1 .

Example 12.8 (continued)

$$\begin{aligned} D \cdot c_1 &= (d_1 \cdot c_1 + d_2 \cdot c_2 + d_3 \cdot c_3 + d_4 \cdot c_4) \cdot c_1 \\ &= d_1 \cdot c_1 \cdot c_1 + d_2 \cdot c_2 \cdot c_1 + d_3 \cdot c_3 \cdot c_1 + d_4 \cdot c_4 \cdot c_1 \\ &= d_1 \times N + d_2 \times 0 + d_3 \times 0 + d_4 \times 0 \\ &= d_1 \times N \end{aligned}$$

When we divide the result by N , we get d_1 .